

RÉPUBLIQUE DU CAMEROUN

PAIX - TRAVAIL - PATRIE

UNIVERSITÉ DE DSCHANG

ÉCOLE DOCTORALE



REPUBLIC OF CAMEROON

PEACE - WORK - FATHERLAND

UNIVERSITY OF DSCHANG

POST GRADUATE SCHOOL

DSCHANG SCHOOL OF SCIENCES AND TECHNOLOGY

UNITÉ DE RECHERCHE EN INFORMATIQUE
FONDAMENTALE, INGÉNIERIE ET APPLICATIONS (URIFIA)

**PRÉDICTION DES CHARGES DE TRAVAIL ET
ATTÉNUATION DU COLD START DANS LES
ARCHITECTURES FAAS: APPROCHES BASÉES SUR LE DEEP
LEARNING**

**Mémoire rédigé en vue de l'obtention du diplôme de
Master en Informatique**

Option : Informatique fondamentale

Spécialité : Intelligence artificielle

Par

FAYAM MBALA MOUEN Alexandre Savi

Matricule : CM-UDS-20SCI751

Licence en informatique fondamentale

Sous la direction de

Pr KENGNE TCHENDJI Vianney
Maître de Conférences

Année académique : 2023/2024

CERTIFICATION

Je soussigné, FAYAM MBALA MOUEN Alexandre Savi, étudiant en Informatique Fondamentale, matricule CM-UDS-20SCI1751, option Intelligence Artificielle, Département de Mathématiques et Informatique, Faculté des Sciences de l'Université de Dschang, certifie que ce mémoire intitulé : **“Prédiction des charges de travail et atténuation du cold start dans les architectures FaaS : approches basées sur le deep learning”** réalisé à l'Unité de Recherche en Informatique Fondamentale, Ingénierie et Applications (URIFIA) sous la supervision de Pr KENGNE TCHENDJI Vianney, Maître de Conférences à l'Université de Dschang (Faculté des Sciences), est le fruit de mes propres travaux de recherche. Ce mémoire est authentique et n'a jamais été présenté pour l'obtention d'un Diplôme Universitaire ou d'un Grade.

Ce mémoire est authentique et n'a jamais été présenté pour l'obtention d'un Diplôme Universitaire ou d'un Grade.

AUTEUR

FAYAM MBALA MOUEN Alexandre Savi

ENCADREUR

Pr KENGNE TCHENDJI Vianney

(Maître de Conférences)

CHEF DE DÉPARTEMENT

Pr NKENLIFACK Marcellin

(Maître de Conférences)

DÉDICACE

À mes parents NTSAMA ESSENGUÉ Salomé Chantal *et* FAYAM Victor.

REMERCIEMENTS

Les réalisations présentées dans ce rapport ne seraient pas possibles sans Dieu et l'assistance précieuse de nombreuses personnes. Je tiens particulièrement à remercier :

- Pr KENGNE TCHENDJI Vianney, mon encadreur, pour sa patience inestimable et ses retours. Il m'a toujours soutenu et guidé dans la bonne direction. Sa rigueur méthodologique, ses critiques et suggestions m'ont été d'un apport considérable dans la réalisation de ce travail ;
- Le corps enseignant du Département de Mathématiques et d'Informatique, qui m'a enseigné les fondamentaux me permettant d'approfondir mes connaissances ;
- Les membres du jury de ce rapport, pour l'immense honneur qu'ils m'accordent en prenant le temps d'apprécier ce travail ;
- Dr LACMOU ZEUTOUO Jerry pour les nombreux échanges que nous avons eus concernant ce travail. Ses nombreuses remarques, conseils et suggestions pertinentes m'ont été très utiles ;
- Mon frère FAYAM ESSENGUÉ Louis Narcisse et mes soeurs YAMBASSA FAYAM Julienne et FAYAM PELEH Jasmine Lynda Epse EBAMA. Je serais négligent de ne pas les mentionner ainsi que tous les autres membres de la famille pour leur soutien physique, émotionnel, moral et financier, quelles que soient les circonstances. Ils ont eu la perspicacité de me rappeler mes objectifs lorsque je me laissais aller à la facilité ;
- Mes aînés académiques pour leurs précieux conseils et leur aide ;
- Mes camarades de classe pour les nombreux échanges que nous avons eus afin d'améliorer ce travail ;
- Et enfin, ceux qui ont contribué de près ou de loin à ce travail.

TABLE DES MATIÈRES

Dédicace	i
Remerciements	ii
Table des matières	iii
Liste des acronymes	vi
Liste des tableaux	viii
Table des figures	ix
Résumé	x
Abstract	xii
Introduction générale	1
Contexte de la recherche	1
Problématique étudiée	2
Objectifs	3
Contributions	3
Organisation du mémoire	4
Chapitre I ► Architecture FaaS et apprentissage automatique	6
I.1 - Introduction	6
I.2 - Concepts de base du cloud computing	7
I.2.1 - Définitions et caractéristiques du cloud computing	7
I.2.2 - Modèles de déploiement	9
I.2.3 - Modèles de services	11
I.2.4 - Virtualisation et conteneurisation	14
I.2.4.1 - Virtualisation	14
I.2.4.2 - Conteneurisation	15
I.2.5 - Avantages et inconvénients du cloud computing	16
I.2.5.1 - Avantages du cloud computing	16
I.2.5.2 - Inconvénients du cloud computing	16
I.3 - Architecture FaaS	17
I.3.1 - Principes fondamentaux de l'architecture FaaS	17
I.3.2 - Plateformes serverless	18

I.3.3 - Bonnes pratiques pour le développement serverless	21
I.3.4 - Cas d'utilisation	22
I.4 - Généralités sur le machine learning	23
I.4.1 - Techniques de machine learning	24
I.4.1.1 - Apprentissage supervisé	24
I.4.1.2 - Apprentissage non supervisé	26
I.4.1.3 - Apprentissage semi-supervisé	27
I.4.1.4 - Apprentissage par renforcement	27
I.4.2 - Deep learning	28
I.4.2.1 - Convolutional neural networks	29
I.4.2.2 - Long short-term memory	30
I.4.2.3 - Transformers	31
I.4.3 - Applications du machine learning	33
I.5 - Conclusion	34
Chapitre II ► État de l'art sur la prédiction des charges de travail et l'atténuation du Cold Start dans les architectures FaaS	35
II.1 - Introduction	35
II.2 - Caractérisation des charges de travail et cold start	36
II.2.1 - Charges de travail	36
II.2.2 - Cold start	38
II.3 - Techniques de prédiction des charges de travail	42
II.3.1 - Solution de Nguyen et al. [1]	42
II.3.2 - Solution de Nelvin et al. [2]	45
II.4 - Stratégies d'atténuation du cold start	47
II.4.1 - Techniques de réduction du délai de cold start	47
II.4.2 - Techniques de réduction de la fréquence de cold start	48
II.4.3 - Quelques approches basées sur le deep learning pour l'atténuation du cold start	50
II.4.3.1 - Solution de Kumari et al. [3]	50
II.4.3.2 - Solution de Vahidinia et al. [4]	53
II.5 - Conclusion	56
Chapitre III ► Contribution à la prédiction des charges de travail dans les architectures FaaS	58
III.1 - Introduction	58
III.2 - Méthodologie proposée : modèle hybride basé sur les CNN-LSTM	59
III.3 - Architecture du modèle de prédiction	62

III.4 - Évaluation des performances du modèle	64
III.4.1 - Données d'entraînement	64
III.4.2 - Cadre expérimental	65
III.4.3 - Analyses des résultats	66
III.4.4 - Discussion	66
III.5 - Conclusion	67
 <i>Chapitre IV</i> ► Contribution à l'atténuation du Cold Start dans les archi- tectures FaaS	 68
IV.1 - Introduction	68
IV.2 - Méthodologie proposée : modèle basé sur les Transformers	69
IV.3 - Architecture du modèle d'atténuation	70
IV.4 - Évaluation des performances du modèle	72
IV.4.1 - Données d'entraînement	72
IV.4.2 - Cadre expérimental	76
IV.4.3 - Analyses des résultats	77
IV.4.3.1 - Résultats pour le délai de cold start	77
IV.4.3.2 - Résultats pour la fréquence de cold start	81
IV.4.4 - Discussions	85
IV.5 - Conclusion	86
 Conclusion générale	 88
Problématique étudiée	88
Analyse critique des résultats obtenus	88
Perspectives	89
 Bibliographie	 91

LISTE DES ACRONYMES

API	Application Programming Interface ;
ARIMA	Autoregressive Integrated Moving Average ;
BaaS	Backend as a Service ;
CC	Cloud Computing ;
CNCF	Cloud Native Computing Foundation ;
CNN	Convolutional Neural Network ;
CPU	Central Processing Unit ;
CSP	Cloud Service Provider ;
DBaaS	Database as a Service ;
DBSCAN	Density-Based Spatial Clustering of Applications with Noise ;
DNN	Deep Neural Network ;
FaaS	Function as a Service ;
HTTP	Hypertext Transfer Protocol ;
IA	Intelligence Artificielle ;
IaaS	Infrastructure as a Service ;
ID	Identifiant ;
IoT	Internet of Things ;
KB	Kilobyte ;
LSTM	Long Short-Term Memory ;
MAPE	Mean Absolute Percentage Error ;
ML	Machine Learning ;
MSE	Mean Squared Error ;
NIST	National Institute of Standards and Technology ;
NLP	Natural Language Processing ;
PaaS	Platform as a Service ;
R ² Score	R-squared Score ;
ReLU	Rectified Linear Unit ;
RL	Reinforcement Learning ;
RMSE	Root Mean Squared Error ;
RNN	Recurrent Neural Network ;

SaaS	Software as a Service ;
SARIMA	Seasonal ARIMA ;
SDK	Software Development Kit ;
SLA	Service Level Agreement ;
sMAPE	Symmetric Mean Absolute Percentage Error ;
TCN	Temporal Convolutional Network ;
VAR	Vector Autoregression ;
VM	Virtual Machine ;

LISTE DES TABLEAUX

I - Résultat des performances : module A	45
II - Résultat des performances : module B	45
III - Résultat des performances : nombre de cold start	52
IV - Détails techniques de l'architecture CNN-LSTM	64
V - Résultats des performances CNN-LSTM vs LSTM	66
VI - Résultats des performances - délai de cold start : dataset (fréquence : heure)	81
VII - Résultats des performances - délai de cold start : dataset (fréquence : minute)	82
VIII - Comparaison des variations de la fenêtre de conteneur inactif et du nombre de cold starts sur OpenWhisk avec et sans modèle de Transfor- mers associé	83
IX - Correspondance entre les fonctions et les ensembles de données Azure	86

TABLE DES FIGURES

1 - Architecture de cloud computing	8
2 - Différentes couches de cloud computing [5]	9
3 - Modèles de déploiement cloud	10
4 - Les trois grands modèles de services cloud	12
5 - Service PaaS : Heroku	13
6 - Relation entre l'intelligence artificielle, l'apprentissage automatique et l'apprentissage profond	24
7 - Classification SVM	25
8 - Structure d'un neurone [6]	28
9 - Structure d'un réseau de neurones convolutionnels (CNN) [7]	29
10 - Module répétitif dans un LSTM composé de quatre couches interactives	31
11 - Architecture des Transformers [8]	32
12 - Processus global d'appel d'une fonction dans les architectures FaaS . .	39
13 - Flux d'exécution FaaS [4]	39
14 - Allocation des ressources d'une fonction	59
15 - Modèle hybride CNN-LSTM proposé	62
16 - Dataset d'entraînement pour le modèle CNN-LSTM	65
17 - Représentation des entrées de l'encodeur et du décodeur	71
18 - Processus global des traitements des datasets à passer en entraînement au modèle Transformers	75
19 - Approche proposée : modèle basé sur les Transformers pour atténuer le cold start	78
20 - Six schémas d'invocations (fonctions) - fréquence : minutes	78
21 - Six schémas d'invocations (fonctions) - fréquence : heure	79
22 - Variations de la fenêtre de conteneur inactif	82

RÉSUMÉ

Le cloud computing est devenu un élément essentiel et incontournable. Son adoption généralisée par les organisations est largement motivée par sa flexibilité, sa capacité d'adaptation et son potentiel à réduire les coûts opérationnels tout en améliorant la disponibilité et la sécurité des données. En offrant des services à la demande, une élasticité accrue et une gestion simplifiée des ressources informatiques, le cloud computing se positionne comme un vecteur clé de transformation digitale pour les organisations de toutes tailles. Parmi ses évolutions les plus notables, les architectures serverless, et en particulier le modèle Function as a Service (FaaS), attirent de nombreux développeurs grâce à leur capacité à libérer ces derniers de la gestion infrastructurelle. Cela permet aux développeurs de se concentrer entièrement sur le développement d'applications. Cependant, un défi majeur persiste au sein de ces architectures FaaS : le cold start. Le cold start se produit lorsqu'une fonction doit être initialisée à partir de zéro, entraînant des retards indésirables qui peuvent affecter les performances des applications. De plus, la gestion efficace des ressources et la prévision précise des charges de travail sont essentielles pour maintenir des performances optimales tout en minimisant les coûts opérationnels. Une mauvaise gestion des ressources peut entraîner un gaspillage de mémoire ou, au contraire, une pénurie qui affecte la performance globale du système. C'est dans cette optique que notre recherche propose une approche efficace basée sur le deep learning pour aborder ces problèmes complexes. Nous avons développé, étudié et évalué en profondeur deux modèles principaux : un modèle hybride combinant des réseaux de neurones convolutifs et des réseaux de neurones à mémoire à court et long terme ; ainsi qu'un modèle basé sur les Transformers. Les résultats obtenus se sont montrés prometteurs. Les modèles proposés ont significativement amélioré la précision des prédictions de charges de travail avec une différence de 2,86 dans le score RMSE comparé à la revue de littérature, et réduit de manière notable les temps de réponse des fonctions FaaS sur le dataset public fourni par Azure, avec des réductions de cold start allant jusqu'à 79%. Ils permettent ainsi de gérer plus efficacement les ressources et à optimiser les performances des applications serverless comparé aux solutions proposées dans la littérature.

Mots clés : Cloud computing, Serverless, FaaS, Deep learning, CNN, LSTM, Transformers, Cold start, Prédiction de charges de travail.

**PREDICTION OF WORKLOADS AND
MITIGATION OF COLD START IN FAAS
ARCHITECTURES : DEEP
LEARNING-BASED APPROACHES**

ABSTRACT

Cloud computing has become an essential and indispensable element. Its widespread adoption by organizations is largely driven by its flexibility, adaptability, and potential to reduce operational costs while improving data availability and security. By offering on-demand services, increased elasticity, and simplified management of computing resources, cloud computing positions itself as a key driver of digital transformation for organizations of all sizes. Among its most notable evolutions, serverless architectures, particularly the Function as a Service (FaaS) model, attract many developers due to their ability to free them from infrastructural management. This allows developers to focus entirely on application development. However, a major challenge persists within these FaaS architectures : the cold start. The cold start occurs when a function needs to be initialized from scratch, causing undesirable delays that can affect application performance. Furthermore, the efficient management of resources and accurate workload forecasting are essential to maintain optimal performance while minimizing operational costs. Poor resource management can lead to memory waste or, conversely, a shortage that affects the overall system performance. It is with this in mind that our research proposes an effective approach based on deep learning to address these complex issues. We have developed, studied, and thoroughly evaluated two main models : a hybrid model combining convolutional neural networks and long short-term memory networks, and a Transformers-based model. The results obtained have been promising. The proposed models have significantly improved the accuracy of workload predictions with a difference of 2.86 in the RMSE score compared to the literature review and notably reduced the response times of FaaS functions on the public dataset provided by Azure, with cold start reductions of up to 79%. This enables more efficient resource management and optimization of serverless application performance compared to the solutions proposed in the literature.

Keywords : Cloud computing, Serverless, FaaS, Deep learning, CNN, LSTM, Transformers, Cold start, Workload prediction.

INTRODUCTION GÉNÉRALE

CONTENU

Contexte de la recherche	1
Problématique étudiée	2
Objectifs	3
Contributions	3
Organisation du mémoire	4

Contexte de la recherche

Avec l'essor rapide du cloud computing, les architectures serverless ont gagné en popularité grâce à leur capacité à abstraire la gestion de l'infrastructure, permettant aux développeurs de se concentrer uniquement sur le code et les fonctionnalités de leurs applications [9]. Le cloud computing est une solution révolutionnaire, car grâce à ses mécanismes de virtualisation, il met à disposition des entreprises ou des individus des ressources informatiques (serveurs, stockage, réseaux, logiciels, etc.) accessibles via internet. Il existe de nombreux services dans le cloud computing adaptés à divers cas d'utilisation particuliers. Parmi ces nombreux services, le Function as a Service (FaaS) en français Fonction en tant que Service, s'est imposé comme une solution innovante pour le déploiement et l'exécution des fonctions en réponse à des événements [10]. Ce modèle se distingue par sa flexibilité, sa scalabilité automatique et sa facturation à l'usage, réduisant ainsi les coûts et les différents efforts de gestion.

L'intelligence artificielle (IA), est une branche de l'informatique très tendance à ce jour, qui permet de concevoir des systèmes capables de prendre des décisions intelligentes de manière autonome (sans intervention humaine) [11]. L'IA est une technologie qui aide les organisations à gagner du temps, à réduire les coûts et à améliorer l'efficacité opérationnelle. L'intégration de l'IA dans le cloud computing

améliore l'efficacité opérationnelle de plusieurs manières. Les algorithmes d'IA intégrés à l'architecture cloud automatisent les processus répétitifs, rationalisant les charges de travail et réduisant le risque d'erreur humaine.

En dépit des multiples avantages qu'il offre, le modèle FaaS présente tout de même plusieurs challenges [12]. L'un des principaux challenges est lié au temps de réponse des fonctions, ce qui peut entraîner des cold starts. Un cold start se produit lorsque les fonctions doivent être initialisées à partir de zéro et chargées dans un conteneur [4]. Cette initialisation entraîne des délais qui peuvent impacter la performance des applications. Un cold start se produit notamment lorsqu'il y a une absence prolongée d'invocations de la fonction, nécessitant le démarrage de nouvelles instances à partir d'un état inactif, ce qui peut prendre plusieurs secondes.

Une gestion efficace des ressources et l'adaptation aux variations des charges de travail s'avèrent également essentielles pour maximiser l'efficacité et minimiser les coûts dans les environnements FaaS [2]. Les variations imprévisibles des charges de travail posent un défi supplémentaire en termes de gestion dynamique des ressources, où une allocation proactive et optimisée des ressources est nécessaire pour maintenir des performances optimales [13].

Problématique étudiée

Compte tenu des comportements très variables et imprévisibles des charges de travail dans ce type d'architecture, afin d'y améliorer les performances et l'efficacité on fait face à des problèmes de :

- **Gestion dynamique des ressources :** Une allocation proactive et optimisée des ressources est nécessaire pour maintenir des performances optimales face à ces variations. Il est crucial d'adapter rapidement l'allocation des ressources pour éviter des situations où les ressources sont soit insuffisantes (affectant non seulement la performance globale du système mais aussi augmentant les coûts opérationnels), soit surdimensionnées (gaspillage considérable qui augmenterait inutilement les coûts).
- **Les temps de réponse et cold start :** le cold start est un problème crucial dans les environnements FaaS. Il se produit lorsque des fonctions doivent être initialisées à partir de zéro et chargées dans un conteneur, entraînant des délais significatifs. Ce délai, souvent très notable dans des scénarios où la réactivité est essentielle, peut compromettre l'efficacité des applications. Une initialisation rapide des fonctions est nécessaire pour minimiser l'impact des cold starts sur les temps de réponse.

Objectifs

L'objectif général de notre recherche est la réduction du temps de réponse des fonctions dans les architectures FaaS, c'est à dire, atténuer considérablement le cold start, tout en améliorant la gestion des ressources, le tout en exploitant les avancées de l'intelligence artificielle, plus précisément du deep learning.

Objectifs spécifiques

- Optimiser la gestion des ressources en minimisant le gaspillage et en assurant une utilisation efficace des conteneurs à l'aide d'un modèle de deep learning, tout en tenant compte des variations de charges de travail pour améliorer la performance globale de la plateforme FaaS.
- Évaluer les performances de ce modèle en termes de précision des prédictions en comparaison à la revue de littérature.
- Concevoir et mettre en œuvre un modèle de deep learning, capable de prendre en compte l'historique des invocations de fonctions pour prédire avec précision les futures invocations, facilitant ainsi une allocation proactive des conteneurs.
- Évaluer la performance de ce modèle en termes de précision des prédictions et de réduction du cold start basé sur des données réelles en comparaison à la revue de littérature, reflétant la diversité et la dynamique des plateformes FaaS, afin de garantir sa pertinence, sa fiabilité et son applicabilité dans des scénarios pratiques et variés.

Contributions

Notre recherche propose plusieurs contributions significatives dans le domaine des architectures FaaS, en intégrant des techniques de deep learning pour améliorer la gestion des ressources et réduire le cold start des fonctions ou applications :

1. Développement d'un modèle de prédiction hybride pour anticiper les charges de travail futures : ce modèle combine des réseaux neuronaux convolutifs (CNN) et des réseaux de mémoire à court et long terme (LSTM). Cette approche permet de capturer des motifs complexes dans les données d'invocation, offrant ainsi des prédictions plus précises. Par rapport aux approches existantes, notre modèle présente une réduction significative de l'erreur quadratique moyenne (RMSE), illustrant une amélioration de la précision des prédictions.

2. Développement d'un modèle basé sur les Transformers pour atténuer le cold start : pour permettre une gestion plus efficace des fonctions afin de réduire les délais associés à leur démarrage et les fréquences qui peuvent survenir, en fonction des prévisions d'utilisation. Les résultats montrent une amélioration notable des prévisions, confirmée par de nombreuses métriques d'évaluation, par rapport aux approches existantes.

Les performances des modèles proposés sont évaluées à travers des simulations sur la plateforme FaaS OpenWhisk et des tests basés sur des jeux de données réels. Ces évaluations incluent des métriques telles que la précision des prédictions, la réduction du délai et de fréquence de cold start. Les résultats obtenus sont comparés à des approches existantes pour démontrer l'efficacité, la simplicité et l'avantage des solutions proposées.

Organisation du mémoire

Notre travail est structuré de la manière suivante :

Chapitre I. Architecture FaaS et apprentissage automatique : présentation détaillée du cloud computing et des architectures FaaS. Introduction aux concepts de l'intelligence artificielle et du machine learning appliqués aux environnements FaaS.

Chapitre II. État de l'art sur la prédiction des charges de travail et l'atténuation du Cold Start dans les architectures FaaS : analyse des travaux existants sur la prédiction des charges de travail et les stratégies d'atténuation du cold start. Évaluation des différentes approches de gestion des ressources dans les architectures FaaS.

Chapitre III. Contribution à la prédiction des charges de travail dans les architectures FaaS : description de la méthodologie utilisée pour développer le modèle de prédiction hybride CNN-LSTM. Présentation des résultats de l'implémentation du modèle, incluant des analyses détaillées des performances.

Chapitre IV. Contribution à l'atténuation du Cold Start dans les architectures FaaS : développement et implémentation des stratégies basées sur les Transformers ; analyse des résultats obtenus et comparaison avec des techniques existantes.

Conclusion générale : récapitulatif des principaux résultats obtenus, analyse de l'impact des contributions sur le domaine des architectures FaaS, et propositions des pistes pour des recherches futures.

I

CHAPITRE

ARCHITECTURE FAAS ET APPRENTISSAGE AUTOMATIQUE

CONTENU

I.1 - Introduction	6
I.2 - Concepts de base du cloud computing	7
I.3 - Architecture FaaS	17
I.4 - Généralités sur le machine learning	23
I.5 - Conclusion	34

I.1. Introduction

La notion de virtualisation et le besoin croissant de ressources dynamiques sur demande pour le stockage, le calcul, le traitement et la mise à l'échelle ont été les moteurs de la révolution du cloud computing dans le domaine de l'informatique. Les avancées technologiques appellent à un nouveau paradigme informatique permettant la fourniture de services sans nécessiter leur installation localement et à moindre coût. Le cloud computing (CC) ainsi, propose un modèle où les services sont offerts via internet de manière élastique et à la demande, avec une tarification basée sur l'utilisation des ressources. Son objectif est de fournir des services sécurisés, de haute qualité, évolutifs, réactifs, rentables et automatisés. Le CC peut de ce fait améliorer les performances des entreprises tout en maîtrisant les coûts liés à la fourniture de ressources informatiques. La virtualisation joue un rôle central dans ce modèle, permettant aux fournisseurs de services cloud de créer des environnements virtuels à partir de ressources physiques. Par ailleurs, l'apprentissage automatique, branche de l'intelligence artificielle visant à doter les machines de capacités d'apprentissage à partir de données via des modèles mathématiques,

prend une place prépondérante, notamment dans le domaine du cloud computing. Le présent chapitre a pour objectifs de clarifier le concept de CC en exposant les principes de la virtualisation ainsi que les caractéristiques et concepts qui y sont associés, de présenter le fonctionnement des architectures serverless, et enfin de fournir un état de l'apprentissage automatique, décrivant ses différentes techniques et applications dans le contexte du cloud computing.

I.2. Concepts de base du cloud computing

I.2.1. Définitions et caractéristiques du cloud computing

Le CC demeure sujet à diverses interprétations, mais se définit essentiellement par son modèle de partage et d'accès aux ressources informatiques à grande échelle. Une étude menée par Vaquero et al. [14], qui ont examiné près de 20 définitions différentes du cloud a ressorti que le CC est une technologie fondée sur la virtualisation, qui implique le partage de ressources informatiques telles que les réseaux, les serveurs, le stockage de bases de données, la puissance de calcul, les applications, etc., configurables au sein de centres de données. Le cloud offre également aux clients des services instantanés, élastiques et à la demande, facturés en fonction de leur utilisation.

Selon le NIST, le CC se caractérise par cinq aspects principaux à savoir [15] :

- Accès libre-service et à la demande : les services sont mis à disposition automatiquement pour tous les clients, sans nécessiter une intervention humaine directe.
- Accès omniprésent au réseau : les capacités du cloud sont accessibles via des mécanismes compatibles avec diverses plates-formes clientes.
- Mise en commun des ressources : les ressources sont partagées entre les différents clients, avec une allocation dynamique selon la demande.
- Indépendance des emplacements : les clients n'ont généralement pas de contrôle direct sur l'emplacement physique des ressources fournies.
- Élasticité rapide : possibilité d'ajuster rapidement l'allocation des ressources en fonction des besoins des utilisateurs.

Une définition complémentaire précise que le cloud est un pool de ressources informatiques virtualisées [16]. Ainsi, le CC est un modèle informatique qui exploite ces ressources virtualisées et partage leurs informations, logiciels et diverses autres ressources avec différents appareils via internet pour fournir une puissance de calcul maximale. L'architecture de base du CC est illustrée dans la figure 1. Une

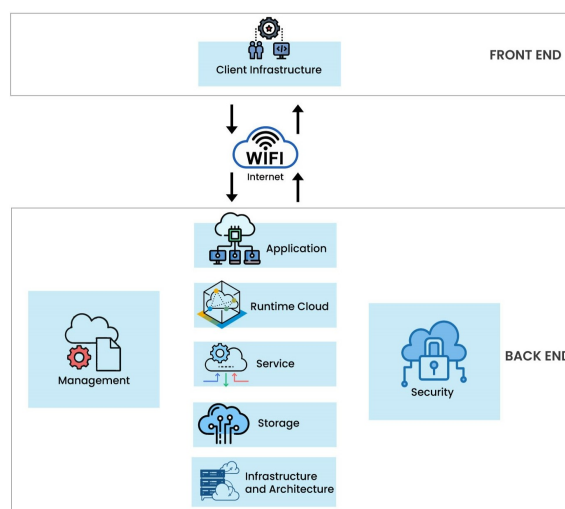


FIGURE 1 – *Architecture de cloud computing*

application cloud typique est composée de deux parties principales : l'avant-plan et l'arrière-plan.

Lorsqu'un utilisateur souhaite utiliser un service cloud, il interagit d'abord avec l'avant-plan de l'application, qui est la partie visible de l'iceberg. Ce plan est accessible via divers appareils tels que des smartphones, des ordinateurs portables, etc. À partir de là, un utilisateur pourra envoyer une demande de service vers l'arrière-plan, où toutes les opérations complexes sont effectuées. L'arrière-plan, correspond à la partie cachée du système, où se trouvent toutes les ressources matérielles et logicielles nécessaires au bon fonctionnement du service cloud. Cela comprend des éléments tels que le stockage de données, les bases de données, ainsi que les algorithmes de traitement des demandes. Une fois la demande du client traitée, la réponse est renvoyée à l'avant-plan pour être présentée à l'utilisateur. Dans l'architecture du cloud computing, le cloud peut être divisé en six couches distinctes, comme illustré dans la figure 2.

Au cœur du fonctionnement du cloud se trouve une structure en couches qui comprend différentes strates, chacune jouant un rôle spécifique dans la fourniture de services informatiques. La couche la plus basse du cloud est constituée de composants matériels tels que des câbles, des prises et des commutateurs. Ces éléments forment l'infrastructure physique sur laquelle repose l'ensemble du système. Juste au-dessus se trouve la couche des serveurs. Le cloud abrite une multitude de serveurs, qui sont installés et maintenus en fonction des besoins spécifiques du fournisseur de services cloud. Ces serveurs varient en taille et en capacité, allant des serveurs d'authentification simples dotés de ressources limitées aux grands serveurs de traitement de données équipés de milliers de cœurs de processeurs. La

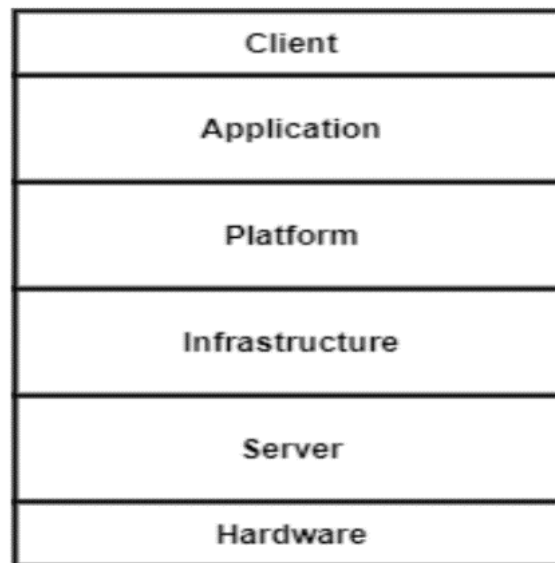


FIGURE 2 – Différentes couches de cloud computing [5]

couche suivante est celle de l'infrastructure, où les ressources disponibles sur un seul serveur ou réparties entre plusieurs serveurs sont combinées à l'aide de techniques de virtualisation. Ces ensembles de ressources virtualisées sont désignés sous le nom d'infrastructures cloud. Chaque infrastructure peut contenir une ou plusieurs plateformes. Les couches de plateformes entrent ensuite en jeu. C'est à ce niveau que sont hébergées une ou plusieurs applications. La couche d'application, quant à elle, regroupe toutes les applications hébergées dans le cloud, offrant ainsi une gamme diversifiée de services aux utilisateurs. Enfin, la couche client fournit des interfaces conviviales pour permettre aux utilisateurs de se connecter facilement aux applications disponibles sur la couche d'application, garantissant ainsi une expérience utilisateur fluide et intuitive.

1.2.2. Modèles de déploiement

Les modèles de déploiement dans le cloud offrent des solutions adaptées aux multiples besoins des utilisateurs en termes d'accessibilité, de sécurité et de contrôle. Chaque modèle possède ses particularités, avantages et inconvénients, ce qui nécessite ainsi une analyse approfondie avant de choisir celui qui conviendrait le mieux à son organisation. Une représentation des différents modèles de cloud est présentée dans la figure 3.

Cloud public : il est ouvert au grand public via internet, accessible aux particuliers, aux entreprises, aux institutions académiques et gouvernementales. Les services cloud publics sont généralement proposés par des prestataires tiers tels qu'Amazon Web Services (AWS), Google Cloud Platform (GCP),

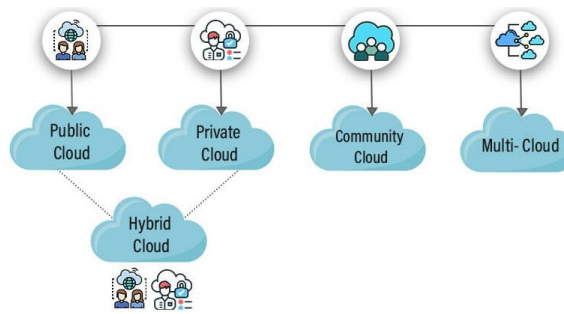


FIGURE 3 – Modèles de déploiement cloud

Microsoft Azure et Salesforce. Malgré son accessibilité et son évolutivité, le cloud public est sujet à des préoccupations concernant la sécurité, car les données ici, sont stockées sur des serveurs partagés. Il est caractérisé par une infrastructure partagée, un modèle de paiement à l'utilisation et des économies d'échelle.

Ses avantages comprennent une disponibilité élevée, une maintenance minimale et des coûts réduits, tandis que ses inconvénients incluent des préoccupations de sécurité et un manque de contrôle sur les systèmes hébergeant les applications métier.

Cloud privé : il est exclusivement dédié à une seule organisation et géré en interne ou par un tiers. Eucalyptus, OpenNebula et OpenStack sont des exemples de solutions pour créer des clouds privés. On distingue deux types de cloud privé : interne, où toutes les ressources sont gérées en interne, et externe, où les ressources sont conçues pour répondre aux besoins de l'entreprise mais en étant hébergées et gérées par un fournisseur tiers. Le cloud privé fournit un niveau élevé de contrôle, de personnalisation et de sécurité des données, mais il est à noter que sa mise en œuvre nécessite des investissements financiers importants pour l'infrastructure et la maintenance. Il est caractérisé par une infrastructure spécifique à l'organisation, une sécurité renforcée et une conformité réglementaire.

Ses avantages comprennent un contrôle complet, une fiabilité accrue et une adaptation aux besoins spécifiques de l'entreprise, tandis que ses inconvénients comprennent des coûts initiaux élevés et une maintenance constante.

Cloud communautaire : il implique le partage d'infrastructures cloud entre plusieurs organisations ayant des préoccupations, des politiques et des exigences similaires. Ces organisations peuvent gérer de façon conjointe une infrastructure cloud pour répondre à leurs besoins communs, ce qui permettra ainsi de réduire les coûts de maintenance. Les clouds communautaires peuvent être

hébergés en interne ou par un tiers et offrent une meilleure sécurité que le cloud public en raison de sa concentration sur des préoccupations communes telles que : la gestion des ressources, la sécurité des données et le déploiement d'applications. Toutefois, la gestion de la bande passante et de la capacité de stockage limitées peut souvent poser des défis supplémentaires.

Cloud hybride : il combine des environnements de cloud public et privé, permettant d'offrir ainsi beaucoup plus de flexibilité et de possibilités d'optimisation des coûts pour les organisations. Il permet le transfert des charges de travail entre les clouds privés et publics en fonction des besoins, ce qui réduit les coûts par rapport à l'extension du cloud privé. Le cloud hybride est caractérisé par une meilleure sécurité et confidentialité, une évolutivité améliorée et une facilité de portabilité des données et des applications.

Ses avantages comprennent la puissance combinée des clouds publics et privés, une meilleure sécurité que le cloud public employé seul et une flexibilité accrue pour concevoir des solutions personnalisées. Cependant, la gestion d'un cloud hybride peut être complexe et coûteuse, et ses fonctionnalités de sécurité ne sont pas aussi robustes que celles d'un cloud privé.

Multi-cloud : il implique l'utilisation de plusieurs CSP publics ou privés, permettant d'offrir ainsi une redondance et une faible probabilité d'interruption de service. Il permet également aux organisations de choisir les services qui répondent le mieux à leurs besoins spécifiques, de sélectionner les meilleures options de fournisseur en fonction des contrats et de la flexibilité de paiement, et de garantir la disponibilité des données à proximité des clients.

Ses avantages comprennent une meilleure flexibilité de configuration, une disponibilité élevée, une optimisation des coûts et une réduction des risques. Cependant, il peut être complexe à gérer et ainsi nécessiter des compétences spécialisées pour gérer plusieurs fournisseurs et services cloud.

Cette diversité de modèles de déploiement dans le cloud permet aux organisations de choisir celui qui correspondra le mieux à leurs besoins, en prenant en considération les facteurs tels que la sécurité, la flexibilité, les coûts et la conformité réglementaire. Chaque modèle présente ses propres avantages et ses inconvénients, et le choix dépendra des objectifs et des exigences spécifiques de chaque organisation.

1.2.3. Modèles de services

On en distingue une multitude parmi lesquels :

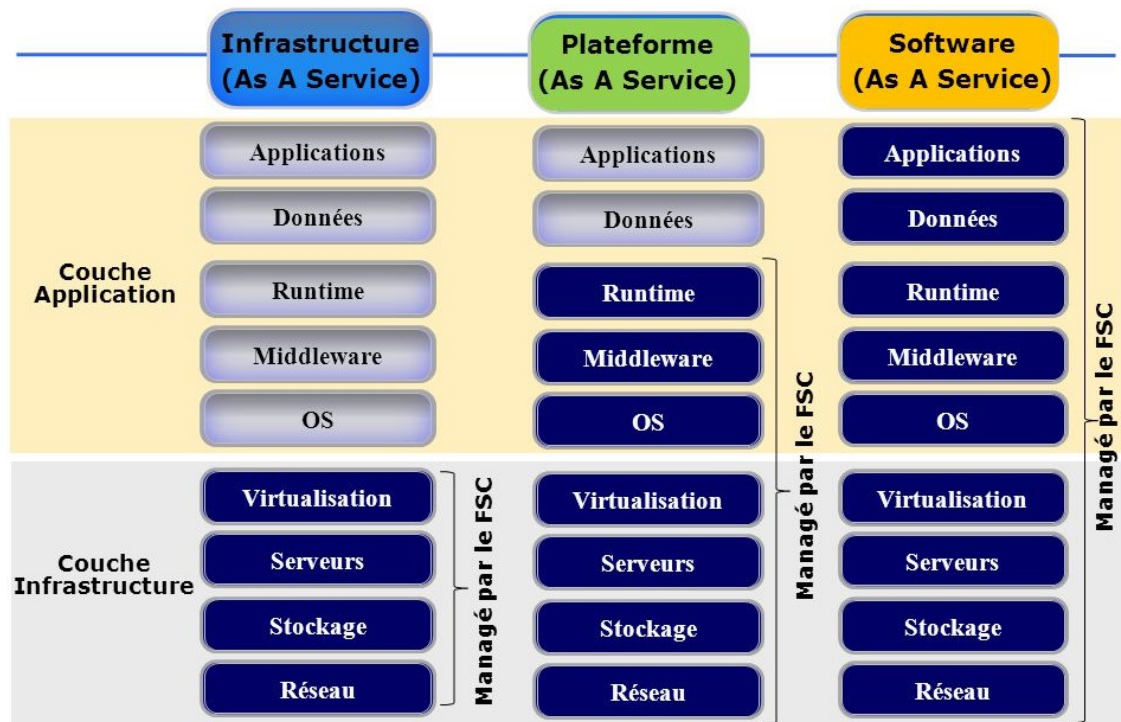


FIGURE 4 – Les trois grands modèles de services cloud

Infrastructure as a Service (IaaS) : il fournit une infrastructure en tant que service, c'est à dire : des serveurs, une puissance de calcul, un réseau, un espace de stockage, une bande passante, etc., pour le déploiement et l'exécution d'applications. Ces services sont fournis sur une base de paiement à l'utilisation sur internet. Les clients de ce service ne paieront que pour les ressources qu'ils ont consommées. Les utilisateurs contrôlent le système d'exploitation, la capacité de stockage et les applications déployées. Amazon EC2 (Amazon Elastic Compute Cloud) est un exemple d'IaaS.

Platform as a Service (PaaS) : il fournit aux développeurs un ensemble d'outils et de langages de programmation pour créer des applications sans se soucier de l'infrastructure sous-jacente. Les développeurs pourront donc se concentrer uniquement sur le développement de leurs applications, tandis que le fournisseur de services va gérer le reste. Cela permettra un déploiement rapide et une scalabilité facile des applications. Google App Engine et Microsoft Azure App Service sont des exemples populaires de PaaS. L'architecture de base du service PaaS est présentée dans la figure 5. Nous pouvons voir le modèle de service du populaire fournisseur de services PaaS, Heroku. Il aide les développeurs avec un écosystème de gestion des conteneurs pour rendre le processus de livraison de logiciel plus centré sur l'application. Les développeurs utilisent tous les outils et fonctionnalités de la plateforme pour créer

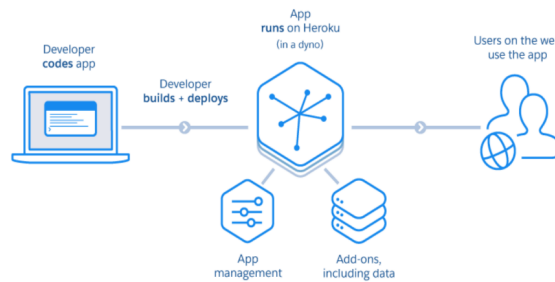


FIGURE 5 – Service PaaS : Heroku

l'application et les services souhaités. La plateforme gère toutes les tâches de maintenance, de version, d'équilibrage de charge et de gestion de packages.

Software as a Service (SaaS) : il est question ici d'un modèle de déploiement d'applications dans lequel un fournisseur loue une application logicielle en tant que service auprès d'un client, à la demande. Il existe de nombreux logiciels qui sont fournis en tant que service tels que : la messagerie en ligne, la gestion de la relation client (CRM), les logiciels de stockage, la gestion des ressources humaines, la comptabilité, Salesforce, etc. La mise en œuvre d'une infrastructure, des serveurs d'hébergement et des systèmes d'exploitation est entièrement fournie par le fournisseur et complètement abstraite pour le client. Toutefois, ces derniers ont la possibilité de configurer ou personnaliser leurs applications en fonction de leurs besoins via une API définie par le fournisseur. Également la mise à jour des applications est une tâche qui incombe au fournisseur.

Function as a Service (FaaS) : ici, les développeurs exécutent un code en réponse à des événements, sans se soucier de l'infrastructure sous-jacente. Ils ne paient que pour le temps d'exécution de la fonction, ce qui rend ce modèle très économique et flexible. AWS Lambda et Azure Functions sont des exemples populaires de FaaS.

Database as a Service (DBaaS) : il permet aux utilisateurs de configurer, d'exploiter et mettre à l'échelle une base de données sans se soucier de l'infrastructure sous-jacente. Les fournisseurs de services gèrent la maintenance, les sauvegardes et les mises à niveau, offrant ainsi une expérience sans tracas pour les utilisateurs. Des exemples de DBaaS sont IBM Cloud Databases, Oracle Autonomous Databases, Google BigQuery, etc.

I.2.4. Virtualisation et conteneurisation

I.2.4.1. Virtualisation

L'introduction de la virtualisation a rendu possible le partage du même matériel pour plusieurs systèmes d'exploitation (SE) connus sous le nom de Machine Virtuelle (VM) [17], offrant la possibilité d'exécuter plusieurs systèmes d'exploitation sur un seul matériel physique. Ce modèle a rapidement évolué pour inclure des techniques plus avancées parmi lesquelles :

- **La virtualisation au niveau du système d'exploitation** : qui est une forme spéciale de virtualisation qui permet l'exécution de plusieurs instances isolées d'espace utilisateur sur le même noyau [16]. Ces conteneurs, ou instances d'espace utilisateur, fonctionnent comme des entités autonomes, partageant le même noyau tout en bénéficiant d'allocations de ressources distinctes. Cette approche offre une flexibilité et une efficacité accrues, mais peut présenter des limites en termes de compatibilité entre les instances invitées et hôtes.
- **La virtualisation du stockage** : qui a révolutionné la manière dont les données sont gérées et réparties dans les environnements informatiques modernes. En consolidant les ressources de stockage physiques et en les présentant comme un seul dispositif harmonieux aux applications, cette technique offre une redondance et une réplication des données, garantissant ainsi la disponibilité et la tolérance aux pannes.
- **La virtualisation du réseau** : qui crée des abstractions des ressources réseau physiques et logicielles pour être fournies par le biais d'une entité administrative centrale. Ces abstractions, appelées réseaux virtuels, rendent le réseau agile, dynamique et efficace, permettant le déploiement d'applications dans des environnements simulés pour évaluer leurs performances.

Applications dans l'informatique moderne La virtualisation trouve des applications dans divers domaines de l'informatique moderne, notamment le cloud computing, le développement d'applications, la gestion des données et la cyber sécurité. En permettant une allocation dynamique des ressources, une isolation des applications et une flexibilité opérationnelle, elle contribue à la création d'infrastructures informatiques agiles et évolutives.

En somme, la virtualisation demeure un pilier essentiel de l'infrastructure informatique moderne, offrant des avantages significatifs en termes d'optimisation des ressources, de tolérance aux pannes et de flexibilité opérationnelle. Son évolution continue promet de façonner l'avenir de l'informatique, en répondant aux

défis croissants de la complexité des systèmes et des exigences changeantes des applications.

1.2.4.2. Conteneurisation

La conteneurisation représente une méthode de déploiement de logiciels où tous les éléments nécessaires à l'exécution d'une application, y compris le code, les bibliothèques et les dépendances, sont encapsulés dans un conteneur isolé. Ce conteneur peut ensuite être déplacé et exécuté de manière cohérente dans divers environnements et infrastructures, indépendamment du système d'exploitation utilisé.

En comparaison, les conteneurs et les VM partagent une similarité dans le fait qu'ils sont tous deux des environnements informatiques encapsulés, mais leurs différences fondamentales résident dans leur scalabilité et leur portabilité.

- Les conteneurs, généralement de taille réduite en mégaoctets, contiennent uniquement les composants nécessaires à l'exécution d'une application spécifique, souvent des fonctions uniques appelées microservices. Étant légers et autonomes, ils peuvent être déplacés entre différents environnements sans difficulté, car ils ne requièrent pas leur propre système d'exploitation. Cette caractéristique de portabilité les rend particulièrement adaptés à des architectures modernes, telles que les applications cloud-native, où la flexibilité et la rapidité de déploiement sont essentielles.
- Les VM sont souvent beaucoup plus volumineuses, en gigaoctets, et intègrent généralement leur propre système d'exploitation. Elles peuvent exécuter simultanément plusieurs fonctions gourmandes en ressources grâce à leur accès à d'importantes ressources. Les machines virtuelles sont utilisées pour isoler des cycles de développement risqués, provisionner des ressources d'infrastructure et exécuter un système d'exploitation dans un autre (comme Unix sous Linux).

Ainsi, les conteneurs sont privilégiés dans des contextes où la légèreté, la scalabilité et la portabilité sont des priorités, comme la création de microservices ou le développement dans le cadre des pratiques DevOps ou CI/CD. Les machines virtuelles quant à elles, sont souvent préférées pour les charges de travail monolithiques, où l'isolement des cycles de développement est crucial, et pour des besoins spécifiques tels que le provisionnement de ressources d'infrastructure et l'exécution de systèmes d'exploitation distincts.

I.2.5. Avantages et inconvénients du cloud computing

Toute évaluation approfondie du CC doit prendre en considération à la fois ses atouts et ses points faibles.

I.2.5.1. Avantages du cloud computing

Le CC offre une multitude d'avantages et de flexibilité aux utilisateurs. En permettant un accès sécurisé à tout moment et depuis n'importe où, il permet de répondre aux besoins d'une société de plus en plus mobile et connectée. Parmi ses nombreux avantages, on peut citer :

- Mises à jour logicielles automatiques : facilite la mise à jour automatique des logiciels, améliorant ainsi la compatibilité entre différents formats de documents et systèmes d'exploitation.
- Collaboration d'équipe simplifiée : permet une collaboration fluide entre des utilisateurs dispersés dans le monde entier, favorisant ainsi le travail sur des projets communs.
- Services à la demande et évolutifs : modèle de paiement à l'utilisation offre des services flexibles et évolutifs, répondant précisément aux besoins des entreprises.
- Disponibilité et performances constantes : permet une provision automatique des ressources pour garantir une disponibilité et des performances constantes, même en cas de pics de charge.
- Réduction des coûts initiaux et de gestion : en éliminant la nécessité d'investissements initiaux et de gestion des ressources, le cloud réduit les coûts et les risques associés à la maintenance d'une infrastructure informatique.
- Élasticité des ressources : offre une capacité de traitement, de stockage et de mise en réseau pratiquement illimitée et élastique, s'adaptant aux besoins changeants des entreprises.
- Infrastructure dynamique : permet une infrastructure dynamique qui réduit les coûts de développement et de maintenance tout en améliorant la qualité des services fournis.
- Impact environnemental réduit : en utilisant uniquement l'espace serveur nécessaire, le cloud computing favorise une informatique respectueuse de l'environnement.

I.2.5.2. Inconvénients du cloud computing

- Sécurité des données : Les données et les applications stockées dans le cloud public peuvent être exposées à des risques de sécurité et de confidentialité accrus.

- Exigences en matière de réseau et de connectivité : Le cloud nécessite un réseau à haut débit et une connectivité permanente pour assurer un accès constant aux services.
- Surveillance et respect des SLA : Une surveillance continue et l'application rigoureuse des accords de niveau de service (SLA) sont essentielles pour garantir des performances optimales et répondre aux exigences des utilisateurs.

I.3. Architecture FaaS

L'informatique sans serveur appartient à la catégorie "Fonctions en tant que Service (FaaS)" [18], qui repose sur la virtualisation au niveau de l'exécution. Dans ce modèle FaaS, le développeur n'a pas besoin de se soucier de l'environnement d'exécution utilisé par le code. Il va tout simplement écrire son code en fonction de la plateforme, en utilisant les méthodes et les bibliothèques fournies par celle-ci.

I.3.1. Principes fondamentaux de l'architecture FaaS

Selon l'article "The Rise of Serverless Computing" [19], l'informatique sans serveur est définie comme étant « une plateforme qui masque l'utilisation des serveurs aux développeurs et exécute le code à la demande, avec une mise à l'échelle automatique et une facturation uniquement pour le temps d'exécution du code ». Aussi, la CNCF définit l'informatique sans serveur comme étant : « le concept de création et d'exécution d'applications qui ne nécessitent pas de gestion de serveur. Il décrit un modèle de déploiement plus fin où les applications, regroupées en une ou plusieurs fonctions, sont téléchargées sur une plateforme, puis exécutées, mises à l'échelle et facturées en fonction de la demande exacte du moment » [20].

Toutes ces définitions touchent des aspects importants de ce concept, mais indépendamment de la définition adoptée, l'un des concepts principaux associés à l'informatique sans serveur est que les utilisateurs n'ont pas besoin de gérer l'administration et l'hébergement des serveurs. Cette responsabilité est transférée aux fournisseurs de cloud. Il existe plusieurs services disponibles dans le concept sans serveur, car les idées de nouvelles approches sans serveur dans différents domaines affluent constamment. Amazon Aurora est un exemple qui diffère un peu de la définition précédemment présentée tout en étant qualifié de sans serveur. D'un côté, il dispose de puissantes capacités d'auto-scalabilité, mais d'un autre côté, il nécessite une mémoire et un CPU minimum alloués et ne peut donc pas se réduire à zéro, ce qui entraîne des coûts continus.

Les architectures sans serveur sont divisées en deux catégories différentes : Backend as a Service (BaaS) et Function as a Service (FaaS). Les deux servent des objectifs différents et apportent leur propre valeur unique :

— **BaaS**

Le BaaS, également connu sous le nom de MBaaS (Mobile Backend as a Service), est un modèle dans lequel les développeurs externalisent le code backend à des fournisseurs spécifiques. Ces fournisseurs proposent des logiciels pré-écrits pour des scénarios courants tels que le stockage de données et de fichiers, la messagerie et les notifications push, l'authentification ou le passage de messages. Avec cela en tête, les développeurs peuvent se concentrer sur le développement de la partie frontend de l'application, tout en intégrant la partie BaaS du code via des API et des SDK.

— **FaaS**

Le FaaS est un modèle qui permet aux développeurs d'exécuter des fonctions autonomes dans le cloud. Il permet à de petits morceaux de code, représentés sous forme de fonctions, de s'exécuter pendant une durée limitée. Il est couramment utilisé pour le traitement en temps réel des données. Il masque l'infrastructure sous-jacente, car il lance les instances nécessaires pour exécuter la fonction avec uniquement les ressources requises, puis les libère à la fin [21]. Les fonctions sont sans état, ce qui signifie qu'elles ne stockent aucune donnée entre les exécutions, et sont déclenchées par des événements tels que : les requêtes d'interface d'application (comme une requête HTTP), les modifications de la base de données et les événements provenant des dispositifs IoT, etc. [4]. Cela permet une mise à l'échelle facile. Il décrit un modèle de déploiement plus fin où les applications sans serveur, constituées d'une ou plusieurs fonctions, sont téléchargées sur une plateforme cloud où elles sont mises à l'échelle et facturées en fonction de leur utilisation.

I.3.2. Plateformes serverless

Diverses plateformes proposent des services serverless, parmi lesquelles :

AWS Lambda

AWS Lambda a été le premier fournisseur de services cloud public à initier le service de computing sans serveur pour les consommateurs en 2014. AWS utilise ses autres services comme AWS S3, AWS Edge, AWS Fargate, DynamoDB, AWS EFS, Amazon Athena, Amazon Kinesis, etc., pour assurer un service isolé et indépendant avec AWS SAM. Le service de computing

événementiel exécute du code en réponse à des événements et gère automatiquement les ressources de computing. AWS démarre un conteneur Lambda et fournit l'environnement pour exécuter des codes pour des cas d'utilisation comme le téléchargement d'images, les téléchargements d'objets, les mises à jour de DynamoDB et les lectures de capteurs provenant de dispositifs IoT connectés. Les conteneurs AWS sont basés sur une architecture Linux native. Le provisionnement basé sur des requêtes HTTP personnalisées pour déclencher des services backend et réduire l'échelle lorsque les ressources ne sont pas nécessaires peut être effectué en utilisant Lambda. L'API Gateway AWS et AWS Cognito peuvent gérer les tâches d'authentification et d'autorisation. AWS prend en charge divers langages de développement comme Node.js, Python, Java, Ruby, Go, C et .NET. La taille maximale du package Lambda compressé est de 50 MB, et la taille maximale décompressée est de 250 MB.

IBM OpenWhisk

IBM a annoncé sa propre version de Function-as-a-Service en 2016 basée sur Apache OpenWhisk. OpenWhisk fonctionne également de manière similaire à AWS Lambda avec une architecture événementielle et permet aux développeurs de déclencher des réponses aux événements. Il permet à l'utilisateur de concevoir des microservices qui exécutent du code lorsqu'un événement comme un clic de souris ou une récupération de données de capteur depuis des dispositifs IoT se produit. Il offre un chaînage pour connecter plusieurs applications développées séparément. Le support intégré des conteneurs élimine le verrouillage du fournisseur en permettant aux développeurs d'exécuter des codes personnalisés sur des conteneurs Docker. Il prend en charge des langages comme Node.js, Swift, PHP, Python et Java. Il est également compatible avec d'autres services cognitifs intégrés comme Watson et Weather. Un point différenciant d'IBM OpenWhisk par rapport à Google Cloud Functions et AWS Lambda est leur accent sur l'intégration des conteneurs Docker. Les cas d'utilisation d'IBM OpenWhisk comprennent les applications web serverless, le développement et l'intégration d'API, les applications mobiles avec backend serverless, la création de vidéos consultables, les microservices, etc. Les clients notables de haut rang incluent GreenQ, Articoolo, SiteSpirit, etc.

Google Cloud Functions

Google Cloud Functions est une plateforme serverless pour développer et exécuter des fonctions cloud et utiliser ces fonctions comme des blocs de construction pour de grandes applications. Elle a été développée en 2016

pour connecter divers services fournis par Google et les mettre à disposition des développeurs afin qu'ils puissent être surveillés à l'aide d'outils comme Cloud Trace ou Cloud Debugger. Ces fonctions permettent aux développeurs de déclencher du code à partir d'autres services Google comme Google Cloud, Firebase et Google Assistant, ou même via des appels HTTP depuis n'importe quelle application web ou mobile. Ces fonctions cloud peuvent être écrites en utilisant JavaScript, Python 3, Go, NodeJs ou les environnements d'exécution Java. Les fonctions cloud ont également accès aux identifiants du compte de service Google, permettant une authentification transparente avec d'autres services cloud comme Cloud Vision. Associer une fonction cloud à un déclencheur d'événement permet aux utilisateurs de capter l'événement et d'agir en conséquence. La nature fine et à la demande de Cloud Functions en fait un candidat idéal pour les API légères et les Webhooks. Les autres cas d'utilisation incluent l'analyse des sentiments, l'analyse vidéo et image, les assistants virtuels et les expériences conversationnelles, le traitement de flux en temps réel, le traitement de fichiers en temps réel, les backends IoT serverless, les backends mobiles serverless, l'intégration avec des API tierces, etc. Les principaux clients de Google Cloud Functions incluent eBay, PayPal, HSBC, 20th Century Fox, LG CNS, etc

Azure Functions

Azure Functions est une plateforme de computing serverless événementielle qui fonctionne avec des déclencheurs et des liaisons de fonctions à ces déclencheurs pour faciliter le développement simplifié et accéléré des applications. La plateforme a commencé son parcours en version limitée en mars 2016 par Microsoft. Azure Functions a été conçue pour étendre les services d'application Azure existants et les doter de capacités comme l'implémentation de code déclenchée par des événements se produisant dans le cloud Azure ou d'autres services tiers. Entièrement intégrée à d'autres services Azure et outils de développement, son expérience de développement de bout en bout permet aux développeurs de créer et de déboguer des fonctions localement sur n'importe quelle plateforme majeure comme Windows, Linux ou macOS. La mise à l'échelle automatique et le provisionnement de nature serverless offrent une orchestration simple pour des défis d'orchestration complexes. Le trait de paiement à l'exécution encourage le déploiement de la même application dans différents environnements d'hébergement selon les besoins de l'application. Azure Functions prend en charge divers langages pour ses développements d'application comme .NET, Python, NodeJs, Java,

etc. Les outils de sécurité et de surveillance intégrés comme Azure Application Insights et Azure Monitor équipent les développeurs pour repérer les goulots d'étranglement et les points chauds de défaillance à travers le pipeline d'architecture de l'application. La plateforme FaaS de niveau entreprise dispose également du support PowerShell et de la fonctionnalité Durable Function, qui offre un moyen de définir des applications à états de manière programmatique et serverless. Les cas d'utilisation d'Azure Functions incluent le traitement basé sur le temps (workloads Corn), le traitement des événements SaaS (Software-as-a-Service), les backends mobiles, le traitement de flux en temps réel (IoT), ou la messagerie en temps réel. Les fonctions Azure de haut niveau incluent Relativity, FujiFilm, direct.one, Hotailors, etc.

1.3.3. Bonnes pratiques pour le développement serverless

L'informatique sans serveur a gagné en popularité grâce à sa flexibilité et son efficacité en termes de coûts, mais de bonnes pratiques de développement s'imposent pour pouvoir en tirer pleinement parti de ses avantages. On notera ainsi :

La scalabilité

Les applications sans serveur doivent être conçues pour pouvoir permettre une mise à l'échelle horizontale et gérer différentes charges de travail de manière efficace. Il est donc ainsi recommandé d'utiliser une architecture de microservices pour ces cas d'applications. En décomposant des applications complexes en plus petits services indépendants, chacun pourra être desservi par des fonctions sans serveur, ce qui va par conséquent améliorer la maintenabilité, favoriser l'autonomie des équipes et permettre une mise à l'échelle granulaire des services.

Il est également crucial d'utiliser une architecture événementielle pour déclencher des fonctions en réponse à des événements tels que des requêtes HTTP, des changements de base de données ou des messages de file d'attente [21]. Cela va permettre à l'application de s'auto-scaler sans nécessiter une intervention manuelle.

Sélection du framework

Choisir le bon environnement d'exécution est très important. Plusieurs facteurs doivent être pris en compte, tels que les performances, le support de la plateforme serverless et les contributions de la communauté. Utiliser un framework bien établi facilite le processus de développement avec des modèles, l'automatisation du déploiement et des composants réutilisables.

Principe de responsabilité unique

Chaque fonction doit avoir un objectif clair, précis et bien défini. Il est recommandé de regrouper des fonctionnalités non liées dans une seule fonction, car cela peut entraîner un dysfonctionnement dans le code et une moindre maintenabilité.

Sécurité

Si les fonctions sans serveur sont gérées en interne, il est essentiel de sécuriser l'API avec une passerelle API. Celle-ci peut gérer le trafic entrant de l'API, comme la limitation du débit et le filtrage des requêtes, pour éviter de surcharger les fonctions sans serveur et protéger contre d'éventuelles attaques par déni de service distribué (DDoS) [22].

Surveillance

Le but de la surveillance des applications sans serveur est d'identifier et de résoudre les problèmes pouvant impacter les performances et la fiabilité des applications, ainsi que d'optimiser l'utilisation des ressources pour une efficacité en termes de coûts. Cela implique de surveiller divers aspects des applications et de l'infrastructure sans serveur, y compris l'exécution des fonctions, l'utilisation des ressources et les métriques de performance [22].

I.3.4. Cas d'utilisation

L'un des plus grands avantages de l'informatique sans serveur est sa polyvalence et sa facilité d'utilisation dans une grande variété de scénarios. Selon AWS [23], il existe trois grandes catégories de cas d'utilisation serverless :

- Traitement de données asynchrone
- Interactions synchrones
- Streaming

Traitement de données asynchrone

Le traitement de données asynchrone permet d'ingérer, de traiter et d'analyser rapidement de grands volumes de données, provenant de diverses sources comme les appareils mobiles, les applications et les infrastructures. Avec les services serverless, il est possible de provisionner des ressources de manière élastique pour gérer ces données à grande échelle, facilitant ainsi des décisions commerciales rapides et éclairées.

Par exemple :

- *Le traitement de fichiers : dans une application de partage de photos, les images téléchargées par les utilisateurs sont stockées dans un bucket*

Amazon S3. Une fonction Lambda peut être configurée pour créer automatiquement des miniatures de ces images, en utilisant les événements de création d'objet d'Amazon S3.

- *L'identification d'images : en utilisant Amazon Rekognition, les images téléchargées peuvent être automatiquement catégorisées. Les visages et objets détectés permettent de regrouper les photos et d'améliorer la recherche.*

Interactions synchrones

L'architecture des microservices permet de diviser les applications en services indépendants et faiblement couplés, facilitant leur mise à l'échelle individuelle. Chaque microservice peut être développé et maintenu de manière autonome, utilisant des API pour communiquer avec les autres services.

Par exemple :

- *Sites web : utiliser Amazon API Gateway pour invoquer des fonctions Lambda via HTTP, permettant au client web de communiquer avec le backend.*
- *Applications mobiles : créer des fonctions Lambda pour traiter les événements générés par des applications mobiles personnalisées.*

Streaming

Le traitement de flux de données en temps réel permet de recueillir des insights analytiques et d'agir rapidement. Les services serverless comme Lambda et Amazon Kinesis sont utilisés pour traiter des flux de données en continu, provenant d'activités d'applications, de transactions, de clics, d'analyses des médias sociaux et de télémétrie des appareils IoT.

Par exemple :

- *Données et analyses : utiliser les streams DynamoDB pour publier des événements de mise à jour d'éléments et une fonction Lambda pour agréger les données et générer des métriques personnalisées.*
- *Surveillance des métriques : Amazon Prime Video utilise Amazon Kinesis Data Streams pour surveiller les métriques des flux vidéo en temps réel, en analysant les données avec Amazon OpenSearch Service pour détecter et résoudre les problèmes de streaming.*

I.4. Généralités sur le machine learning

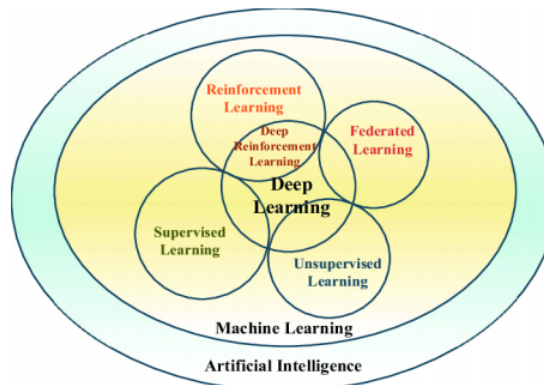


FIGURE 6 – Relation entre l'intelligence artificielle, l'apprentissage automatique et l'apprentissage profond

I.4.1. Techniques de machine learning

Le machine learning, ou apprentissage automatique, représente une application de l'IA qui confère au système la capacité d'apprendre et de s'améliorer de manière autonome à partir de l'expérience, sans nécessiter une programmation explicite. Cette discipline se concentre sur le développement de programmes informatiques capables d'accéder aux données et de les utiliser pour apprendre par eux-mêmes [11]. Le machine learning peut être catégorisé en plusieurs groupes principaux, notamment l'apprentissage supervisé, l'apprentissage semi-supervisé, l'apprentissage non supervisé et l'apprentissage par renforcement.

I.4.1.1. Apprentissage supervisé

L'apprentissage supervisé est une méthode d'apprentissage automatique où l'algorithme apprend à faire des prédictions ou à classer des données en se basant sur des données d'entraînement étiquetées. En apprentissage supervisé, l'algorithme est formé sur un ensemble de données déjà étiqueté avec les bonnes réponses ou résultats, et il apprend à généraliser à partir des exemples étiquetés pour faire des prédictions sur de nouvelles données non vues.

Voici les équations de base pour l'apprentissage supervisé :

- **Entrée (X)** : données que le modèle utilise pour faire des prédictions,
- **Sortie (Y)** : étiquette correcte ou du résultat pour chaque exemple dans les données d'entraînement,
- **Sortie (f)** : fonction que l'algorithme apprend à mapper les entrées aux sorties. L'objectif est de trouver le meilleur modèle qui peut prédire avec précision la sortie correcte pour de nouvelles données d'entrée.

La formule générale pour un modèle d'apprentissage supervisé est : $Y = f(X) + \text{erreur}$

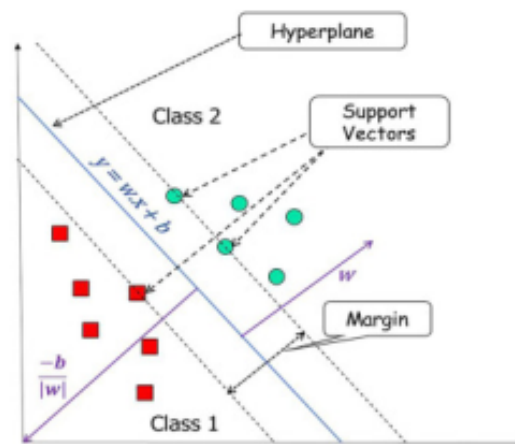


FIGURE 7 – Classification SVM

où *erreur* représente la différence entre la sortie prédite et la sortie réelle.

On l'appelle apprentissage supervisé car le processus d'apprentissage d'un algorithme à partir de l'ensemble de données d'entraînement peut être assimilé à celui d'un enseignant supervisant le processus d'apprentissage. Nous connaissons les bonnes réponses, l'algorithme fait des prédictions de manière itérative sur les données d'entraînement et est corrigé par l'enseignant. L'apprentissage s'arrête lorsque l'algorithme atteint un niveau de performance acceptable. Les algorithmes d'apprentissage supervisé peuvent être regroupés en algorithmes de régression et de classification.

Classification

Dans les tâches de classification, l'objectif est de prédire un résultat catégorique, tel que si un e-mail est un spam ou non. La variable de sortie est une valeur discrète, généralement représentée par une étiquette ou une catégorie. Les algorithmes courants de classification incluent la régression logistique, les arbres de décision et les machines à vecteurs de support.

Régression

Dans les tâches de régression, l'objectif est de prédire un résultat continu, tel que le prix d'une maison ou la température extérieure. La variable de sortie est une valeur continue, généralement représentée par une valeur numérique. Les algorithmes courants de régression incluent la régression linéaire, les arbres de décision et les réseaux neuronaux.

Il existe une grande variété d'algorithmes d'apprentissage supervisé qui peuvent être utilisés pour différents types de tâches. Voici quelques exemples d'algorithmes d'apprentissage supervisé courants :

- **Machines à vecteurs de support (SVM)** : utilisées à la fois pour les tâches de classification et de régression. Elles trouvent l'hyperplan qui sépare le mieux les données en différentes classes ou prédit la valeur de sortie.
- **Réseau neuronal** : utilisé pour une grande variété de tâches, y compris la reconnaissance d'images, le traitement du langage naturel et la reconnaissance vocale. Il se compose de nœuds interconnectés qui traitent l'information et apprennent à faire des prédictions en fonction des données d'entrée.
- **Régression linéaire** : utilisée pour les tâches de régression, où l'objectif est de prédire une valeur numérique continue. Elle utilise une fonction linéaire pour modéliser la relation entre les variables d'entrée et de sortie.
- **Régression logistique** : utilisée pour les tâches de classification binaire, où l'objectif est de prédire l'une des deux issues possibles. Elle utilise une fonction logistique pour modéliser la probabilité de chaque résultat.
- **Arbres de décision** : utilisés à la fois pour les tâches de classification et de régression. Ils créent un modèle en forme d'arbre des décisions et de leurs conséquences possibles, en fonction des variables d'entrée.
- **Forêt aléatoire** : c'est une extension des arbres de décision, où plusieurs arbres sont créés et leurs résultats sont combinés pour améliorer la précision et réduire le surajustement.
- **K plus proches voisins (KNN)** : utilisé pour les tâches de classification, où il classe un nouveau point de données en trouvant les K points de données les plus proches dans l'ensemble d'entraînement et en prenant un vote majoritaire.

1.4.1.2. Apprentissage non supervisé

L'apprentissage non supervisé est une forme d'apprentissage automatique où l'algorithme apprend à partir de données non étiquetées. Contrairement à l'apprentissage supervisé, l'apprentissage non supervisé ne dispose pas d'une variable cible, et l'algorithme tente de trouver des motifs et des relations dans les données sans aucune sortie prédéfinie. Cette approche est largement utilisée dans l'analyse exploratoire des données et peut aider à détecter des anomalies, à regrouper des points de données similaires et à réduire la dimensionnalité des données.

Plusieurs types d'algorithmes d'apprentissage non supervisé existent, notamment **le clustering, la réduction de dimensionnalité et l'apprentissage de règles d'association.**

- Le clustering consiste à diviser les données en clusters en fonction de la similarité des caractéristiques. L'objectif est de regrouper les points de données similaires tout en maintenant la séparation entre les groupes différents.
- La réduction de dimensionnalité vise à réduire le nombre de caractéristiques dans les données tout en préservant les informations les plus importantes. Cette technique permet de simplifier les données et de les rendre plus gérables pour l'analyse.
- L'apprentissage par règles d'association cherche à identifier les relations entre les variables dans les données. Son objectif est de trouver les cooccurrences les plus fréquentes entre les éléments d'un ensemble de données.

Parmi les exemples courants d'algorithmes d'apprentissage non supervisé, on trouve l'algorithme K-Means pour les problèmes de clustering, l'analyse en composantes principales (ACP) pour les problèmes de réduction de dimensionnalité, et l'algorithme A priori pour les problèmes d'apprentissage de règles d'association.

1.4.1.3. Apprentissage semi-supervisé

La problématique des données d'entrée abondantes (X) dont seule une fraction est étiquetée (Y) définit ce que l'on appelle un problème d'apprentissage semi-supervisé. Cette catégorie de problèmes se situe en un point intermédiaire entre l'apprentissage supervisé et non supervisé, illustrant la complexité croissante des tâches d'analyse et de prédiction dans le domaine de l'apprentissage automatique.

Cette situation découle généralement d'une pénurie de données étiquetées. En effet, l'étiquetage des données est souvent chronophage et onéreux, nécessitant souvent l'intervention d'experts dans le domaine concerné. En contraste, les données non étiquetées sont aisément accessibles et économiques à collecter et à stocker. Il est ainsi possible de recourir à des méthodes d'apprentissage non supervisé pour sonder et appréhender la structure latente des variables d'entrée.

De surcroît, les techniques d'apprentissage non supervisé offrent la possibilité d'établir des prédictions pertinentes sur des données non étiquetées. Ces prédictions peuvent ensuite être intégrées dans un cadre d'apprentissage supervisé, enrichissant ainsi le jeu de données d'entraînement. Le modèle ainsi construit devient alors capable de formuler des estimations implicites sur de nouvelles données, offrant ainsi une solution prometteuse pour les cas où l'étiquetage complet des données est difficile ou coûteux.

1.4.1.4. Apprentissage par renforcement

Le Reinforcement Learning, ou apprentissage par renforcement, représente une catégorie de l'apprentissage automatique dans laquelle un agent apprend à prendre

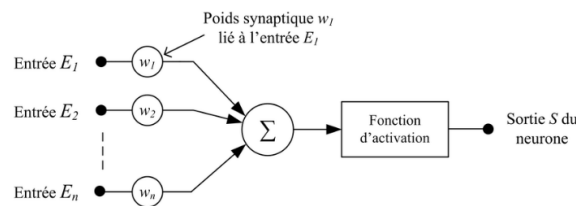


FIGURE 8 – Structure d'un neurone [6]

des décisions en interagissant avec son environnement. L'agent reçoit des retours sous forme de récompenses ou de punitions en fonction de ses actions, et apprend à optimiser son comportement afin de maximiser sa récompense. Le Reinforcement Learning trouve des applications variées telles que la robotique, les jeux, ou encore les véhicules autonomes.

À un niveau conceptuel élevé, le problème du reinforcement learning peut être formulé comme un processus de décision markovien (MDP). Un MDP est composé d'un ensemble d'états de l'environnement finis S , d'un ensemble d'actions possibles $A(s)$ dans chaque état, d'une fonction de récompense réelle $R(s)$, et d'un modèle de transition $P(s', s|a)$ qui définit la probabilité de passer d'un état à un autre en prenant une action donnée.

Les problèmes de reinforcement learning se décomposent comme suit :

- **L'environnement** : le monde physique dans lequel l'agent opère ;
- **L'état** : la situation actuelle de l'agent ;
- **La récompense** : le retour de l'environnement ;
- **La politique** : la méthode pour mapper l'état de l'agent vers les actions ;
- **La valeur** : la récompense future qu'un agent recevrait en prenant une action dans un état particulier.

1.4.2. Deep learning

Le deep learning est un sous-domaine de l'apprentissage automatique qui implique l'entraînement de réseaux neuronaux artificiels avec plusieurs couches afin d'accomplir des tâches complexes telles que la reconnaissance d'images, la reconnaissance vocale, le traitement du langage naturel et la conduite autonome [24]. Contrairement aux algorithmes traditionnels d'apprentissage automatique qui nécessitent un génie des caractéristiques manuel, les algorithmes de deep learning peuvent apprendre automatiquement des caractéristiques de haut niveau à partir de données brutes, les rendant très efficaces pour les tâches où les caractéristiques pertinentes ne sont pas bien comprises.

Le perceptron est un type de neurone artificiel utilisé dans l'apprentissage automatique et conçu pour classer les données d'entrée dans l'une des deux caté-

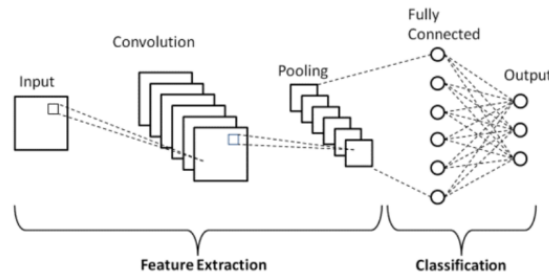


FIGURE 9 – Structure d'un réseau de neurones convolutionnels (CNN) [7]

gories possibles. Il a été introduit pour la première fois par Frank Rosenblatt en 1957. Le perceptron reçoit un vecteur de valeurs d'entrée, chacune étant associée à un poids. La somme pondérée des entrées passe ensuite par une fonction d'activation, qui produit une sortie binaire de 0 ou 1, selon que la somme pondérée dépasse ou non un certain seuil. La structure d'un neurone est présenté à la figure 8.

Le réseau neuronal se compose de couches de nœuds interconnectés, appelés neurones, qui effectuent des calculs simples sur les données d'entrée. La sortie d'une couche est transmise en tant qu'entrée à la couche suivante, permettant au réseau d'apprendre des représentations de plus en plus complexes des données [25]. Il existe de nombreuses architectures de deep learning, dont certaines seront présentées dans la section suivante.

1.4.2.1. Convolutional neural networks

Les CNN tirent leur appellation de l'utilisation de l'opération mathématique de convolution. Ils représentent une catégorie spécifique de réseaux neuronaux qui recourent à la convolution à la place de la multiplication matricielle générale dans au moins l'une de leurs couches. Les CNN se distinguent par leur efficacité dans l'extraction de caractéristiques pertinentes à partir de données locales corrélées, grâce à l'opération de convolution. Cette opération permet de détecter des motifs significatifs dans les données en préservant la relation spatiale entre les points, avant d'appliquer une fonction d'activation non linéaire. Cette dernière non seulement facilite l'apprentissage des abstractions mais aussi introduit la non-linéarité dans l'espace des caractéristiques, favorisant ainsi la représentation de différentes réponses et la distinction de nuances sémantiques dans les images [26].

La structure des CNN est généralement composée de plusieurs couches d'apprentissage, combinant des couches convolutives, des unités de traitement non linéaires et des couches de sous-échantillonnage [26, 27]. La Figure 9 illustre la configuration typique d'un réseau CNN [7].

- **Couche de convolution** : cette première couche extrait les entités fondamentales d'une image en préservant les relations spatiales entre les pixels. L'opération de convolution consiste à appliquer un filtre ou un noyau à l'image d'entrée, permettant ainsi l'apprentissage des caractéristiques essentielles de celle-ci [28, 29].
- **Couche de pooling** : placée généralement entre deux couches de convolution, cette couche vise à réduire la dimensionnalité de la représentation tout en préservant les caractéristiques essentielles [30]. Différents types de pooling sont utilisés, notamment le pooling moyen qui calcule la moyenne des valeurs dans une région sélectionnée, et le pooling maximal qui retient la valeur maximale dans cette région.
- **Couche entièrement connectée** : cette dernière couche est similaire aux réseaux entièrement connectés traditionnels. Elle reçoit en entrée la sortie de la phase précédente (comprenant les opérations de convolution et de pooling) et effectue un produit scalaire entre le vecteur de poids et le vecteur d'entrée pour obtenir la sortie finale [29].

1.4.2.2. Long short-term memory

Les LSTM représentent une catégorie particulière de Réseaux de Neurones Récurrents (RNN), réputés pour leur capacité à appréhender les dépendances à long terme. L'introduction de cette architecture novatrice est attribuée à Hochreiter et Schmidhuber en 1997, et depuis, leur développement et leur popularité ont été renforcés par un éventail de chercheurs. Les LSTM se sont avérés particulièrement efficaces dans une diversité de problèmes, les rendant aujourd'hui omniprésents dans de nombreux domaines [31].

Conçus explicitement pour surmonter les limitations des dépendances à long terme, les LSTM sont intrinsèquement adaptés à retenir des informations sur de vastes périodes, ce qui les distingue nettement de leurs prédécesseurs. Contrairement aux RNN classiques, caractérisés par une structure répétitive simple, les LSTM adoptent une architecture plus élaborée. Chaque LSTM est composé de plusieurs modules interagissant de manière complexe. Un composant fondamental des LSTM est la cellule mémoire, où deux états cruciaux sont transférés : l'état de la cellule et l'état caché.

L'état cellulaire agit comme un vecteur principal de flux de données, permettant aux informations de circuler de manière quasi inchangée, tout en étant sujet à des transformations linéaires via des portes sigmoïdes. Ces portes, fonctionnant comme des couches de traitement, contrôlent l'ajout ou la suppression sélective

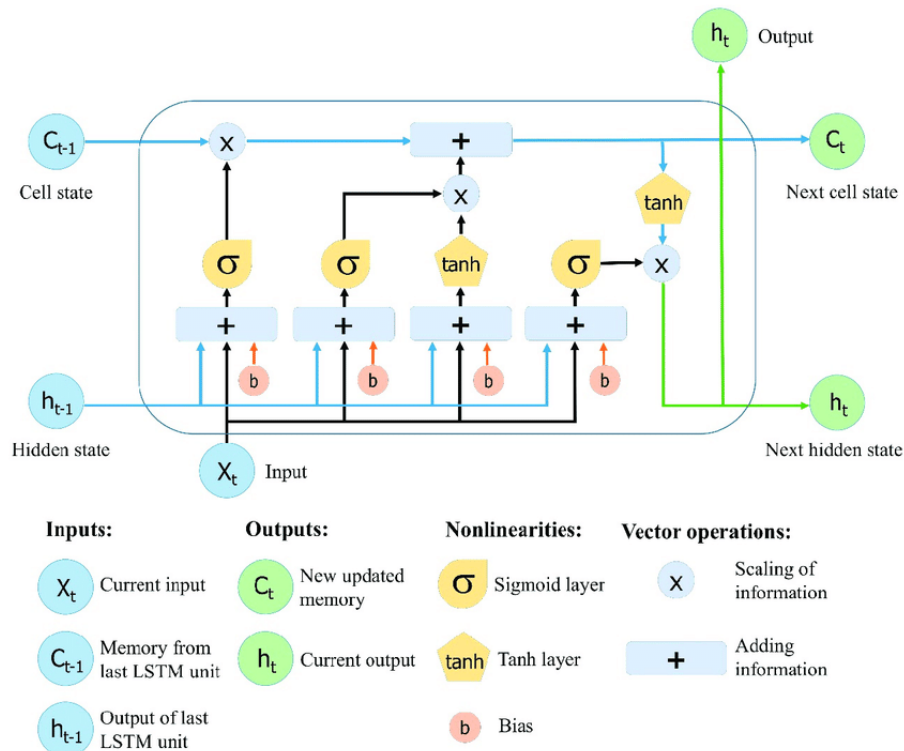


FIGURE 10 – Module répétitif dans un LSTM composé de quatres couches interactives

d'informations à l'état cellulaire. Cette conception vise explicitement à résoudre le défi des dépendances à long terme en offrant un mécanisme de régulation précis.

La construction d'un LSTM passe par plusieurs étapes clés. D'abord, l'identification des informations à conserver est déterminée par une porte d'oubli, alimentée par une fonction sigmoïde. Ensuite, l'intégration des nouvelles informations est orchestrée par une combinaison de portes sigmoïdes et de couches de transformation, ajustant ainsi l'état de la cellule. Enfin, la génération de la sortie finale est guidée par une autre série de portes et de couches, permettant une sortie filtrée et adaptée.

En conclusion, les LSTM représentent une avancée significative dans le domaine des réseaux neuronaux, offrant une solution robuste au défi des dépendances à long terme et ouvrant la voie à une multitude d'applications prometteuses.

1.4.2.3. Transformers

L'avènement du Transformateur en 2017, porté par Vaswani et ses pairs, marque un jalon significatif dans le domaine de l'apprentissage profond et du traitement du langage naturel. Initialement conçu pour répondre aux défis complexes de la traduction automatique et de la modélisation linguistique, cet écosystème neural a rapidement transcendé ses applications d'origine pour s'imposer comme un ou-

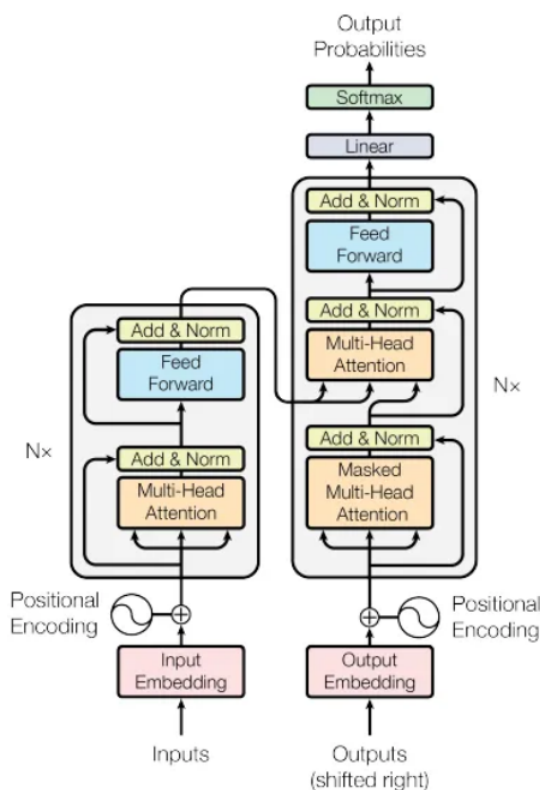


FIGURE 11 – Architecture des Transformers [8]

til polyvalent, adapté à une pléthore de tâches, de la manipulation d'images à la transcription vocale.

Au cœur de son architecture réside le concept d'auto-attention, une innovation radicale qui permet au réseau de neurones de hiérarchiser les différentes parties d'une séquence d'entrée pendant le processus de traitement. Concrètement, l'entrée d'un Transformateur se présente sous la forme d'une séquence de vecteurs, chacun représentant une unité discrète de données, telle qu'un mot dans une phrase ou un pixel dans une image. Chaque vecteur est ainsi associé à une position spécifique dans la séquence, permettant au modèle d'appréhender la structure linéaire de l'information.

L'auto-attention révolutionne le paysage de l'apprentissage en permettant à chaque vecteur de la séquence de calculer son degré d'importance par rapport à tous les autres. Cette capacité à capturer des dépendances complexes, y compris celles s'étendant sur de longues distances dans le langage naturel, représente un saut qualitatif dans la capacité des réseaux neuronaux à modéliser des données séquentielles.

Le mécanisme d'auto-attention opère en évaluant un ensemble de scores entre chaque vecteur de la séquence et tous les autres. Ces scores, exprimant l'attention

accordée à chaque élément, sont ensuite utilisés pour pondérer la contribution de chacun à la sortie finale, avant d'être agrégés à travers un réseau neuronal feedforward pour produire le résultat désiré. Comparativement à ses prédécesseurs, tels que les RNN et les CNN, le Transformateur présente plusieurs avantages distinctifs. Il excelle dans le traitement de séquences d'entrée de longueurs variables, grâce à son architecture intrinsèquement parallèle, et se révèle particulièrement efficace dans la capture de dépendances à longue portée, dépassant ainsi les limitations des approches séquentielles traditionnelles.

L'adoption généralisée du Transformateur comme une architecture de prédilection dans le domaine du traitement du langage naturel atteste de son impact remarquable. Il a non seulement démontré sa capacité à surpasser les modèles classiques dans des tâches telles que la traduction automatique et la modélisation du langage, mais il a également ouvert de nouvelles perspectives dans des domaines tels que la génération de texte et la compréhension du langage naturel.

1.4.3. Applications du machine learning

Les applications du ML sont devenues omniprésentes, envahissant divers domaines de la vie moderne. Parmi ceux-ci, on peut citer :

- **Le traitement du langage naturel** : le ML révolutionne la façon dont les machines comprennent et génèrent le langage humain, alimentant des avancées majeures dans la traduction automatique, la génération de texte et la compréhension du langage naturel.
- **La vision par ordinateur** : grâce au ML, les ordinateurs peuvent désormais interpréter et analyser les images et les vidéos de manière autonome, ouvrant de nouvelles possibilités dans des domaines tels que la reconnaissance d'objets, la détection d'anomalies et la réalité augmentée.
- **La banque et la finance** : dans le secteur financier, le ML est utilisé pour l'analyse des risques, la détection de fraudes, la prédiction des tendances du marché et la personnalisation des services financiers, contribuant ainsi à une gestion plus efficace des ressources et des risques.
- **L'éducation** : dans le domaine de l'éducation, le ML est utilisé pour personnaliser les programmes d'apprentissage, identifier les lacunes des élèves et fournir des recommandations pédagogiques adaptées à chaque apprenant, favorisant ainsi l'efficacité et l'engagement.
- **La recherche scientifique** : le ML accélère la découverte de nouveaux médicaments, la modélisation climatique, et la compréhension des phénomènes

complexes en analysant de vastes ensembles de données, ouvrant de nouvelles perspectives dans de nombreux domaines de la recherche scientifique.

En ce qui concerne l'intégration du ML avec le CC, plusieurs secteurs bénéficient de cette synergie, notamment :

- **La sécurité** : le ML est utilisé dans le CC pour renforcer la sécurité des données, en détectant les activités suspectes et en prévenant les menaces potentielles, garantissant ainsi la confidentialité et l'intégrité des informations stockées dans le cloud.
- **L'expérience utilisateur** : en exploitant les vastes quantités de données disponibles dans le cloud, le ML permet de personnaliser les expériences utilisateur, en anticipant les besoins individuels et en offrant des recommandations et des services adaptés à chaque utilisateur.
- **La réduction des coûts** : grâce à l'automatisation et à l'optimisation des processus, le ML contribue à réduire les coûts opérationnels dans le cloud computing, en rationalisant les workflows, en optimisant les ressources et en prévenant les inefficacités.
- **L'analyse des données massives (Big Data)** : le ML et le CC permettent le traitement et l'analyse rapides de grandes quantités de données, facilitant l'identification de tendances, de modèles et de insights précieux pour les entreprises et les organisations.
- **La gestion des ressources informatiques** : le ML est utilisé dans le CC pour optimiser l'allocation des ressources informatiques, en ajustant dynamiquement les capacités de stockage et de calcul en fonction des besoins changeants, ce qui permet d'optimiser les performances et de réduire les coûts.

I.5. Conclusion

En résumé, ce chapitre a exploré différents concepts essentiels liés à la compréhension de notre sujet. Le concept de virtualisation, ainsi que son rôle dans le cloud computing permettant la fourniture de services à la demande adaptés aux besoins des utilisateurs ont été analysés. Nous avons ensuite approfondi la compréhension des architectures FaaS, et enfin, nous avons examiné le ML et ses multiples applications dans le CC. Il apparaît clairement que les techniques de ML trouvent de nombreuses applications dans le cloud, notamment en renforçant les mesures de sécurité et l'expérience utilisateur. Dans le prochain chapitre, nous explorerons en détail comment le ML contribue à réduire la latence dans les services cloud.

II

CHAPITRE

ÉTAT DE L'ART SUR LA PRÉDICTION DES CHARGES DE TRAVAIL ET L'ATTÉNUATION DU COLD START DANS LES ARCHITECTURES FAAS

CONTENU

II.1 - Introduction	35
II.2 - Caractérisation des charges de travail et cold start	36
II.3 - Techniques de prédiction des charges de travail	42
II.4 - Stratégies d'atténuation du cold start	47
II.5 - Conclusion	56

II.1. Introduction

Les architectures FaaS dans le domaine du CC représentent une avancée majeure car ils offrent une flexibilité sans précédent pour le déploiement des applications à des échelles variables. Rappelons que les FaaS permettent aux développeurs de se concentrer sur la logique applicative sans se soucier de la gestion de l'infrastructure sous-jacente [9]. La tarification est basée sur le nombre de requêtes effectuées sur la fonction et le temps nécessaire à l'exécution du code de la fonction. Cette dernière varie également en fonction du nombre de ressources telles que la mémoire et les cœurs de CPU alloués à la fonction, et ils sont automatiquement adaptés pour offrir la meilleure performance [32]. Cependant, cette abstraction vient avec ses propres défis, parmi lesquels : **la prédiction des charges de travail et la gestion du cold start**. De ce fait, il s'avère nécessaire d'étudier les

mécanismes de prédiction des charges de travail et les stratégies pour atténuer le cold start dans le but d'optimiser les performances des architectures FaaS. Ce chapitre se penche sur les travaux de recherche actuels concernant la prédiction des charges de travail et les techniques d'atténuation du cold start dans les systèmes FaaS. Nous analyserons les principales solutions proposées par divers chercheurs et explorerons les approches basées sur l'IA pour améliorer ces processus.

II.2. Caractérisation des charges de travail et cold start

II.2.1. Charges de travail

Contrairement aux architectures traditionnelles où les applications sont hébergées sur des serveurs dédiés ou des VM avec une capacité fixe, les FaaS permettent une exécution élastique et à la demande des fonctions. Ce paradigme transfère la complexité de l'allocation et de la provision des ressources aux CSP, qui devra donner une illusion de ressources toujours disponibles (c'est-à-dire des invocations de fonctions rapides) au coût de ressources le plus bas possible, et pour ce faire, le CSP doit comprendre en profondeur les caractéristiques des charges de travail FaaS [12]. La quantité de mémoire nécessaire joue un rôle déterminant dans cette optimisation. Les politiques de gestion efficace résultent d'une bonne exécution de la caractérisation des charges de travail [1], de la prédiction des performances, de la planification des ressources et du dimensionnement de ces ressources : la scalabilité peut être verticale ou horizontale. La scalabilité horizontale se caractérise par l'ajout et l'élimination de conteneurs existants, tandis que la scalabilité verticale vise à augmenter (ajouter plus de cœurs) ou à réduire (retirer des cœurs) les capacités des conteneurs existants [13].

Les charges de travail sont caractérisées par leurs **variabilités et imprévisibilités**. Plusieurs facteurs contribuent à cette variabilité tels que [12] :

- **Les horaires** : les applications peuvent connaître des pics de trafic soudain à des moments spécifiques de la journée en fonction de leur public cible. *Par exemple, une application de commerce électronique peut voir une forte augmentation des transactions pendant les heures de bureau ou en soirée.*
- **Les événements saisonniers et spéciaux** : tels que des soldes, des campagnes de marketing ou des lancements de produits peuvent provoquer des augmentations soudaines et significatives du trafic. Pendant ces périodes, la quantité de mémoire doit être suffisante pour gérer l'afflux massif de demandes en évitant les erreurs d'exécution et les ralentissements. *Par exemple,*

un site de vente en ligne peut connaître un trafic massif lors d'une promotion spéciale ou pendant les fêtes de fin d'année.

- **Les comportements utilisateurs :** ils peuvent influencer les modèles de demande. Pour ces pics, la quantité de mémoire nécessaire doit être suffisante pour assurer une expérience utilisateur fluide. *Par exemple, une application de streaming vidéo peut voir des pics de trafic lors de la sortie d'un nouvel épisode d'une série populaire.*

La nature dynamique et imprévisible des charges de travail serverless rend difficile la prédiction précise de la quantité de ressources nécessaires pour gérer les requêtes entrantes. Cette imprévisibilité peut entraîner une sous-utilisation des ressources ou une sur-provisionnement, ce qui augmente les coûts. Les techniques de ML peuvent aider à relever ce défi en permettant aux plateformes serverless de provisionner dynamiquement et de façon automatique les ressources pour répondre aux demandes tout en minimisant les coûts [2].

Durant la phase de déploiement d'une fonction, certaines caractéristiques essentielles telles que la méthode de déploiement, l'outil, la mémoire allouée, etc., doivent être considérées par les développeurs pour éviter les échecs de déploiement et les mauvaises performances [33]. La gestion efficace des charges de travail dans les environnements FaaS repose sur une allocation optimale des ressources, notamment la mémoire et la puissance de calcul. Les défis liés à l'allocation des ressources sont nombreux et variés.

Quantité de mémoire nécessaire Chaque fonction nécessite une quantité spécifique de mémoire pour s'exécuter de façon efficace. La mémoire allouée doit être suffisante pour couvrir les besoins de l'application, y compris le chargement des dépendances et le traitement des données. Une sous-allocation de mémoire peut entraîner des erreurs d'exécution et des ralentissements qui affecteront la performance de l'application. Une surallocation de mémoire peut augmenter les coûts sans bénéfice supplémentaire, ce qui rend l'optimisation de la mémoire critique pour une gestion efficace et économique des ressources. Dans le cas des Azure Functions, chaque application a besoin d'une certaine quantité de mémoire pour faire son travail. Un examen de la quantité de mémoire dont ces applications ont besoin montre qu'il y a une grande variation dans la quantité de mémoire utilisée par différentes applications. Par exemple, certaines applications n'ont besoin que de 170 Mo de mémoire au maximum, tandis que d'autres peuvent avoir besoin jusqu'à 400 Mo. Cela signifie que la gestion de la mémoire est très importante pour s'assurer que toutes les applications fonctionnent correctement et de manière efficace [12].

Puissance de calcul La puissance de calcul allouée à chaque fonction doit correspondre à la complexité de la tâche à accomplir. Les fonctions qui exécutent des opérations intensives en calcul, telles que le traitement d'images ou l'analyse de données, nécessitent plus de CPU. Les CSP offrent souvent des options pour ajuster la puissance de calcul en fonction des besoins spécifiques, permettant une allocation plus granulaire et économique. Cependant, cela doit être équilibré avec la quantité de mémoire nécessaire pour garantir une exécution sans faille. Dans AWS Lambda, les fonctions sont déployées individuellement via une API de gestion. La mémoire peut être configurée de 128 MiB à 10 GiB, avec une mise à l'échelle proportionnelle des ressources. Azure Functions attribue automatiquement 1,5 GiB de mémoire et un CPU par instance sur le plan de consommation, sans possibilité de configuration. Le plan premium offre une performance accrue avec une pré-provision des ressources. Google Cloud Functions permet aux utilisateurs de configurer la mémoire de leurs fonctions de 128 MiB à 2 GiB. Les ressources CPU sont attribuées en conséquence, bien que cela ne soit pas proportionnel [34].

II.2.2. Cold start

Le processus global d'exécution d'une fonction (action) est illustré à la figure 12. Le **cold start** est un phénomène critique dans les architectures FaaS qui peut fortement impacter les performances et l'expérience utilisateur des applications déployées sur ces plateformes. Il se produit lorsqu'une fonction est invoquée pour la première fois ou après une période d'inactivité prolongée, nécessitant l'initialisation de l'environnement d'exécution. Ce processus peut entraîner des délais de latence significatifs avant que la fonction puisse répondre à la requête, ce qui peut être problématique pour les applications sensibles à la latence. Contrairement à un **warm-start** qui réutilise l'instance d'une fonction déjà prête à l'emploi [34]. Les fonctions sans serveur sont déployées sur des conteneurs [35, 36]. Dès qu'une fonction est déclenchée, un nouveau conteneur est créé et la fonction avec toutes ses dépendances y est déployée [4]. Il convient de noter que si plusieurs invocations se produisent simultanément pour exécuter une fonction, de nouvelles instances de la fonction sont créées en fonction du nombre de demandes, et chaque instance est déployée et exécutée indépendamment sur un conteneur distinct [37].

Les fonctions dans les architectures FaaS peuvent être déclenchées par plusieurs types d'événements [1, 10], on distingue par exemple :

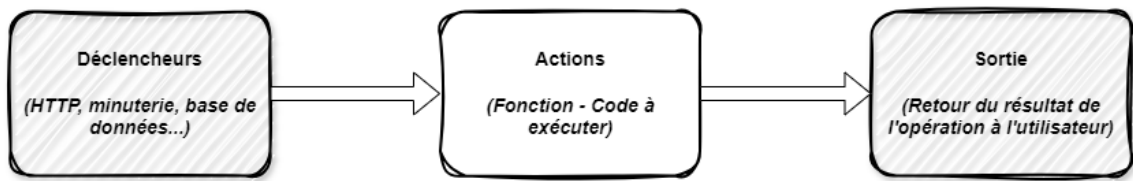


FIGURE 12 – Processus global d'appel d'une fonction dans les architectures FaaS

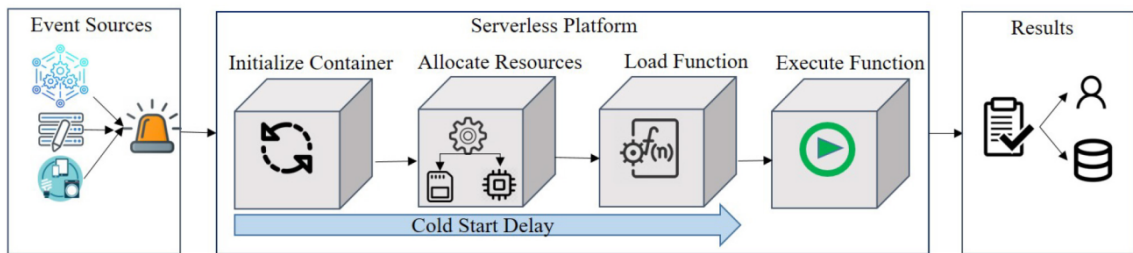


FIGURE 13 – Flux d'exécution FaaS [4]

- **Les requêtes HTTP** : les fonctions peuvent être déclenchées par des requêtes HTTP entrantes. Cela permet aux applications web d'exécuter des tâches spécifiques en réponse aux demandes des utilisateurs.
- **Les événements de base de données** : les changements dans une base de données, tels que l'ajout, la modification ou la suppression de données, peuvent déclencher des fonctions pour effectuer des actions en réponse à ces événements.
- **Les minuteries** : les fonctions peuvent être planifiées pour s'exécuter à des moments spécifiques ou à des intervalles réguliers. Cela est utile pour les tâches de maintenance, les mises à jour de données périodiques, etc.
- **Les notifications** : telles que les notifications push provenant d'applications mobiles ou de services de messagerie peuvent déclencher des fonctions pour répondre à des actions spécifiques.
- **Les événements IoT** : les dispositifs IoT peuvent générer des événements, tels que des capteurs détectant des changements d'état, qui déclenchent des fonctions pour traiter les données en temps réel.

Ces sources d'événements offrent une flexibilité considérable pour déclencher des fonctions dans les architectures FaaS en réponse à diverses actions, activités ou changements dans l'environnement [13, 32]. Observons à la figure 13, les étapes qui doivent être effectuées lorsqu'un appel de fonction se produit à partir d'une source d'événement [13, 38].

1. **Initialiser un nouveau conteneur** : préparation de l'environnement du conteneur (définition de variables d'environnement, connexion à la base de données, authentification des utilisateurs, etc). Lorsqu'une fonction est invoquée

pour la première fois ou après une période d'inactivité prolongée, un nouveau conteneur est instancié pour l'exécuter.

2. **Allouer de la mémoire et des ressources informatiques** : il est ensuite nécessaire d'allouer des ressources (mémoire, unités de traitement, etc.) nécessaires à l'exécution de la fonction. Ceci pour garantir que la fonction a suffisamment de puissance de calcul pour fonctionner correctement.
3. **Charger la fonction et les bibliothèques requises** : la fonction elle-même doit être chargée dans le conteneur (chargement du code source, bibliothèques, dépendances externes requises, etc.).
4. **Exécuter la fonction** :, avec l'environnement configuré, les ressources allouées et la fonction chargée, l'étape d'exécution commence. Le code de la fonction est exécuté selon la logique prévue.
5. **Renvoyer les résultats à l'utilisateur et enregistrer les journaux dans la base de données** : les résultats sont enfin générés. Ces résultats peuvent être renvoyés à l'utilisateur, par exemple en renvoyant une réponse HTTP avec les données demandées. En même temps, les journaux ou les informations de suivi peuvent être enregistrés dans une base de données ou un autre système de suivi pour une analyse ultérieure ou à des fins de débogage.

À la fin de l'exécution de la fonction, le conteneur et toutes ses ressources seront libérés. C'est ce qu'on appelle la mise à l'échelle à zéro (scale-to-zero) [39, 40] qui est une caractéristique innovatrice dans les FaaS et qui favorise la réduction des coûts [41]. Comme on peut l'observer, les trois premières étapes du processus nécessitent beaucoup plus de temps et par conséquent entraînent un retard dans le temps de réponse global de la fonction. Ce délai est appelé **démarrage à froid (cold start)**; parce que lorsque de nouveaux appels se produisent, toutes les étapes précédentes doivent être répétées [42].

Facteurs contribuant aux cold starts [43]

On distingue ici :

- **Le langage de programmation** : les langages de programmation peuvent être compilés ou interprétés. Les langages compilés, comme Java et C, nécessitent la mise en place d'un environnement d'exécution spécifique (comme une VM Java) avant l'exécution de la fonction. Cela peut entraîner des délais plus élevés par rapport aux langages interprétés, comme JavaScript, qui peuvent être exécutés plus rapidement car ils n'ont pas besoin de cette préparation.

- **La taille du package de déploiement** : lorsqu'une fonction cloud est déployée, elle est généralement emballée dans un package contenant le code de la fonction ainsi que ses dépendances. Plus ce package est volumineux, plus il faudra du temps pour le transférer vers le serveur, le charger en mémoire et le préparer pour l'exécution.
- **La taille de la mémoire/ les paramètres CPU** : La taille de la mémoire allouée à une fonction cloud ainsi que les paramètres CPU déterminent les ressources disponibles pour exécuter la fonction. Les fonctions avec plus de mémoire et de puissance CPU peuvent être chargées plus rapidement dans le conteneur d'exécution.
- **Le nombre de dépendances** : les fonctions peuvent dépendre de bibliothèques et de modules externes. Le chargement de ces dépendances peut prendre du temps lors du démarrage d'une fonction : plus il y a de dépendances à charger, plus le temps de démarrage peut être prolongé.
- **Le niveau de concurrence** : on fait référence ici aux nombres de requêtes ou d'appels de fonction effectués simultanément. Chaque requête entraîne le démarrage d'un nouveau conteneur pour exécuter la fonction. Un niveau de concurrence élevé peut conduire à un plus grand nombre de conteneurs à démarrer simultanément, ce qui peut affecter la disponibilité des ressources.
- **Les exécutions précédentes** : certaines plates-formes FaaS peuvent conserver des conteneurs en cours d'exécution pour éviter les cold starts fréquents. Cependant, si un conteneur est arrêté pour économiser des ressources, la prochaine exécution de la fonction entraînera un cold start. Le comportement dépend donc de la politique d'optimisation du fournisseur.

Impact sur les performances : [44]

On distingue ici :

- **Temps de réponse accru** : lorsqu'une fonction subit un cold start, le temps nécessaire pour son exécution initiale peut être plus long que pour un warm start. Cela peut être particulièrement problématique pour les applications nécessitant des réponses rapides [41].
- **Latence des applications** : Le délai introduit par le cold start peut varier de quelques centaines de millisecondes à plusieurs secondes, ce qui peut être inacceptable pour les applications en temps réel ou les services nécessitant une réponse rapide. Les utilisateurs peuvent percevoir cette latence comme une lenteur ou une inefficacité de l'application, affectant leur expérience et potentiellement leur satisfaction.

- **Variabilité des performances** : en raison de la nature aléatoire de l'allocation des ressources et des mécanismes de planification, les cold starts peuvent introduire une variabilité dans les performances des fonctions. Cette variabilité peut rendre difficile la prédiction et la garantie des performances constantes.
- **Utilisation inefficace des ressources** : les ressources allouées aux cold starts sont utilisées principalement pour l'initialisation et la configuration, ce qui peut être considéré comme une utilisation inefficace des ressources, en particulier lorsque de nombreuses fonctions subissent des cold starts simultanément.
- **Coûts supplémentaires** : les cold starts peuvent entraîner des coûts supplémentaires, car le temps d'exécution et l'utilisation des ressources sont plus élevés lors des phases de démarrage. Cela peut affecter la rentabilité des modèles de facturation basés sur l'utilisation.

En comprenant ces impacts, il devient évident que la résolution du problème du cold start est cruciale pour garantir des performances optimales et une expérience utilisateur satisfaisante dans les architectures FaaS. Les stratégies d'atténuation et les solutions pour minimiser les cold starts jouent un rôle clé dans la réalisation de cet objectif.

II.3. Techniques de prédiction des charges de travail

La prédiction des charges de travail est cruciale pour anticiper les besoins en ressources et minimiser les latences associées aux cold starts. Diverses approches ont été proposées pour améliorer la précision des prédictions dans les environnements FaaS.

II.3.1. Solution de Nguyen et al. [1]

La méthodologie adoptée par Nguyen repose les capacités du TCN à prédire les arrivées des instances de fonctions 10 à 15 minutes à l'avance, Nguyen propose une politique d'ensemble divisée en deux chemins parallèles (A et B). Le chemin A utilise les prévisions des noms des instances pour anticiper les demandes futures en infrastructure et provisionner les ressources nécessaires. Le chemin B analyse les codes des fonctions basés sur les temps d'arrivée prédits, optimise les dépendances des codes et priorise les instances en fonction des ressources disponibles et des besoins futurs.

Le modèle utilise le jeu de données Azure Functions Trace 2019, qui contient des informations collectées sur 14 jours à partir de l'environnement Microsoft Azure Function FaaS.

L'exploration des données menée a montré que :

- La majorité des temps d'exécution des fonctions sont égaux ou inférieurs au temps nécessaire pour initialiser les ressources requises, soulignant le problème du cold-start.
- Moins de 20% des applications sont responsables de 99,6% de toutes les invocations, indiquant une grande variation dans les invocations de fonctions parmi les applications.
- Le temps moyen entre les arrivées de fonctions varie considérablement entre les applications.
- Les déclencheurs HTTP jouent un rôle crucial, avec 64% des applications ayant au moins une fonction déclenchée par HTTP et 43% des applications ayant uniquement des fonctions déclenchées par HTTP.
- La mémoire est un facteur important influençant les décisions de warm-up, d'allocation et de maintien des fonctions.

La méthodologie vise à atteindre deux principaux objectifs de prévision :

1. *Prédire les noms des instances de fonctions HTTP-triggered qui seront invoquées ;*
2. *Prédire le moment où ces instances seront invoquées.*

Ces prévisions sont cruciales pour gérer efficacement les ressources et minimiser les temps de démarrage à froid.

L'architecture du TCN de Nguyen est conçue pour capturer les séquences temporelles et effectuer des prévisions précises. Elle est définie par de multiples couches à savoir :

- La couche pré-mix qui a pour fonction de mélanger les données d'entrée en une série de signaux. Ces données incluent les arrivées historiques des instances de fonctions déclenchées par HTTP, le temps d'exécution moyen pour chaque instance de fonction et le nombre d'instances provisionnées de la même fonction.
- Le stack de couches de convolution dilatées : chaque couche de convolution dilatée traite les signaux mélangés, permettant de capturer les relations temporelles à différentes échelles sans augmenter considérablement la taille du réseau. Le réseau est organisé en blocs, chaque bloc ayant une dilatation exponentielle (2, 4, 8, etc.), ce qui permet au modèle de voir plus loin dans le passé à chaque couche supérieure.

- Les couches résiduelles : chaque bloc contient plusieurs cellules résiduelles. Une cellule résiduelle comporte deux couches de convolution causale, suivies de normalisation et d'une activation non linéaire (ReLU). Les sorties des cellules résiduelles sont combinées avec leurs entrées initiales, facilitant ainsi l'apprentissage de caractéristiques complexes tout en stabilisant le processus d'entraînement.
- La couche de prévision coalesce les signaux de sortie des couches de convolution pour générer les prévisions. Cette couche génère les prédictions des noms des fonctions (module A) et de leurs temps d'arrivée (module B).

Pipeline TCN : l'architecture du TCN est organisée en deux modules : Module A et Module B. Le Module A prédit les noms des futures instances de fonctions, tandis que le Module B prédit le temps d'arrivée de ces instances. Les prévisions du Module A sont utilisées comme entrées pour le Module B, ce qui facilite la gestion proactive des ressources.

Les résultats obtenus à partir de l'implémentation du modèle TCN de Nguyen ont été évalués de manière exhaustive en utilisant divers métriques appropriées pour les tâches de prévision des séries temporelles. Les principales métriques d'évaluation incluent la variance expliquée, l'erreur absolue moyenne en pourcentage (MAPE), l'erreur quadratique moyenne (RMSE) et le coefficient de détermination (R2).

Performance des Modèles :

- **Variance expliquée :** représente la diminution en pourcentage de la variance des données originales par rapport à la variance des erreurs. Plus la variance expliquée est proche de 1, meilleure est la performance du modèle.
- **MAPE :** mesure la différence moyenne entre une valeur prédite et la valeur réelle. Plus le MAPE est proche de 0, meilleure est la précision du modèle.
- **RMSE :** représente la racine carrée de la différence quadratique entre une valeur prédite et la valeur réelle. Un RMSE normalisé (divisé par l'étendue des données) proche de 0 indique une meilleure performance.
- **R2 Score :** décrit la diminution de l'erreur quadratique moyenne par rapport à la variance totale des données. Un R2 proche de 1 indique une bonne performance.

Le modèle TCN a été comparé à d'autres algorithmes de machine learning populaires pour les séries temporelles. Pour la prédiction des noms des fonctions (module A), le modèle TCN s'est avéré compétitif avec les meilleurs modèles non basés sur l'apprentissage profond (comme Prophet), malgré les limitations de ressources rencontrées lors de l'entraînement. Pour la prédiction des temps d'arrivée

des instances de fonctions (module B), le TCN a surpassé les modèles non basés sur l'apprentissage profond. Ses différentes performances peuvent être observées sur les tableaux 1 et 2.

En résumé, le modèle TCN de Nguyen offre une solution robuste et performante pour gérer le problème du cold-start dans les environnements cloud serverless. Grâce à une méthodologie rigoureuse, une architecture de modèle innovante et une évaluation détaillée, cette étude démontre le potentiel des TCN pour améliorer significativement la gestion des ressources et la performance des services cloud.

TABLE I – Résultat des performances : module A

Metric	TCN	Arima	ES ^a	Prophet
Explained variance	-0.002	-0.002	~0	0.012
Mean absolute percentage error	1968	136	126	142
Normalized RMSE	0.200	0.238	0.286	0.220
R2 score	-0.05	-0.29	-0.62	-0.12
Spearman Correlation	-0.042	0.078	-1	0.15

^a ES : Exponential Smoothing.

TABLE II – Résultat des performances : module B

Metric	TCN	Arimax	Arima	ES ^a	Prophet
Explained variance	0.50	0.32	0.20	~0	-0.002
Mean absolute percentage error	15.79	230	210	291	252
Normalized RMSE ^a	0.16	0.18	0.20	0.27	0.27
R2 score	-0.87	-0.39	-0.59	-0.80	-0.80
Spearman Correlation	0.92	0.32	0.32	-1	-0.05

^a ES : Exponential Smoothing.

II.3.2. Solution de Nelvin et al. [2]

L'étude de Nevlin et al. sur la prédiction des charges de travail dans des environnements serverless utilise une approche d'apprentissage automatique pour analyser les données historiques des charges de travail et identifier des motifs et des tendances qui permettent de prédire les besoins futurs en ressources.

Les données historiques des charges de travail serverless, extraites d'un sous-ensemble du dataset Azure sont utilisées [45]. Les attributs moyens de la mémoire allouée sont particulièrement pris en compte. Les données sont ensuite divisées en ensembles d'entraînement (80%) et de test (20%). Trois modèles d'apprentissage

automatique sont sélectionnés pour l'expérimentation : ARIMA, LSTM et VAR. Ces modèles sont choisis en raison de leur capacité à capturer différentes dynamiques dans les séries temporelles des charges de travail.

- **ARIMA** : adapté pour les séries temporelles stationnaires et utilisé pour modéliser les données en capturant les relations temporelles.
- **LSTM** : particulièrement utile pour les séries temporelles présentant des tendances et des motifs complexes.
- **VAR** : modèle multivarié de séries temporelles qui prend en compte plusieurs variables temporelles interconnectées. Il permet de modéliser les dépendances entre ces variables, ce qui est crucial pour comprendre les interactions dans les données de charges de travail serverless.

Chaque modèle est entraîné en utilisant l'ensemble d'entraînement. Les performances des modèles sont évaluées en utilisant l'ensemble de test. La principale métrique utilisée est le RMSE, qui mesure la différence moyenne entre les valeurs prédites et les valeurs réelles.

Architecture des Modèles

- **ARIMA** : se compose de trois composantes principales : la composante autoregressive (AR), la composante de différence intégrée (I), et la composante de moyenne mobile (MA). L'analyse de l'auto-corrélation dans les données montre que chaque observation est indépendante des autres, indiquant une association faible ou inexistante entre les variables à différents décalages.
- **LSTM** : constitué de plusieurs couches : une couche LSTM de 64 unités qui traite les séquences d'entrée, capturant les dépendances à long terme dans les données de charges de travail. Une couche dense de 32 unités, qui applique une transformation linéaire aux sorties de la couche LSTM, utilisant la fonction d'activation ReLU pour introduire la non-linéarité. Une couche Dropout avec un taux de 0,2, qui applique une régularisation pour éviter le surapprentissage. Une couche dense finale qui comprend une unité et produit la prédiction finale. Le modèle LSTM compte un total de 19 009 paramètres entraînaables.
- **VAR** : composé de 10 équations avec chaque équation ayant 4 colonnes représentant le coefficient, l'erreur standard, la statistique t, et la probabilité. Ce modèle est capable de capturer les relations entre plusieurs variables temporelles, permettant une modélisation plus riche des interactions complexes dans les données de charges de travail.

Les résultats expérimentaux montrent des performances variées pour les trois modèles testés :

1. **ARIMA** : malgré sa simplicité, montre une capacité modérée à prédire les charges de travail serverless avec un RMSE de 104.697.
2. **LSTM** : performe le mieux parmi les trois modèles testés. Avec un RMSE de 6.967450623763335.
3. **VAR** : bien qu'ayant une capacité à modéliser des relations multivariées, affiche des performances moins efficaces avec un RMSE de 32494.192745620116.

En conclusion, cette étude met en lumière l'efficacité des techniques d'apprentissage automatique pour la prédiction des charges de travail dans des environnements serverless. Le modèle LSTM, en particulier, se distingue par sa capacité à capturer les motifs complexes et à fournir des prédictions précises, permettant une optimisation proactive des ressources et une réduction des coûts. Ces résultats ouvrent la voie à des améliorations significatives dans la gestion des ressources pour les applications serverless, bien que des défis persistent, notamment en ce qui concerne la nature dynamique et imprévisible des charges de travail serverless et le traitement des données incomplètes ou bruitées. Avec la croissance continue de l'adoption de l'informatique serverless, cette recherche a le potentiel de transformer les pratiques de déploiement et de gestion des applications dans le cloud.

II.4. Stratégies d'atténuation du cold start

Ces stratégies peuvent être classées en deux catégories principales : **la réduction du délai de cold start et la réduction de la fréquence de cold start**. L'atténuation du cold start est un aspect crucial de l'optimisation des performances dans les architectures FaaS.

II.4.1. Techniques de réduction du délai de cold start

Les techniques visant à réduire le délai de cold start se concentrent sur l'optimisation du temps nécessaire pour initialiser une fonction. Plusieurs techniques peuvent être employées pour atteindre cet objectif :

La plateforme OpenWhisk [46] fait usage des conteneurs de cellules souches préchauffés pour catégoriser les conteneurs en fonction de la mémoire et du langage de programmation, réduisant ainsi le temps de préparation du conteneur et ainsi le délai de démarrage à froid. Par défaut, elle met à disposition deux conteneurs avec l'environnement d'exécution Node.js. Cela permet une allocation rapide de conteneurs préchauffés lorsqu'une demande est reçue. Toutefois, il y a dépendance à l'environnement Node.js par défaut et la nécessité de maintenir une

certaine quantité de ressources préchauffées en attente, ce qui peut causer un gaspillage de ressources.

Des nombreux outils existants, tels que CloudWatch [23], Dashbird.io [47] et Lambda Warmer [48] sont également utilisés pour définir des règles d'invocation périodique des fonctions et les chauffer des conteneurs à des intervalles réguliers, indépendamment de la demande réelle. Cette approche ne tient pas compte des fluctuations de la demande et peut entraîner un gaspillage de ressources en cas d'inactivité prolongée ou de rafales soudaines de demandes

Liu et al.. [49] propose FaaSLight, une approche basée sur l'optimisation d'application, qui ne va charger que la partie principale du code de l'application pour réduire la latence de démarrage à froid. Ce qui pourra la réduire en chargeant uniquement les parties essentielles de l'application, mais elle peut ne pas être applicable à toutes les applications, en particulier celles avec des structures complexes ou des dépendances importantes.

Lee et al.. [43] ont proposé une méthode basée sur la fusion de fonctions pour réduire la latence de démarrage à froid. Si deux fonctions séquentielles sont combinées en une seule fonction, le démarrage à froid de la deuxième fonction est éliminé et il n'y a qu'un seul délai de démarrage à froid. Dans les opérations basées sur la fusion, même si le démarrage à froid de la deuxième fonction est éliminé lorsque des fonctions parallèles sont combinées, dans l'informatique sans serveur, le temps de traitement peut augmenter car les fonctions sont exécutées séquentiellement.

Jegannathan et al.. [38] proposent SARIMA, un modèle de prévision de séries temporelles basé sur l'apprentissage, nommé Prediction Based Autoscaler (PBA), qui prédit le nombre de futures invocations de fonctions et alloue les ressources informatiques en conséquence, ce qui permet de démarrer les conteneurs avant un trafic élevé d'invocations de fonctions. Cela aide également à réduire le nombre de ressources qui auraient été gaspillées en l'absence de besoins de mise à l'échelle. Cela permet d'éviter des gaspillages de ressources en ajustant dynamiquement l'allocation de ressources, mais nécessite des données précises pour la prédiction, ce qui peut être difficile à obtenir dans des environnements changeants.

II.4.2. Techniques de réduction de la fréquence de cold start

Réduire la fréquence des cold starts implique de maintenir des instances de fonctions en mémoire plus longtemps. Voici quelques-unes des stratégies employées :

De nombreuses plateformes telles que Google Cloud Functions (GCF) [50], AWS Lambda [23], Microsoft Azure, [51] Open Lambda [52] et OpenWhisk [46], mettent en pause les conteneurs et leurs ressources pendant quelques minutes après

l'exécution d'une fonction. Cela permet la réutilisation des conteneurs si une demande est reçue pendant cette période. Sur OpenWhisk, par exemple, le conteneur reste actif pendant 50 ms après une exécution, puis est mis en pause pendant 10 minutes avant d'être libéré. Cela implique moins de délai pour redémarrer un conteneur en pause que pour allouer un nouveau conteneur, mais il y a absence de seuil pour libérer les conteneurs après l'exécution, et cela peut entraîner une utilisation inefficace des ressources et des coûts supplémentaires.

Également, la plupart des plateformes commerciales telles qu'AWS et GCF maintiennent les conteneurs en vie pendant une certaine période après l'exécution de la fonction. Pendant cette période, les demandes qui peuvent arriver sur le serveur sont attribuées à ces conteneurs qui sont maintenus en vie, et ainsi il est prévu de prévenir l'occurrence d'un démarrage à froid. Cependant, cette politique gaspille des ressources car elle ne peut pas détecter le moment exact de l'occurrence du démarrage à froid.

Agarwal et al.. [53, 54] ont utilisé l'algorithme Q-learning pour déterminer dynamiquement le nombre d'instances de fonction. Le nombre d'instances de fonction disponibles et la consommation de CPU discrétisée de chaque fonction sont considérés comme des états environnementaux, et l'action est augmentée ou diminuée des instances de fonction. Toutefois, on dénote une complexité de son environnement.

Shahrad et al.. [12] ont présenté une politique de gestion des ressources appelée Hybrid Histogram Policy qui détermine la valeur optimale de la fenêtre de conteneur inactif en fonction des caractéristiques des fonctions.

Kumari et al.. [3] ont proposé une nouvelle méthode basée sur des conteneurs chauds. En utilisant des réseaux de neurones profonds et des modèles LSTM, le nombre de conteneurs chauds et les durées de maintien des conteneurs sont déterminés. Ainsi, ils essaient de suggérer une approche basée sur la gestion des ressources pour éviter un démarrage à froid réduisant ainsi le délai et la fréquence de cold start.

Dans une autre étude similaire à celle de Kumari, Vahidinia et al. [4] utilisent une méthode d'apprentissage par renforcement et des modèles LSTM pour déterminer le nombre de conteneurs chauds et le temps pendant lequel ils sont maintenus au chaud. Toutefois ils travaillent des données générées avec un manque de variété dans leurs échantillons de tests

II.4.3. Quelques approches basées sur le deep learning pour l'atténuation du cold start

II.4.3.1. Solution de Kumari et al. [3]

Kumari et al.. présentent un modèle adaptatif visant à prédire la durée de la fenêtre du conteneur inactif et le nombre requis de conteneurs préchauffés à l'avance pour atténuer le démarrage à froid. Ils ont utilisé un réseau neuronal profond et un modèle LSTM pour capturer les schémas d'invocation de fonction précédents, et déterminer préalablement la durée de la fenêtre du conteneur inactif. La méthodologie adoptée en deux phases, est à la fois complexe et méthodique, exploitant une approche avec des techniques de modélisation adaptatives basées sur des DNN et des LSTM.

Le DNN est utilisé dans la première phase pour prédire la durée optimale de la fenêtre du conteneur inactif. Son architecture est la suivante : une couche d'entrée qui reçoit les données d'entrée ; le DNN comprend plusieurs couches cachées, ces couches cachées permettent au modèle d'apprendre des représentations de plus en plus abstraites des données d'entrée ; les fonctions d'activation, telles que ReLU, sont utilisées pour introduire des non-linéarités dans le modèle et lui permettre de capturer des relations complexes entre les variables ; la couche de sortie génère les prédictions finales de la durée de la fenêtre du conteneur inactif. La structure du réseau neuronal comprend quatre couches cachées, deux couches de sortie et une couche d'entrée. Chacune des couches cachées comprend 32 neurones et une fonction d'activation ReLU. Une fonction d'activation Softplus est utilisée à la couche de sortie pour la normalisation.

L'architecture du modèle LSTM utilisé dans la deuxième phase est la suivante : il comprend plusieurs couches LSTM empilées les unes sur les autres. Chaque couche LSTM est conçue pour capturer les dépendances à long terme dans les séquences de données. Comme pour le DNN, les fonctions d'activation, telles que ReLU, sont utilisées dans les couches LSTM pour introduire des non-linéarités dans le modèle. La couche de sortie du modèle LSTM génère les prédictions finales du nombre d'invocations concurrentes. La structure du modèle LSTM comprend cinq couches cachées, chacune ayant 32 neurones. Une fonction d'activation ReLU est utilisée à chaque couche.

En combinant ces deux architectures de modèles, Kumari et al.. ont développé un cadre complet et adaptatif pour atténuer les retards de démarrage à froid dans les environnements sans serveur, en exploitant les avantages des réseaux neuronaux

profonds et des modèles LSTM pour modéliser efficacement les modèles d'invocation des fonctions et optimiser l'allocation des ressources.

Phase I- réduction de la fréquence du démarrage à froid : l'objectif est de prédire la durée de la fenêtre du conteneur inactif, qui peut varier dynamiquement en fonction du modèle d'invocation d'une application. Cette phase comprend plusieurs étapes clés :

- Extraction du modèle d'invocation : le modèle d'invocation des fonctions enregistré dans les journaux de la base de données est extrait, ce modèle comprend des informations telles que les intervalles de temps entre les invocations et le statut de chaque fonction : froid ou chaud.
- Modélisation avec un DNN : un réseau neuronal profond est utilisé pour traiter ces informations et prédire la durée optimale de la fenêtre du conteneur inactif, ce réseau est formé pour équilibrer la fréquence du démarrage à froid et la consommation de ressources, en minimisant une fonction de coût qui tient compte de ces deux aspects.
- Optimisation de la durée de la fenêtre du conteneur inactif : le modèle de DNN fournit des estimations de la durée de la fenêtre du conteneur inactif en fonction du modèle d'invocation observé.
- Cette durée est ensuite mise à jour dans le système via un module de configuration intégré à la plateforme sans serveur.

Phase II- Réduction du délai de démarrage à froid : l'objectif est de préparer les conteneurs préchauffés avant l'invocation des fonctions, afin de réduire le délai de démarrage à froid. Cette phase comprend également plusieurs étapes importantes :

- Prédiction du nombre d'invocations concurrentes : prédire le nombre d'invocations concurrentes qui peuvent être exécutées simultanément sur une période donnée. Pour ce faire, un modèle LSTM est utilisé pour capturer les dépendances entre les modèles de demande d'invocation passés et le nombre de retards de démarrage à froid. Ce modèle LSTM est formé sur les données historiques d'invocation des fonctions.
- Allocation de conteneurs préchauffés : en utilisant les informations prédites sur le nombre d'invocations concurrentes, un nombre approprié de conteneurs préchauffés est préparé à l'avance. Cela garantit que les conteneurs sont prêts à répondre aux demandes d'invocation sans subir de délai de démarrage à froid.

La méthodologie est évaluée à l'aide de quatre applications réelles (send-emails, Image-resizer, Serverless-form-handler et Chromium) sans serveur différentes,

déployées sur plusieurs plateformes (AWS Lambda, Microsoft Azure, Openfaas et OpenWhisk). Les performances sont mesurées en termes de temps de réponse global, de délai de démarrage à froid et de fréquence du démarrage à froid, et comparées avec les stratégies de mitigation existantes. La plateforme utilisée pour leur évaluation était OpenWhisk, avec l'intégration des modèles de ML proposés pour atténuer les cold start. Les performances du modèle adaptatif proposé par Kumari et al. ont été évaluées selon plusieurs paramètres clés, à savoir : **le temps de réponse, le délai de démarrage à froid et la fréquence du retard de démarrage à froid.**

Temps de réponse : il a été mesuré pour évaluer l'efficacité du modèle adaptatif. Les résultats de cette évaluation montrent une amélioration significative du temps de réponse avec l'utilisation du modèle adaptatif par rapport aux autres stratégies existantes. Par exemple, dans l'application de redimensionnement d'image, le modèle adaptatif a enregistré une amélioration moyenne du temps de réponse d'environ 42%, tandis que les autres plateformes telles qu'AWS Lambda, Microsoft Azure, Openfaas et OpenWhisk ont enregistré des améliorations moyennes du temps de réponse de 13%, 16%, 12% et 15% respectivement.

Fréquence du retard de démarrage à froid : représente le nombre d'occurrences de démarrages à froid sur une période donnée, a également été prise en compte. Le modèle adaptatif a permis de réduire de manière significative la fréquence des démarrages à froid par rapport aux autres plateformes. Par exemple, dans une expérience où 100 demandes ont été émises sur une période de temps définie, le modèle adaptatif a enregistré une réduction substantielle du nombre de démarrages à froid par rapport aux autres plateformes évaluées, comme on peut l'observer sur le tableau 3.

TABLE III – Résultat des performances : nombre de cold start

	Send-email-ses	Image resizer	Serverless-form-h	Chromium
AWS Lambda	70	67	77	73
Microsoft Azure	79	59	74	67
Openfaas	63	48	66	56
OpenWhisk	72	38	54	50
Adaptive model	40	29	40	43

En résumé, les résultats de l'évaluation des performances montrent que le modèle adaptatif proposé par Kumari et al. offre une amélioration significative du temps de réponse, du délai de démarrage à froid et de la fréquence des démarrages à froid par rapport aux autres stratégies existantes dans les environnements

de cloud sans serveur. Ces résultats démontrent l'efficacité et la pertinence de leur approche pour améliorer l'expérience utilisateur et l'efficacité opérationnelle dans ces environnements critiques.

Bien que leur évaluation ait montré des améliorations significatives dans plusieurs cas d'utilisation spécifiques, l'efficacité de leur approche pourrait varier dans différents scénarios d'application. Des conditions de charge de travail spécifiques pourraient nécessiter des ajustements supplémentaires ou une révision de l'approche proposée.

II.4.3.2. Solution de Vahidinia et al. [4]

Vahidinia et al.. proposent une approche en deux étapes également pour atténuer le cold start. Leur méthode utilise l'apprentissage par renforcement et les LSTM, pour ajuster dynamiquement la fenêtre de conteneurs inactifs et prédire le nombre optimal de conteneurs prérechauffés nécessaires pour répondre à la charge de travail.

La méthodologie utilisée dans l'approche de Vahidinia et al.. pour atténuer le problème du démarrage à froid dans les environnements de cloud sans serveur est basée sur deux étapes principales : **la réduction de l'occurrence du démarrage à froid et la réduction du retard de démarrage à froid.**

Phase I- Réduction de l'occurrence du démarrage à froid : pour apprendre à ajuster dynamiquement ils font usage d'un algorithme acteur-critique à avantage basé sur les différences temporelles (TD), qui est un algorithme de RL adapté aux environnements avec des états et des actions continus. L'agent intelligent observe les journaux d'exécution des fonctions pour déterminer l'état de l'environnement, y compris les intervalles de temps entre les invocations et le type de démarrage (froid ou chaud). L'agent sélectionne une action appropriée pour ajuster la fenêtre de conteneurs inactifs, cherchant à minimiser à la fois le nombre de démarrages à froid et la consommation de mémoire. Les récompenses sont basées sur la réduction des retards de démarrage à froid et la gestion efficace de la consommation de mémoire. L'agent reçoit des récompenses positives pour des actions qui réduisent les retards de démarrage à froid et des récompenses négatives pour des actions qui augmentent la consommation de mémoire sans réduire les retards de démarrage à froid.

Phase I- Réduction du retard de démarrage à froid : utilisation de réseaux LSTM : Pour prédire le nombre maximal d'invocations concurrentes sur une période donnée, Vahidinia et al.. utilisent des réseaux de neurones LSTM,

qui sont capables de capturer les relations temporelles dans les séquences de données. Prédiction du nombre d'invocations futures : Le réseau LSTM est formé à partir de données historiques d'invocation des fonctions cloud, en utilisant les intervalles de temps entre les invocations pour prédire les invocations futures. Cette prédiction permet de déterminer le nombre optimal de conteneurs prérechauffés nécessaires pour répondre à la charge de travail attendue.

Dans la première couche de l'approche de Vahidinia et al., l'algorithme de RL est utilisé pour déterminer la fenêtre du conteneur inactif. Le réseau neuronal de l'acteur comporte trois couches cachées, chacune avec 32 neurones, une fonction d'activation ReLU, et une fonction de poids uniforme. Ce réseau comprend également deux couches de sortie pour (la moyenne) et (l'écart type), avec une fonction d'activation Softplus. Le réseau neuronal du critique inclut deux couches avec 32 et 16 neurones et une fonction d'activation ReLU, avec une couche de sortie utilisant une fonction d'activation linéaire pour calculer la valeur de l'état suivant. Pour la deuxième couche de l'approche, le réseau LSTM (Long Short-Term Memory) est utilisé. Ce réseau comporte cinq couches cachées, chacune avec 32 neurones et une fonction d'activation ReLU. Pour éviter le surajustement, une couche de "Dropout" avec une valeur de 0,5 est utilisée. Le réseau dispose également d'une seule sortie avec une fonction d'activation linéaire. L'erreur quadratique moyenne (MSE) est utilisée comme fonction de perte pour entraîner ce réseau.

Pour évaluer la première couche un dataset contenant des invocations séquentielles est requis. Les invocations sont générées selon un processus de Poisson, 10 000 temps d'entrée avec des taux moyens d'entrée d'invocations de 5, 10 et 20 invocations par heure. Ces temps d'entrée représentent l'intervalle de temps entre les invocations et ont été utilisés pour entraîner et évaluer la première couche de leur modèle. Pour évaluer la deuxième couche de l'approche, un dataset contenant des invocations concurrentes est requis, 10 000 invocations générées de manière aléatoire, y compris des invocations concurrentes.

La plateforme OpenWhisk est mise en œuvre sur un serveur utilisant Ubuntu comme système d'exploitation, et Kubernetes est utilisé pour gérer l'exécution de conteneurs et la gestion des ressources sur ce serveur. La configuration des paramètres correspondant à l'approche proposée sur la plateforme est essentielle.

Deux simulations ont été effectuées :

1. En utilisant de la plateforme OpenWhisk avec ses paramètres par défaut.
2. En configurant de nouveaux paramètres pour OpenWhisk obtenus à partir de l'approche proposée.

L'approche proposée a **permis d'adapter dynamiquement la fenêtre du conteneur inactif en fonction des schémas d'invocation, ce qui a conduit à une diminution notable du nombre d'occurrences de démarrage à froid.**

Variation de la fenêtre du conteneur Inactif :

- Pour un taux moyen d'invocations par heure de 5 : La fenêtre du conteneur inactif a varié de 3 à 13 minutes selon l'approche proposée, tandis qu'elle était fixée à 10 minutes pour la plateforme OpenWhisk. Environ 73% des invocations ont eu une fenêtre du conteneur inactif inférieure à 10 minutes avec l'approche proposée, contre 67% avec OpenWhisk.
- Pour un taux moyen d'invocations par heure de 10 : La fenêtre du conteneur inactif a varié de 7 à 15 minutes avec l'approche proposée, alors qu'elle restait fixée à 10 minutes pour OpenWhisk. Environ 67% des invocations ont eu une fenêtre du conteneur inactif inférieure à 10 minutes avec l'approche proposée, comparé à OpenWhisk.
- Pour un taux moyen d'invocations par heure de 20 : La fenêtre du conteneur inactif a varié de 8 à 13 minutes selon l'approche proposée, contre une fenêtre fixe de 10 minutes pour OpenWhisk. Environ 76% des invocations ont eu une fenêtre du conteneur inactif inférieure à 10 minutes avec l'approche proposée, contre une valeur fixe pour OpenWhisk.

Nombre d'occurrences de cold start :

- Pour un taux moyen d'invocations par heure de 5 : 123 occurrences de cold start avec l'approche proposée contre 126 avec OpenWhisk (sur un total de 200).
- Pour un taux moyen d'invocations par heure de 10 : 84 occurrences de cold start avec l'approche proposée contre 89 avec OpenWhisk (sur un total de 200).
- Pour un taux moyen d'invocations par heure de 20 : 53 occurrences de cold start avec l'approche proposée contre 55 avec OpenWhisk (sur un total de 200).

Réduction du Délai de cold start :

L'approche a également réussi à prédire avec précision le nombre d'invocations concurrentes, contribuant ainsi à réduire le délai de démarrage à froid. Les résultats détaillés sont les suivants : Sur un total de 106 invocations concurrentes, l'approche a correctement prédit et exécuté 56 invocations sur des conteneurs préchauffés. En comparaison, la plateforme OpenWhisk, qui dispose de deux conteneurs préchauffés en permanence, n'a exécuté que 32 demandes sur des conteneurs préchauffés. Cela démontre une amélioration significative de 22,65% dans l'exécution des in-

vocations sur les conteneurs préchauffés avec l'approche proposée par rapport à OpenWhisk.

Bien que cette approche offre des perspectives intéressantes pour atténuer le problème du démarrage à froid dans les environnements de cloud sans serveur, elle est sujette à plusieurs limitations importantes :

- **Utilisation de deux modèles de ML différents** : l'approche nécessite l'application de deux modèles de machine learning distincts. Cette complexité accrue augmente les exigences en termes de ressources de calcul et de gestion des modèles.
- **Données générées de manière aléatoire** : les données utilisées pour évaluer l'approche ne sont pas nécessairement tirées de schémas d'invocation réels, mais plutôt générées de manière aléatoire. Cela peut introduire des biais dans l'évaluation de la performance de l'approche, car les modèles peuvent ne pas être exposés à la diversité et à la complexité des schémas d'invocation réels présents dans les environnements de production.
- **Sensibilité à la qualité des données d'entraînement** : comme pour tout modèle d'apprentissage automatique, la performance de l'approche dépend de la qualité et de la représentativité des données d'entraînement. La disponibilité de données historiques précises et complètes est essentielle pour permettre aux modèles de généraliser efficacement et de prendre des décisions pertinentes dans des situations nouvelles et complexes.
- **Manque de généralisabilité** : les performances de l'approche peuvent varier selon les caractéristiques spécifiques des environnements cloud et des charges de travail. Ce qui fonctionne bien dans un contexte peut ne pas nécessairement être efficace dans un autre, ce qui limite la capacité de généralisation de l'approche à des scénarios variés.
- **Limitations inhérentes aux modèles utilisés** : Les algorithmes d'apprentissage par renforcement et les réseaux de neurones LSTM ont leurs propres limitations intrinsèques en termes de capacité à représenter et à généraliser à partir des données. Ces limites peuvent affecter la capacité de l'approche à capturer la complexité des schémas d'invocation et à prendre des décisions précises dans des environnements dynamiques et évolutifs.

II.5. Conclusion

Ce chapitre a exploré les différentes méthodes de prédiction des charges de travail et les stratégies pour atténuer le cold start dans les architectures FaaS. Les tech-

niques de prédiction basées sur le deep learning et les méthodes hybrides montrent un potentiel significatif pour améliorer la précision des prévisions, permettant une allocation plus efficace des ressources. De plus, les approches d'atténuation du cold start, qu'elles soient basées sur l'optimisation des délais ou la réduction de la fréquence des cold starts, offrent des solutions pratiques pour améliorer la réactivité des systèmes FaaS. Les solutions basées sur l'intelligence artificielle, en particulier celles utilisant les réseaux de neurones et le reinforcement learning, ont démontré des résultats prometteurs dans la réduction des latences et l'optimisation des performances. Ces avancées permettent de surmonter les défis inhérents aux architectures FaaS, favorisant une adoption plus large et plus efficace de ces technologies dans divers domaines d'application.

III

CHAPITRE

CONTRIBUTION À LA PRÉDICTION DES CHARGES DE TRAVAIL DANS LES ARCHITECTURES FAAS

CONTENU

III.1 - Introduction	58
III.2 - Méthodologie proposée : modèle hybride basé sur les CNN-LSTM	59
III.3 - Architecture du modèle de prédiction	62
III.4 - Évaluation des performances du modèle	64
III.5 - Conclusion	67

III.1. Introduction

Les architectures FaaS apportent une évolution significative dans le domaine du CC. Toutefois, on a pu relever parmi les challenges de ce type d'architecture : la gestion des charges de travail. Une mauvaise prévision de la charge peut causer des temps de réponse plus longs, une utilisation inefficace des ressources ou encore entraîner des coûts excessifs. Par conséquent, une prédiction précise de la charge de travail est cruciale pour une meilleure optimisation des performances et des coûts. Le travail mené par Nevlin et al. [2] a mis en évidence une approche remarquable basée sur les séries temporelles et diverses techniques de machine learning afin de prédire de façon optimale la quantité de mémoire moyenne nécessaire au bon fonctionnement des applications FaaS. C'est en s'inspirant de ses différentes recherches que nous avons eu l'idée d'envisager une approche encore plus performante.

Dans ce chapitre, nous présenterons notre contribution à la prédiction des charges de travail dans les architectures FaaS. Nous proposerons une méthodologie efficace

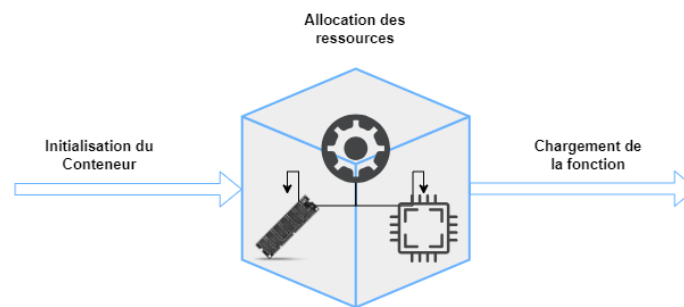


FIGURE 14 – Allocation des ressources d'une fonction

qui fait usage d'une architecture de modèle de prédiction combinant les CNN et les LSTM; nous allons décrire en détail la méthodologie proposée, l'architecture de notre modèle, les données utilisées pour son entraînement et son évaluation, et enfin nous analyserons les différents résultats obtenus.

III.2. Méthodologie proposée : modèle hybride basé sur les CNN-LSTM

La prédiction des charges de travail dans les environnements FaaS présente des défis en raison de la nature élastique et dynamique de ces architectures. Pour assurer une allocation optimale des ressources et une performance constante (*voir figure 14*), il est impératif de développer des modèles capables de prédire avec précision les variations des charges de travail.

Notre méthodologie repose sur l'utilisation d'un modèle hybride qui combine les CNN et les LSTM. Cette approche permet de tirer parti des avantages de ces deux types de réseaux pour capturer à la fois les caractéristiques locales et les dépendances dans nos données.

Réseaux de Neurones Convolutifs (CNN)

Les CNN sont largement utilisés pour leur capacité à extraire des caractéristiques locales (variations, tendances, cycles, etc.) [55] à partir de données structurées. Initialement développés pour l'analyse d'images, les CNN ont prouvé leur efficacité dans diverses applications, y compris l'analyse de séries temporelles. Pour effectuer la prédiction des charges de travail dans un environnement serverless, les CNN jouent un rôle important dans l'extraction de motifs récurrents et de tendances locales à partir des séries de données [56]. Les CNN fonctionnent sur des séries temporelles univariées de cette façon [57] :

- **Convolutions 1D** : les convolutions unidimensionnelles sont adaptées aux séries temporelles univariées [58]. Un filtre de convolution (noyau) glisse le long de la séquence de données, et réalise des opérations de convolution afin d'extraire les motifs locaux.
- **Strides et Padding** : les strides déterminent le pas avec lequel le filtre se déplace tout le long d'une séquence. Le padding permet de contrôler la taille des cartes de caractéristiques en ajoutant des bordures à la séquence, assurant que les convolutions couvrent entièrement les données d'entrée. Le choix des strides et du padding est important pour conserver des informations pertinentes tout en réduisant la dimensionnalité.
- **Activation ReLU** : après chaque opération de convolution, une fonction d'activation non linéaire telle que ReLU est appliquée dans le but d'introduire de la non-linéarité dans le modèle, et de capturer des motifs encore plus complexes.
- **Pooling** : les couches de pooling, comme le max-pooling, sont utilisées pour réduire la dimensionnalité des cartes de caractéristiques. Cela aide à minimiser le surapprentissage et améliore l'efficacité computationnelle.

Les CNN se sont montrés particulièrement adaptés pour identifier des motifs locaux et des régularités dans les données [58, 59]. S'agissant des séries temporelles univariées, ces motifs peuvent être : des variations périodiques, des tendances à court terme, ou encore des comportements récurrents. L'application des CNN permet d'automatiser l'extraction de ces caractéristiques, ce qui élimine le besoin de techniques manuelles d'ingénierie des caractéristiques.

Réseaux de Neurones à Mémoire à Long Terme (LSTM)

Les LSTM sont une extension des RNN. Les LSTM sont particulièrement efficaces pour modéliser les dépendances temporelles à long terme dans les séries temporelles. Les LSTM fonctionnent de cette façon :

- **Cellules LSTM** : les LSTM se composent de plusieurs cellules de mémoire capables de conserver des informations sur de longues périodes. Chaque cellule LSTM est composée de trois portes principales : une porte d'entrée, une porte d'oubli et une porte de sortie. Ces portes régulent le flux d'informations et permettent à la cellule de mémoriser ou d'oublier des informations en fonction de leur pertinence.
- **Porte d'entrée** : elle décide quelles nouvelles informations doivent être ajoutées à la cellule de mémoire. Elle utilise une fonction sigmoïde pour

décider quelles valeurs passer, suivie d'une fonction tanh pour réguler le niveau d'importance des nouvelles informations.

- **Porte d'oubli** : elle décide quelles informations de la cellule de mémoire doivent être oubliées. Elle utilise également une fonction sigmoïde pour évaluer l'importance des informations actuelles dans la cellule.
- **Porte de sortie** : elle décide quelles informations de la cellule de mémoire doivent être sorties. La sortie est déterminée par une fonction sigmoïde appliquée à l'état de la cellule, multipliée par une fonction tanh.
- **Modélisation des dépendances temporelles** : les LSTM traitent séquentiellement les éléments de la série temporelle, capturant les relations complexes et les tendances à long terme. Cela permet de modéliser des comportements futurs basés sur des données historiques.

Dans le contexte des architectures FaaS, les LSTM pourront capturer les effets à long terme des variations des charges de travail.

Modèle CNN-LSTM

La combinaison des CNN et des LSTM dans un modèle hybride permet de tirer parti des avantages des deux architectures, offrant ainsi une approche puissante dans le cadre de notre étude. L'utilisation combinée des CNN et des LSTM dans un modèle hybride offre plusieurs avantages [60] parmi lesquels :

- **Une extraction efficace des caractéristiques** : les CNN sont capables d'extraire automatiquement des caractéristiques locales pertinentes à partir des données d'entrée, des motifs locaux complexes, éliminant le besoin de techniques manuelles d'ingénierie des caractéristiques souvent fastidieuses et sujettes aux erreurs. Tandis que les LSTM capturent les tendances temporelles à long terme. Cette combinaison assure une modélisation riche et précise des séries temporelles.
- **Une efficacité computationnelle** : la réduction de la dimensionnalité par les couches de pooling et l'utilisation des LSTM pour modéliser les dépendances temporelles permettent une modélisation rapide et efficace des séries temporelles.
- **De la flexibilité et généralisabilité** : le modèle CNN-LSTM est flexible et peut être appliqué à diverses séries de données, rendant cette méthodologie adaptable à différents types de charges de travail dans les architectures FaaS.
- **De la capture des dépendances temporelles** : Les LSTM sont spécialement conçus pour modéliser les séquences de données où les dépendances temporelles sont importantes. Ils peuvent capturer les effets à long terme

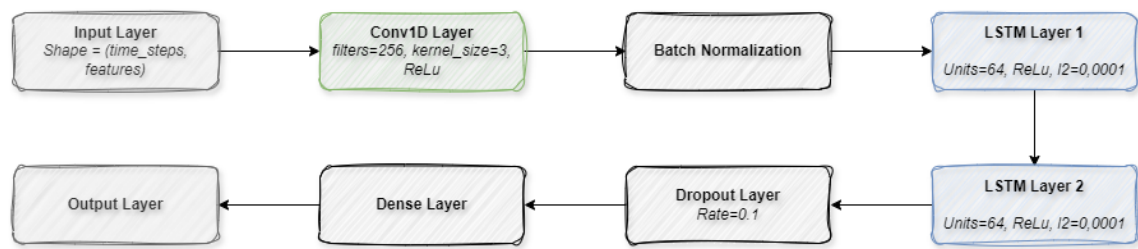


FIGURE 15 – Modèle hybride CNN-LSTM proposé

des variations de la charge de travail, comme les tendances saisonnières, les comportements de pointe, et les périodes de faible demande.

- **De la robustesse et de la précision :** La combinaison des deux types de réseaux permet de bénéficier de la robustesse et de la précision des CNN et des LSTM, offrant ainsi une solution de prédiction plus fiable et précise pour la gestion proactive des ressources dans les environnements FaaS.

Cette méthodologie offre une approche structurée et efficace pour la modélisation des charges de travail des applications serverless, permettant ainsi une prédiction plus précise et une gestion optimisée des ressources dans les environnements FaaS. En combinant les forces des CNN et des LSTM, cette approche assure une modélisation riche et complète des séries temporelles, facilitant ainsi une gestion plus efficace et rentable des ressources dans les architectures serverless.

III.3. Architecture du modèle de prédiction

L'architecture de notre modèle CNN-LSTM se compose de plusieurs couches (observable à la figure 15). Chacune de ces couches jouent un rôle spécifique dans le processus de prédiction :

La couche d'entrée spécifie la forme des données d'entrée. Cette couche ne comporte pas de paramètres et sert uniquement à définir la structure des données d'entrée.

La première couche est une couche convolutive unidimensionnelle (Conv1D). Cette couche applique 256 filtres de convolution sur les données d'entrée avec un noyau de taille 3. La fonction d'activation ReLU (Rectified Linear Unit) est utilisée pour introduire de la non-linéarité dans le modèle, permettant de capturer des motifs complexes dans les données.

La couche de normalisation par lots (BatchNormalization) est utilisée pour normaliser les activations de la couche précédente. Elle accélère l'entraînement

et améliore la stabilité du modèle en réduisant les variations internes covariantes. La normalisation par lots standardise les activations en fonction de la moyenne et de l'écart type du lot, ce qui permet de stabiliser et de régulariser l'entraînement du modèle.

La première couche LSTM comporte 64 unités et utilise une fonction d'activation ReLU. La régularisation L2 avec un coefficient de 0.0001 est appliquée pour éviter le surapprentissage. Cette couche est essentielle pour capturer les dépendances temporelles à long terme dans les séries temporelles.

La deuxième couche LSTM comporte également 64 unités et utilise la même régularisation L2 et la fonction d'activation ReLU que la première couche. Cette couche ne renvoie que la dernière sortie de la séquence, condensant ainsi les informations temporelles capturées par les couches précédentes en une seule sortie. Cette sortie sera utilisée pour les prédictions finales.

La couche Dropout est utilisée pour régulariser le modèle et prévenir le surapprentissage. Avec un taux de 0.1, cette couche ignore aléatoirement 10% des unités pendant l'entraînement. Cela empêche les neurones de devenir trop dépendants les uns des autres et améliore la généralisation du modèle.

La couche finale est une couche dense (fully connected) avec un nombre de neurones égal au nombre de pas de temps futurs que nous voulons prédire. Cette couche convertit les caractéristiques extraites par les couches précédentes en une prédiction de la charge de travail future.

Le modèle est compilé en utilisant l'erreur quadratique moyenne (mean squared error, MSE) comme fonction de perte. Le MSE mesure la moyenne des carrés des erreurs entre les valeurs prédites et les valeurs réelles, ce qui est approprié pour les problèmes de régression. L'optimiseur Adam est utilisé pour ajuster les poids du modèle. Adam est connu pour bien aux problèmes de séries temporelles.

Le nombre de paramètres (*voir tableau 4*) dans un modèle de réseau de neurones fait référence aux poids et biais qui sont ajustés pendant l'entraînement pour minimiser la fonction de perte. Le nombre total de paramètres dans un modèle inclut tous les poids et biais de chaque couche. C'est la somme de tous les paramètres nécessaires pour les calculs du réseau. Notre modèle a un total de 117 638 paramètres au total, ce qui correspond à environ 459.52 KB en termes de mémoire. Les paramètres entraînables sont ceux qui sont modifiés pendant le processus d'entraînement du modèle, il s'agit des poids et des biais des couches du réseau que l'algorithme d'optimisation ajuste pour minimiser la fonction de perte. Sur les 117 638 paramètres totaux, 117 126 paramètres sont ajustés pendant l'entraînement,

TABLE IV – *Détails techniques de l'architecture CNN-LSTM*

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 10, 256)	1024
batch_normalization (BatchNormalization)	(None, 10, 256)	1024
lstm (LSTM)	(None, 10, 64)	82176
lstm_1 (LSTM)	(None, 64)	33024
dropout (Dropout)	(None, 64)	0
dense (Dense)	(None, 6)	390
Total params : 117638 (459.52 KB)		
Trainable params : 117126 (457.52 KB)		
Non-trainable params : 512 (2.00 KB)		

en termes de mémoire, cela représente environ 457.52 KB. Les paramètres non-entraînés sont les paramètres qui ne sont pas modifiés pendant l'entraînement. Ces paramètres peuvent être liés à des couches spécifiques comme la Batch Normalization où certaines variables (comme la moyenne et la variance) sont calculées mais ne sont pas optimisées par rétropropagation.

Notre architecture offre une solution robuste et efficace pour la modélisation et la prédiction des charges de travail dans les environnements FaaS. En combinant les capacités d'extraction de caractéristiques des CNN et la puissance de modélisation temporelle des LSTM, ce modèle hybride permettra de capturer les variations locales et globales des séries temporelles univariées, fournissant ainsi des prédictions précises et fiables. Cette approche est particulièrement adaptée aux environnements des architectures serverless, où une gestion proactive des ressources est cruciale pour maintenir des performances optimales et une utilisation efficace des ressources.

III.4. Évaluation des performances du modèle

III.4.1. Données d'entraînement

Le dataset Azure Functions Trace 2019 fournit une vue détaillée de la charge de travail sur la plateforme Azure Functions [45]. Ce dataset mis à disposition [12], contient des informations sur les invocations de fonctions, les temps d'exécution et les allocations de mémoire des applications Azure Functions. Pour notre modèle, nous nous intéresserons à la distribution de l'allocation de mémoire des applications (statistiques sur la quantité de mémoire allouée à chaque application), notam-

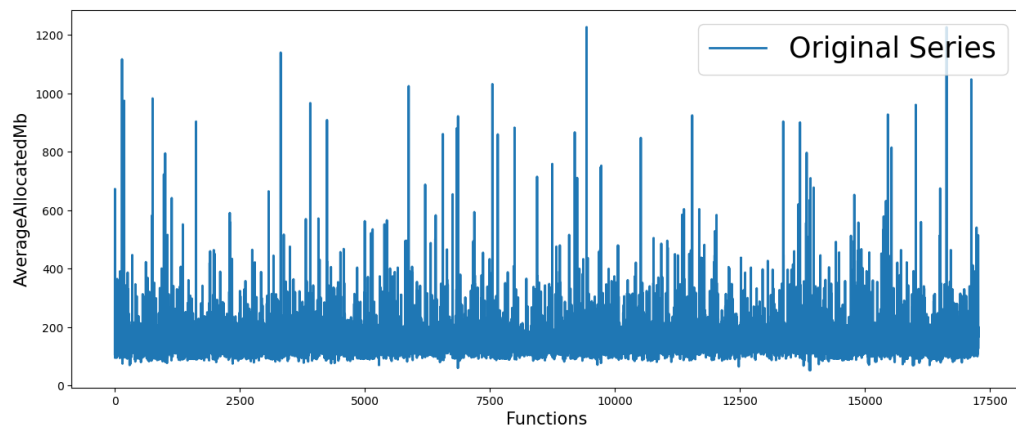


FIGURE 16 – Dataset d’entraînement pour le modèle CNN-LSTM

ment les percentiles d’allocation de mémoire moyenne pour chaque application sur une période de 24 heures (*Voir figure 16*). On peut y observer et comprendre les variations et les exigences de mémoire des différentes applications, ce qui est crucial pour la conception de modèles prédictifs efficaces.

Avant d’entraîner notre modèle, nous avons effectué un prétraitement des données. Le prétraitement est une étape cruciale pour garantir que le modèle pourra apprendre efficacement et faire des généralisations face à des nouvelles données. La suppression des valeurs aberrantes réduit le risque de biais dans l’entraînement du modèle, tandis que la normalisation permet d’éviter les problèmes de convergence lente ou d’explosion des gradients.

Enfin, les données ont été divisées en ensembles d’entraînement (80%) et de test (20%) pour évaluer la performance du modèle de manière rigoureuse et impartiale.

III.4.2. Cadre expérimental

Le cadre expérimental a été soigneusement conçu pour maximiser la performance et la validité des résultats de notre modèle hybride CNN-LSTM. Le modèle a été entraîné sur un ensemble de données représentant 80% du dataset total, en utilisant une approche itérative et optimisée. Nous avons utilisé l’Adam optimizer, connu pour ses capacités d’adaptation dynamique des taux d’apprentissage, pour accélérer la convergence du modèle. Les hyperparamètres, tels que la taille des lots, le taux d’apprentissage initial et le nombre d’époques, ont été ajustés par le biais de validation croisée pour trouver les paramètres optimaux.

Le modèle CNN-LSTM combine des réseaux neuronaux convolutifs (CNN) pour extraire les caractéristiques locales des données de charge de travail et des réseaux de mémoire à long court terme (LSTM) pour capturer les dépendances temporelles à long terme. Cette combinaison permet de tirer parti des points forts

de chaque architecture, offrant une capacité accrue à prédire les variations complexes des charges de travail.

III.4.3. Analyses des résultats

L'évaluation des performances du modèle a été réalisée en utilisant les 20% restants des données de test. La métrique principale utilisée pour évaluer le modèle est *le RMSE* qui mesure l'écart type des résidus (erreurs de prédiction).

Le RMSE est une mesure standard utilisée pour évaluer la précision des modèles de régression, car elle pénalise fortement les grandes erreurs, offrant ainsi une vue claire de la performance globale du modèle.

Les performances de notre modèle hybride CNN-LSTM ont été comparées avec celles du modèle LSTM développé par Nevlin et al. [2]. Les résultats obtenus sont observables sur le tableau 5.

Modèle	Métrique : RMSE
VAR	32494.19
ARIMA	104.69
LSTM	6,96
CNN-LSTM (<i>Notre modèle</i>)	4,10

TABLE V – Résultats des performances CNN-LSTM vs LSTM

III.4.4. Discussion

Les résultats montrent que notre modèle hybride CNN-LSTM surpasse significativement le modèle LSTM de Nevlin en termes de précision de prédiction. Cette amélioration peut être attribuée à la capacité de notre modèle à capturer à la fois les caractéristiques locales (grâce aux couches CNN) et les dépendances temporelles à long terme (grâce aux couches LSTM).

En comparaison avec le modèle LSTM de Nevlin, *notre modèle hybride CNN-LSTM a démontré une meilleure capacité à capturer les dynamiques complexes des données de charge de travail des applications Azure Functions*. L'utilisation de couches convolutionnelles permet d'extraire efficacement les caractéristiques locales, tandis que les couches LSTM capturent les dépendances temporelles à long terme. Les résultats obtenus montrent clairement que *notre approche hybride CNN-LSTM offre une précision supérieure par rapport aux approches précédentes*. Cette précision accrue se traduit par une meilleure capacité à prévoir les besoins en mémoire des applications Azure Functions, ce qui est essentiel pour optimiser la gestion des ressources dans un environnement cloud.

III.5. Conclusion

En conclusion, la méthodologie proposée pour modéliser la charge de travail dans les architectures FaaS en utilisant un modèle CNN-LSTM s'est avérée efficace. Cette approche hybride combine les forces des CNN pour l'extraction de caractéristiques locales et des LSTM pour la modélisation des dépendances temporelles, offrant ainsi une solution robuste et précise pour la prédiction de la charge de travail. L'étude a démontré que l'intégration de CNN et LSTM dans un modèle hybride améliore considérablement la précision de la prédiction de la charge de travail des fonctions serverless. Ce modèle non seulement surpasse les modèles précédents, mais offre également des insights précieux pour optimiser la gestion des ressources dans des environnements cloud tels que Azure Functions. Les résultats obtenus soulignent l'importance d'utiliser des architectures hybrides pour capturer les dynamiques complexes des séries de données dans des applications pratiques.

IV

CHAPITRE

CONTRIBUTION À L'ATTÉNUATION DU COLD START DANS LES ARCHITECTURES FAAS

CONTENU

IV.1 - Introduction	68
IV.2 - Méthodologie proposée : modèle basé sur les Transformers	69
IV.3 - Architecture du modèle d'atténuation	70
IV.4 - Évaluation des performances du modèle	72
IV.5 - Conclusion	86

IV.1. Introduction

Dans les architectures FaaS, le cold start représente un grand challenge, car il impacte de manière significative les performances et les expériences des utilisateurs. Lorsqu'une fonction n'a pas été invoquée pendant un certain temps, le système doit démarrer une nouvelle instance pour la traiter, c'est ce démarrage qui entraîne un délai supplémentaire connu sous le nom de cold start ou démarrage à froid. Ce temps de démarrage peut causer des dégradations dans les performances globales des applications serverless. Face à ces enjeux, il est devenu crucial de mettre en place des stratégies efficaces pour atténuer ces impacts et améliorer la réactivité des plateformes FaaS. Nous allons de ce fait présenter une solution visant à atténuer le phénomène de cold start. Plus spécifiquement, nous allons :

- Décrire la méthodologie et l'architecture de notre solution basée sur les Transformers adaptés aux séries temporelles.

- Détailler le processus général de notre solution (acquisition des données, traitements, étapes de clusterisation, etc.).
- Présenter et analyser les résultats obtenus sur la réduction du délai et de la fréquence de cold start.
- Comparer les performances de notre modèle avec celles d'un modèle LSTM sur des métriques clés telles que sMAPE, RMSE, Normalized RMSE, R2 SCORE, Spearman Correlation, et Explained Variance.
- Discuter les avantages de notre approche par rapport aux travaux antérieurs et démontrer sa pertinence et son efficacité en production.

En suivant cette structure, nous avons pour but de démontrer la pertinence et l'efficacité de notre approche pour atténuer le cold start, en fournissant une solution robuste et applicable à des scénarios réels.

IV.2. Méthodologie proposée : modèle basé sur les Transformers

Notre approche est basée sur l'utilisation des Transformers adaptés aux séries temporelles, ce qui constitue une avancée par rapport aux techniques traditionnelles telles que les RNN et les LSTM. Les Transformers ont révolutionné le domaine du NLP grâce à leur capacité à détecter les dépendances à long terme dans les données. Cette même capacité a ainsi pu être exploitée pour les séries temporelles, où les relations temporelles à long terme jouent un rôle clé. Adapter les Transformers pour les séries temporelles, permet de bénéficier de leur architecture unique basée sur le mécanisme d'attention, qui permettra de traiter efficacement les dépendances temporelles sans les limitations des LSTM telles que la difficulté à gérer les longues séquences et le problème de gradient évanescent. La solution proposée pour la prévision des séries temporelles est basée sur un modèle de Transformers vanilla adapté aux séries temporelles, par Kashif [61].

Aussi, les approches traditionnelles ne permettent pas efficacement modéliser des séries temporelles multiples et simultanées, limitant ainsi leur capacité à apprendre des représentations généralisées. En revanche, les Transformers pour les séries temporelles est un modèle global, capable d'apprendre des représentations latentes à partir de nombreuses séries temporelles, et de fournir des prévisions probabilistes plutôt que des prévisions ponctuelles. Cela signifie que le modèle apprend une distribution à partir de laquelle on peut échantillonner des valeurs futures, fournissant ainsi des informations sur l'incertitude des prévisions [62].

Notre solution se distingue ainsi par les éléments suivants :

- Sa capacité à capter les relations à long terme, contrairement aux LSTM qui ont une capacité limitée pour retenir les informations sur de très longues séquences.
- Sa simplicité et son efficacité : un modèle unique utilisé pour résoudre à la fois le problème du délai et de la fréquence de cold start, contrairement aux approches existantes qui nécessitent plusieurs modèles.
- Sa robustesse en production : notre modèle a été testé sur des schémas d'invocations réels et variés, prouvant son efficacité et sa fiabilité en environnement de production.

IV.3. Architecture du modèle d'atténuation

Les Transformers adaptés aux séries temporelles sont basés sur l'architecture des Transformers vanilla, avec des modifications pour traiter les données temporelles. Ils sont composés de deux blocs :

- **Un encodeur** : qui prend en entrée une longueur de contexte de valeurs de la série temporelle (appelées `pastValues`).
- **Un décodeur** : qui prédit une longueur de prédiction de valeurs futures de la série temporelle (appelées `futureValues`).

Pendant l'entraînement, on doit fournir au modèle des paires de (`pastValues` et `futureValues`).

Caractéristiques supplémentaires

- **`pastTimeFeatures`** : des caractéristiques temporelles ajoutées aux `pastValues` servant d'encodages positionnels pour l'encodeur. On peut avoir par exemple : "jour du mois", "mois de l'année", etc. Si une valeur de la série temporelle a été obtenue le 11 août, le vecteur de caractéristiques temporelles pourrait être [11, 8].
- **`futureTimeFeatures`** : des caractéristiques temporelles ajoutées aux `futureValues` servant d'encodages positionnels pour le décodeur.
- **`staticCategoricalFeatures`** : des caractéristiques catégorielles statiques dans le temps (ayant la même valeur pour toutes les `pastValues` et `futureValues`), comme l'identifiant d'une fonction ou d'une application. Ajoutent des informations contextuelles statiques qui sont constantes pour toutes les valeurs passées et futures d'une série temporelle.

Flux de données (Voir figure 17)

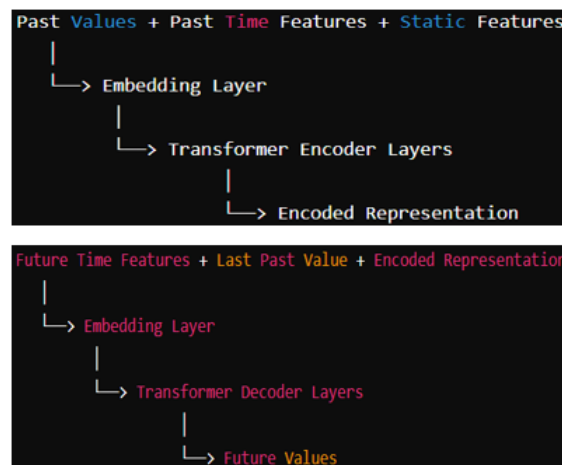


FIGURE 17 – Représentation des entrées de l'encodeur et du décodeur

- Les pastValues et les pastTimeFeatures sont combinés et passés à l'encodeur.
- Les staticCategoricalFeatures et staticRealFeatures sont ajoutées à l'entrée de l'encodeur.
- L'encodeur produit une représentation encodée qui résume les informations des pastValues.
- Les futureTimeFeatures et la dernière valeur des pastValues sont combinées et passées au décodeur.
- Le décodeur génère les futureValues en utilisant les représentations encodées et les futureTimeFeatures.

Le modèle Transformers adapté aux séries temporelles est une architecture sophistiquée qui tire parti des Transformers pour effectuer des prévisions probabilistes des séries temporelles, en intégrant des informations temporelles et contextuelles pour améliorer la précision des prédictions.

Pour mettre en perspective les avantages de notre modèle Transformers, nous l'avons comparé avec un LSTM dont l'architecture est la suivante : *Le réseau LSTM a cinq couches cachées, chacune avec 32 neurones et une fonction d'activation ReLU. De plus, pour prévenir le surapprentissage, une couche Dropout avec une valeur de 0,5 est utilisée. Le réseau a également une seule sortie et une fonction d'activation linéaire. L'erreur quadratique moyenne (MSE) est également utilisée comme fonction de perte.*

1. Gestion des dépendances à long terme

- a) **Transformers** : utilisent le mécanisme d'attention pour mieux capturer des dépendances à longue portée, de manière plus efficace et parallèle,

ce qui permet un entraînement plus rapide et des prédictions plus précises sur de grandes séquences de données.

- b) **LSTM** : utilise des cellules de mémoire pour conserver l'état à long terme, ce qui peut être limité par des problèmes de gradient dégradé ou explosif. L'architecture est séquentielle, ce qui rend l'entraînement et l'inférence plus lents.

2. Efficacité de calcul

- a) **Transformers** : ici, les opérations d'attentions peuvent être parallélisées, ce qui accélère l'entraînement et l'inférence.
- b) **LSTM** : ici, les calculs doivent être effectués de manière séquentielle en raison de la nature récurrente, ce qui ralentit le processus.

3. Complexité de l'architecture

- a) **Transformers** : requièrent un encodage positionnel pour traiter les séquences temporelles et peut être plus complexe à implémenter.
- b) **LSTM** : plus simples à implémenter et comprennent une architecture bien établie pour les séries temporelles.

4. Performance sur les tâches

- a) **Transformers** : s'avèrent excellents pour capturer les relations globales et les dépendances complexes, prouvant leur efficacité dans notre contexte FaaS.
- b) **LSTM** : bon pour les séquences plus courtes et les dépendances locales, mais peut être moins performant sur des séquences très longues et complexes.

Par ces observations, on peut déjà déduire que notre solution basée sur les Transformers propose une approche efficace pour atténuer les problèmes de cold start, pouvant surpasser les modèles traditionnels comme les LSTM sur plusieurs aspects critiques.

IV.4. Évaluation des performances du modèle

IV.4.1. Données d'entraînement

Le dataset initial utilisé dans cette étude provient de l'offre Azure Functions de Microsoft, collectée en juillet 2019 [45, 12].

Ce grand dataset met à disposition plusieurs sous ensemble de données parmi lesquels :

- **Comptes et déclencheurs des invocations de fonctions** : données sur le nombre d'invocations de chaque fonction par minute sur une période de 24 heures.
- **Distributions du temps d'exécution des fonctions** : informations sur les temps d'exécution des fonctions.
- **Distributions de l'allocation de mémoire des applications** : données sur la quantité de mémoire allouée pour chaque application.

Pour notre étude, nous avons particulièrement utilisé les fichiers relatifs aux comptes des invocations de fonctions, structurés de la manière suivantes :

- **14 fichiers** : un fichier par période de 24 heures (*figure 18-(1)*).
- **1440 champs par fichier** : chaque champ représente le nombre d'invocations de la fonction pour chaque minute de la période de 24 heures.

Les champs inclus dans chaque fichier sont :

- **HashOwner** : ID unique du propriétaire de l'application, anonymisé.
- **HashApp** : ID unique de l'application, anonymisé.
- **HashFunction** : ID unique de la fonction au sein de l'application, anonymisé.
- **Trigger** : Type de déclencheur de la fonction (http, timer, event, queue, storage, orchestration, others).

Lors du traitement de ces données, seules les instances de fonctions déclenchées par HTTP ont été sélectionnées. Cette décision a été prise car les invocations HTTP sont les plus imprévisibles et les plus populaires dans les architectures FaaS. Le processus global de traitement des données est illustré à la figure 18.

Ainsi,

- Tout d'abord, un filtrage a été appliqué sur ce grand ensemble de données, afin de sélectionner uniquement des fonctions déclenchées par HTTP, identifiées via le champ "Trigger" (*figure 18-(2 3)*).
- Ensuite, les 14 fichiers contenant les invocations de chaque journée, ont été fusionnés en un seul et même fichier (*figure 18-(4 5)*).
- Pour regrouper les fonctions par similarité, nous avons utilisé l'algorithme DBSCAN (*figure 18-(6)*).

DBSCAN

Il s'agit d'un algorithme de clustering utilisé dans le domaine de l'apprentissage non supervisé. Contrairement à d'autres méthodes de clustering comme K-means qui exigent de connaître à l'avance le nombre de clusters, DBSCAN est capable de détecter les clusters de forme arbitraire dans un ensemble de données sans avoir besoin de cette information préalable.

Principe de fonctionnement

- **Détection des points centraux (core points) :** DBSCAN commence par identifier les points dans l'espace des données qui ont un nombre minimum de points voisins à une certaine distance fixée. Ces points sont appelés points centraux.
- **Extension des clusters :** à partir de ces points centraux, DBSCAN explore les clusters en reliant les points à proximité. Il continue à étendre chaque cluster tant qu'il y a des points voisins qui répondent aux critères de densité.
- **Marquage des points de bordure et des points de bruit :** les points qui ne sont pas assez proches d'un point central mais qui sont proches d'un point de ce type sont marqués comme des points de bordure, et les points qui ne sont pas assez proches de tout point central sont marqués comme des points de bruit, c'est-à-dire qu'ils ne sont pas affectés à un cluster.

Avantages

- **Capacité à détecter des clusters de forme arbitraire :** contrairement à K-means, qui suppose que les clusters sont globulaires, DBSCAN peut identifier des clusters de formes complexes.
- **Robuste face au bruit :** DBSCAN est capable de gérer efficacement les données bruitées, car il classe les points isolés comme du bruit plutôt que de les affecter à des clusters.
- **Pas besoin de spécifier le nombre de clusters à l'avance :** DBSCAN détermine automatiquement le nombre de clusters dans les données, ce qui le rend très pratique dans de nombreuses situations réelles où le nombre de clusters n'est pas connu à l'avance.
- **Efficace pour les grandes bases de données :** DBSCAN peut être très efficace pour les ensembles de données de grande taille car il utilise un algorithme basé sur la recherche de voisins proches.

Les résultats de la clusterisation ont montré que les fonctions HTTP déclenchées pouvaient être regroupées en 33 clusters distincts. Chaque cluster représente un ensemble de fonctions présentant des comportements d'invocation similaires (*figure 18-(7)*).

Pour chaque cluster, des fonctions représentatives ont été sélectionnées de façon aléatoire dans chacun d'eux (*figure 18-(8)*).

Pour le **problème de délai de cold start**, les fonctions populaires ou fréquemment utilisées ont été prises en considération. Ces fonctions présentent des motifs

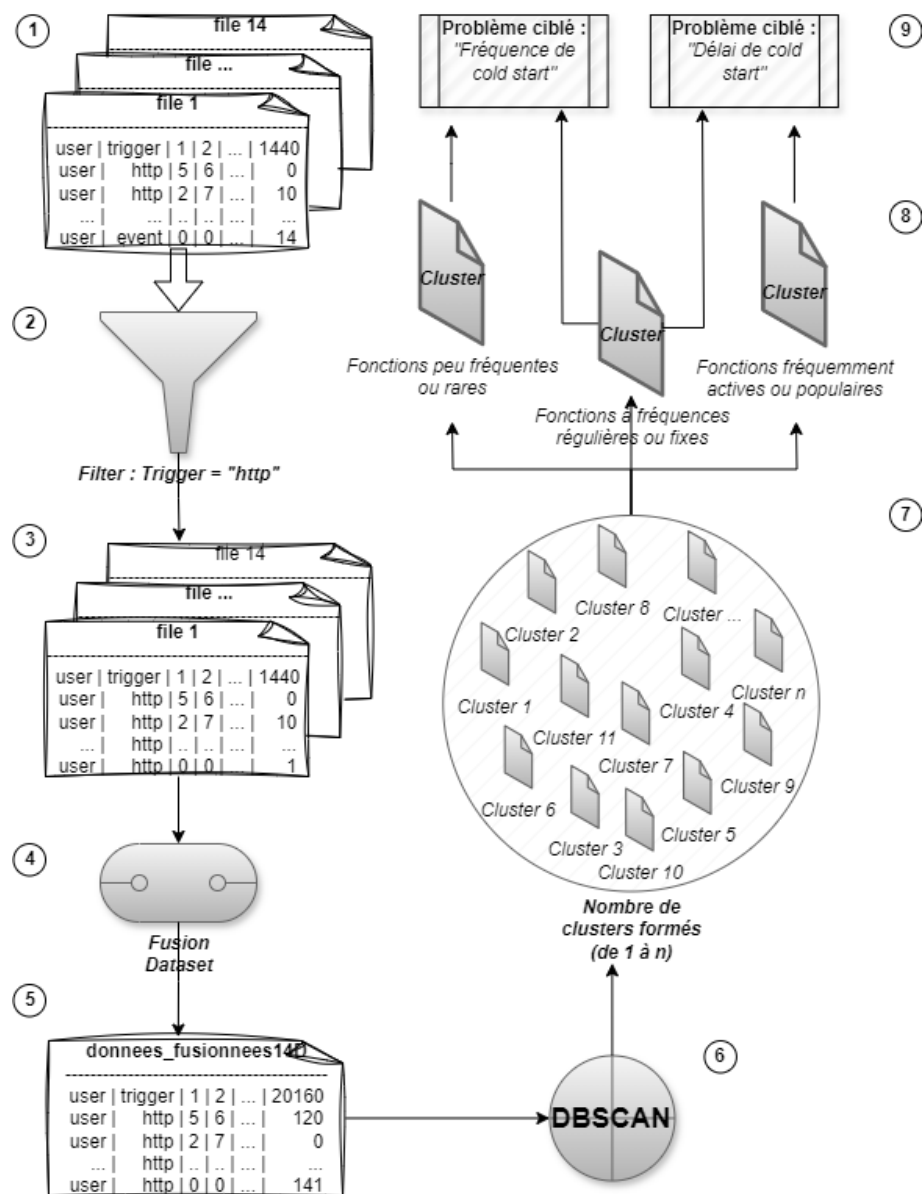


FIGURE 18 – Processus global des traitements des datasets à passer en entraînement au modèle Transformers

d'invocation qui permettent de prédire les pics de demandes, facilitant ainsi la gestion des délais de cold start (*figure 18-(9)*).

Pour **le problème de fréquence de cold start**, les fonctions utilisées à des fréquences irrégulières et à faible demande ont été identifiées. L'analyse des intervalles d'invocation entre les appels de fonctions a permis d'adapter les sous-datasets en conséquence (*figure 18-(9)*).

Pour **les fonctions à fréquences régulières ou fixes**, on peut y appliquer **une solution de délai de cold start** ou **une solution de fréquence de cold start**. Ces fonctions présentent des motifs fixes au fil du temps, qui ne changent pas. Par conséquent, le modèle appliqué sur ce type de données, peu importe le problème traité retournera une précision de 100% (*figure 18-(9)*).

IV.4.2. Cadre expérimental

Les hyperparamètres définissent les configurations internes du modèle TimeSeriesTransformers utilisé pour la prédiction des séries temporelles. Les principaux hyperparamètres incluent :

- **predictionLength** : longueur de la séquence de prédiction.
- **contextLength** : longueur du contexte utilisé pour la prédiction
- **lagsSequence** : séquence des lags (périodes d'observations passées dans le passé).
- **cardinality** : cardinalité des séries temporelles (nombre de séries distinctes).
- **embeddingDimension** : dimension des embeddings.
- **encoderLayers** : nombre de couches dans l'encodeur (4).
- **decoderLayers** : nombre de couches dans le décodeur (4).
- **dModel** : dimension du modèle (32).

L'entraînement du modèle a été réalisé sur Google Colab en utilisant le backend Google Compute Engine. Les spécifications de l'environnement incluent :

- **Python 3 (GPU)** : utilisation de l'accélération matérielle pour les calculs intensifs.
- **RAM du système** : 12.7 GB, permettant de charger et traiter de grandes quantités de données.
- **RAM du GPU** : 15.0 GB, permettant d'entraîner des modèles de deep learning complexes avec des réseaux de neurones profonds.

Le processus global de notre solution, de l'acquisition et la préparation des datasets jusqu'à l'entraînement de notre modèle, permet ainsi de développer une approche efficace pour **la prédiction des invocations de fonctions à une période**

donnée (pour le délai de cold start) et la détermination des intervalles de temps d'arrivée entre ces dernières (pour la fréquence de cold start).

La méthode DBSCAN a facilité la segmentation des données en clusters pertinents, tandis que les adaptations des sous-datasets ont adressé efficacement les problèmes de délai et de fréquence de cold start.

Pour évaluer et déployer notre modèle Transformers, nous avons utilisé **la plateforme serverless OpenWhisk**. OpenWhisk offre une flexibilité et une scalabilité accrues grâce à ses divers composants et configurations, ce qui en fait un choix idéal pour notre projet.

Pour optimiser les performances et réduire les délais de cold starts, OpenWhisk utilise des conteneurs préchauffés. Ces conteneurs sont préalablement créés et prêts à être utilisés dès qu'une action est déclenchée. La gestion des conteneurs préchauffés peut être configurée de manière statique, réactive ou non, permettant une scalabilité dynamique en fonction de la demande, mais ces paramètres par défaut ne permettent pas totalement de répondre de façon automatique et satisfaisante aux demandes variés et soudaines des clients.

Notre modèle Transformers a été intégré dans cet environnement pour améliorer les performances et l'adaptabilité de la plateforme, **en configurant les paramètres d'auto-scaling et de la fenêtre de conteneur inactif (durée de vie d'un conteneur, pour atténuer la fréquence de cold start).**

L'utilisation d'OpenWhisk combinée à notre modèle Transformers permet de créer une plateforme adaptative et performante, capable de répondre efficacement à la demande variable des utilisateurs et d'améliorer les performances globales. Cette configuration optimise non seulement les ressources mais assure également une réactivité accrue, essentielle pour les applications modernes. Le processus global de notre approche proposée pour rendre la plateforme OpenWhisk adaptative est illustré à la figure 19.

IV.4.3. Analyses des résultats

IV.4.3.1. Résultats pour le délai de cold start

Le délai de cold start représente le temps nécessaire pour démarrer un conteneur après une période d'inactivité. Ce délai impacte directement la latence des applications, ce qui peut affecter négativement l'expérience utilisateur. De ce fait, on veut par cette phase, préchauffer des conteneurs juste avant qu'ils ne soient demandés, ainsi, ce temps de préparation ne sera plus remarquable par les utilisateurs.

Pour ce faire nous avons besoin des données sur le nombre d'invocations faites dans le passé, pour prédire la demande future.

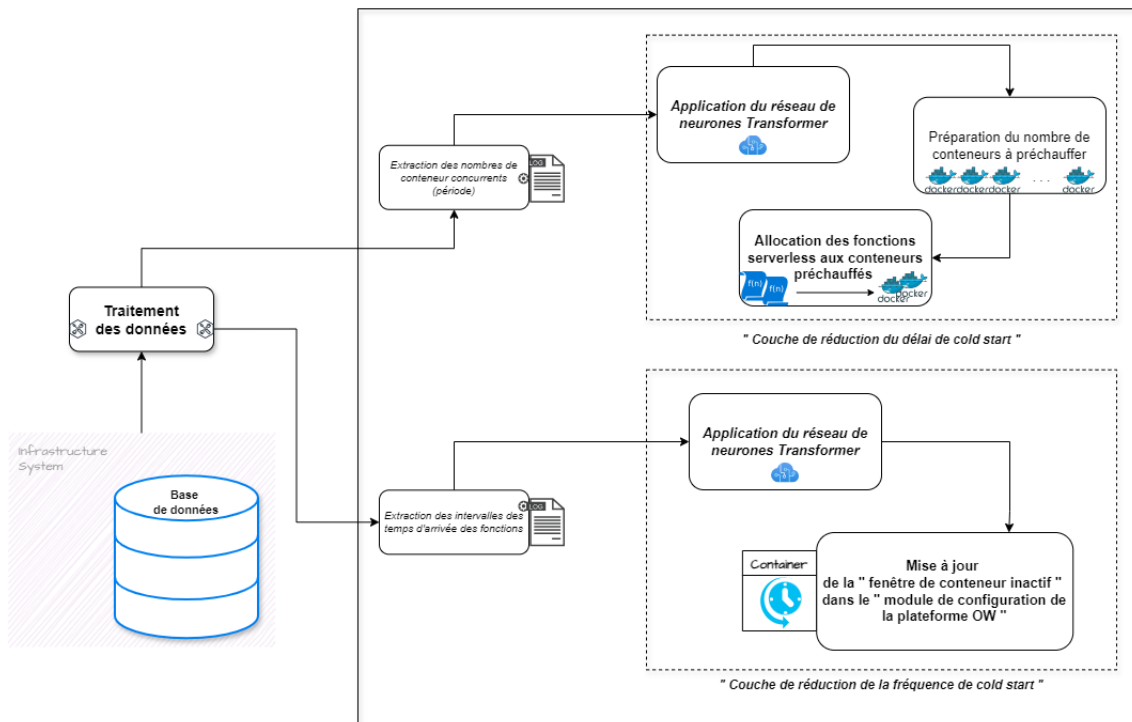


FIGURE 19 – Approche proposée : modèle basé sur les Transformers pour atténuer le cold start

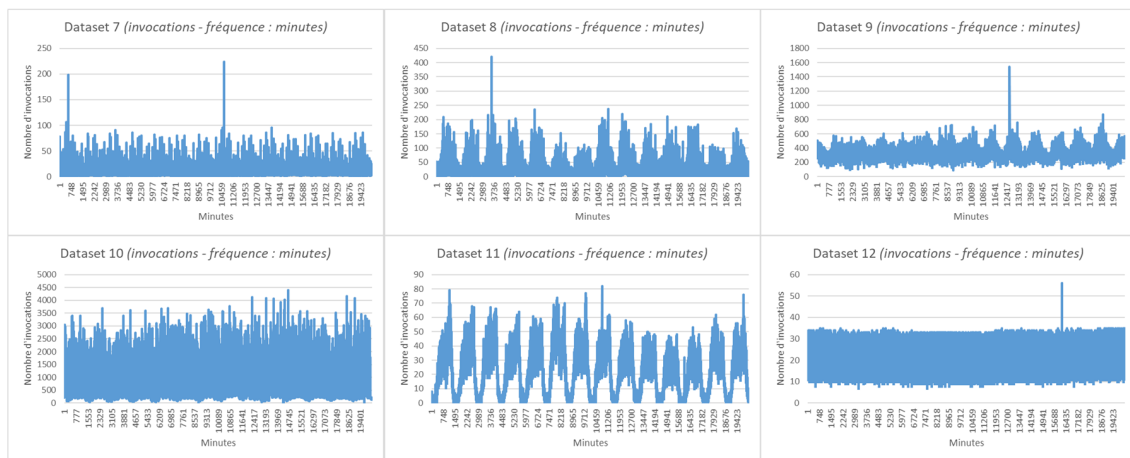


FIGURE 20 – Six schémas d'invocations (fonctions) - fréquence : minutes

Dans notre vaste ensemble de schémas d'invocations (fonctions), nous en avons sélectionnés aléatoirement 12 différents comme datasets, pour évaluer notre modèle. D'abord :

- En considérant le dataset original, sans l'altération. C'est à dire, des invocations qui arrivent de façon continue, minute après minute (voir figure 20).
- Puis, en ne considérant que les invocations simultanées chaque heure (datasets altérés, pour simuler une ressemblance aux données employées dans la revue de littérature) (voir figure 21).

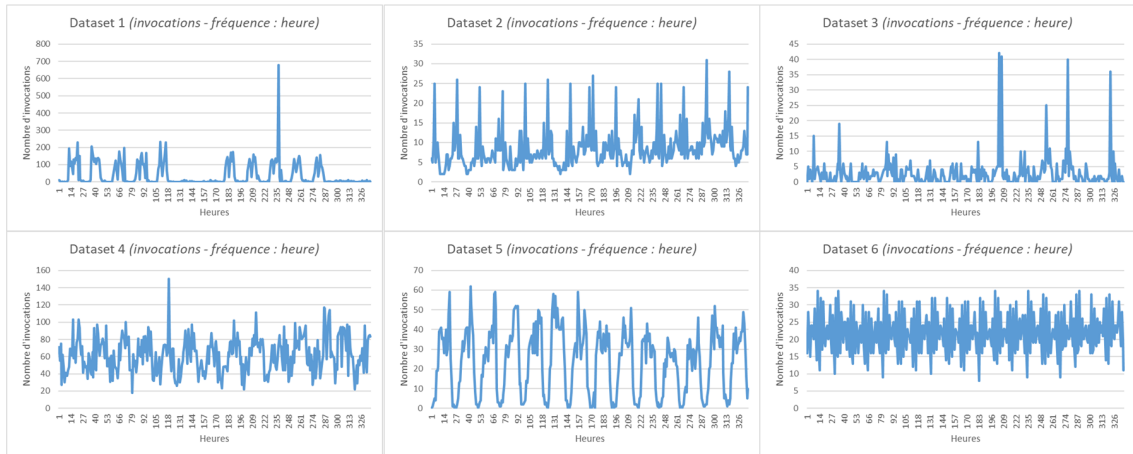


FIGURE 21 – Six schémas d'invocations (fonctions) - fréquence : heure

Les performances de notre modèle de Transformers pour cette phase de délai ont été comparées par rapport au modèle LSTM défini plus haut, sur différentes métriques standard. Ces métriques offrent une vue d'ensemble détaillée des performances prédictives, en prenant en compte différents aspects des erreurs et des variances des prédictions :

- **sMAPE** [63] : mesure l'erreur de prédiction en pourcentage, prenant en compte à la fois les sur-prédictions et les sous-prédictions. Une sMAPE plus basse indique une meilleure précision des prédictions. Elle est symétrique car elle traite les erreurs de sur-prédiction et de sous-prédiction de manière égale.

$$\text{sMAPE} = \frac{1}{n} \sum_{i=1}^n \frac{|F_i - A_i|}{(|A_i| + |F_i|)/2} \times 100$$

où F_i est la valeur prédite et A_i est la valeur réelle.

- **Explained Variance** [64] : quantifie la proportion de la variance totale des données expliquée par le modèle. Une valeur proche de 1 indique que le modèle explique bien la variance des données, tandis qu'une valeur proche de 0 signifie que le modèle n'explique pas bien la variance.

$$\text{Explained Variance} = 1 - \frac{\text{Var}(y - \hat{y})}{\text{Var}(y)}$$

où y est la valeur réelle et $\hat{y}$ est la valeur prédite.

- **RMSE** [63] : mesure l'erreur quadratique moyenne des prédictions, mettant plus de poids aux grandes erreurs. Le RMSE est une mesure absolue de l'erreur. Plus le RMSE est bas, meilleure est la précision des prédictions. Il

est sensible aux grandes erreurs, ce qui signifie qu'il pénalise sévèrement les grandes erreurs de prédiction.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (F_i - A_i)^2}$$

- **Normalized RMSE** [65] : version du RMSE qui permet de comparer les performances entre différents jeux de données en les normalisant. En normalisant le RMSE, on obtient une mesure relative de l'erreur qui peut être comparée à travers différents contextes ou jeux de données.

$$\text{Normalized RMSE} = \frac{\text{RMSE}}{\text{Range of Values}}$$

- **R² Score** [63] : indique la proportion de la variance des données qui est expliquée par le modèle. Un R² proche de 1 signifie que le modèle explique bien la variance des données, tandis qu'un R² proche de 0 signifie qu'il n'explique pas bien la variance.

$$R^2 = 1 - \frac{\sum_{i=1}^n (A_i - F_i)^2}{\sum_{i=1}^n (A_i - \bar{A})^2}$$

où $\bar{A}$ est la moyenne des valeurs réelles.

- **Spearman Correlation** [66] : mesure la corrélation entre les rangs des valeurs prévues et observées, capturant la tendance monotone entre elles. Une corrélation de Spearman proche de 1 indique une forte tendance monotone positive entre les valeurs prévues et observées. Cette métrique est utile pour évaluer la capacité du modèle à préserver l'ordre des données.

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

où d_i est la différence entre les rangs des valeurs prédites et observées.

L'utilisation de ces différentes métriques permet d'obtenir une évaluation complète et nuancée des performances des modèles prédictifs. Elles permettent de capturer différents aspects de l'erreur de prédiction, de la variance expliquée et des corrélations entre les données prévues et observées, offrant ainsi une vue d'ensemble détaillée de l'efficacité des modèles Transformers et LSTM.

Les résultats en considérant comme fréquence des invocations chaque heure (voir tableau 6) ou chaque minute (voir tableau 7), montrent que **le modèle Transformers dépasse systématiquement le modèle LSTM sur presque toutes les métriques et pour tous les datasets testés**. Le Transformers montre une plus

TABLE VI – Résultats des performances - délai de cold start : dataset (fréquence : heure)

Fonctions	Modèle	sMAPE	E.V. ^a	RMSE	N. RMSE ^b	R2 Score	S. C. ^c
Dataset 1	LSTM	1,204	-3,195	8,468	0,705	-5,607	-0,034
	Transformers	1,611	-0,781	3,659	0,365	-0,876	0,065
Dataset 2	LSTM	0,321	0,202	4,641	0,193	0,196	0,499
	Transformers	0,358	0,710	3,874	0,161	0,537	0,498
Dataset 3	LSTM	1,256	-0,546	8,982	0,249	-0,546	-0,045
	Transformers	1,446	-0,059	7,493	0,208	-0,069	0,320
Dataset 4	LSTM	0,274	0,053	18,988	0,256	0,040	0,321
	Transformers	0,229	0,250	16,931	0,228	0,247	0,602
Dataset 5	LSTM	0,368	0,797	6,767	0,140	0,796	0,762
	Transformers	0,444	0,837	6,412	0,133	0,814	0,797
Dataset 6	LSTM	0,107	0,788	2,957	0,147	0,686	0,865
	Transformers	0,103	0,893	2,352	0,106	0,835	0,957

^a E.V. : Explained Variance. ^b N. RMSE : Normalized RMSE. ^c S. C. : Spearman Correlation.

grande capacité à capter les relations à long terme et à fournir des prédictions plus précises et robustes, ce qui est particulièrement visible dans des scénarios alternatifs et des datasets variés et **permet de prédire plus précisément les périodes de forte demande et de démarrer les conteneurs de manière proactive**. En réduisant les délais de démarrage, les Transformers améliorent la réactivité des applications et minimisent les interruptions de service.

IV.4.3.2. Résultats pour la fréquence de cold start

La fréquence de cold start, est un autre indicateur clé de performance. Ici, il est question de déterminer la période optimale pendant laquelle on devrait laisser un conteneur actif après qu'il a terminé son exécution (fenêtre de conteneur inactif), afin de servir une future demande, sans pour autant gaspiller des ressources. Une fréquence élevée de cold start peut indiquer une inefficacité dans la gestion des ressources et une mauvaise anticipation des besoins futurs. Dans notre étude, nous avons observé une réduction significative de la fréquence des cold starts grâce à l'utilisation de modèles Transformers.

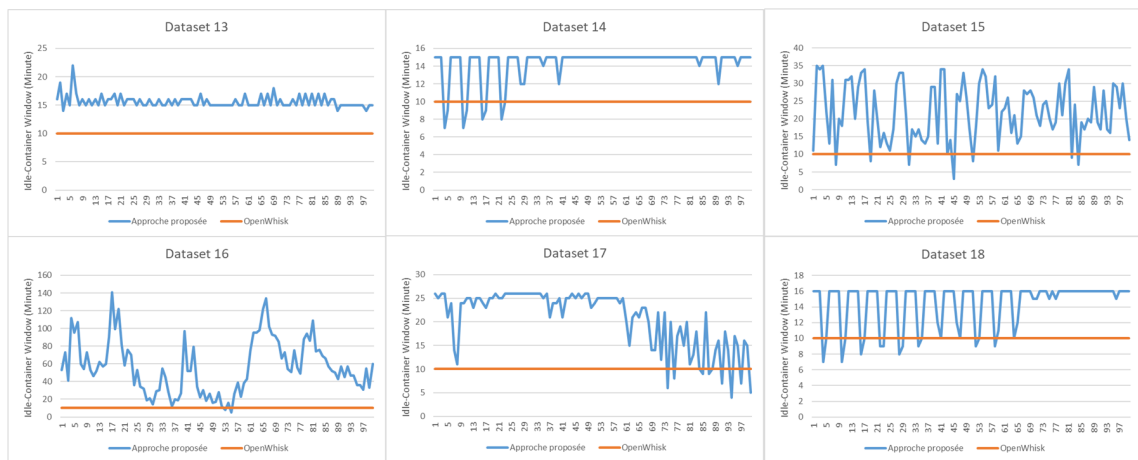
Les métriques observés ici sont :

- **La variation de la fenêtre du conteneur inactif ;**
- **Le nombre de cold start ;**

TABLE VII – Résultats des performances - délai de cold start : dataset (fréquence : minute)

Fonctions	Modèle	sMAPE	E.V. ^a	RMSE	N. RMSE ^b	R2 Score	S. C. ^c
Dataset 7	LSTM	0,333	0	4,163	0,143	-0,009	0,001
	Transformers	0,201	0,717	2,368	0,081	0,700	0,659
Dataset 8	LSTM	1,187	-0,225	15,601	0,288	-0,330	-0,062
	Transformers	1,371	0,274	12,360	0,228	0,141	0,441
Dataset 9	LSTM	0,096	-0,183	53,663	0,177	-0,277	0,189
	Transformers	0,096	0,193	51,213	0,169	-0,164	0,461
Dataset 10	LSTM	0,166	0,747	196,030	0,119	0,738	0,753
	Transformers	0,128	0,805	167,645	0,101	0,802	0,777
Dataset 11	LSTM	0,425	0,458	3,518	0,152	0,450	0,704
	Transformers	0,412	0,522	3,236	0,140	0,522	0,731
Dataset 12	LSTM	0,108	0,813	3,060	0,122	0,711	0,898
	Transformers	0,043	0,958	1,179	0,047	0,958	0,974

^a E.V. : Explained Variance. ^b N. RMSE : Normalized RMSE. ^c S. C. : Spearman Correlation.

**FIGURE 22 – Variations de la fenêtre de conteneur inactif**

Les résultats du tableau 8 présentent une analyse comparative des fréquence des cold starts pour différentes configurations d'OpenWhisk à savoir : la configuration par défaut d'OpenWhisk et la configuration adaptative utilisant le modèle Transformers pour ajuster dynamiquement la fenêtre de conteneur inactif. Pour ce faire, nous avons encore sélectionné 6 schémas d'invocations différents correspondant au problème de cette phase. Voici les détails des résultats et leurs interprétations :

Variations de la fenêtre de conteneur inactif (figure 22)

TABLE VIII – *Comparaison des variations de la fenêtre de conteneur inactif et du nombre de cold starts sur OpenWhisk avec et sans modèle de Transformers associé*

Fonction	Plateforme	Variation de la FCI ^a	Nombre de CS ^b /100
Dataset 13	OpenWhisk OW^c + Transf^d	fixe = 10 min <i>Entre 14 min et 22 min</i>	100 45
Dataset 14	OpenWhisk OW^c + Transf^d	fixe = 10 min <i>Entre 7 min et 15 min</i>	66 14
Dataset 15	OpenWhisk OW^c + Transf^d	fixe = 10 min <i>Entre 3 min et 35 min</i>	72 45
Dataset 16	OpenWhisk OW^c + Transf^d	fixe = 10 min <i>Entre 5 min et 141 min</i>	71 25
Dataset 17	OpenWhisk OW^c + Transf^d	fixe = 10 min <i>Entre 4 min et 26 min</i>	62 38
Dataset 18	OpenWhisk OW^c + Transf^d	fixe = 10 min <i>Entre 7 min et 16 min</i>	66 17

^a FCI : Fenêtre de conteneur inactif.^b CS : Cold start.^c OW : OpenWhisk.^d Transf : Transformers.

Dans la configuration par défaut d'OpenWhisk, la fenêtre de conteneur inactif est statique et fixée à 10 minutes pour toutes les fonctions. Cette approche statique ne prend pas en compte les variations de la charge de travail, ce qui conduit à une inefficacité dans la gestion des conteneurs. Une fenêtre fixe ne peut pas s'ajuster aux périodes de forte ou de faible activité, ce qui peut entraîner soit un sur-provisionnement (et donc des coûts supplémentaires qui pouvaient être évités), soit un sous-provisionnement (entraînant des cold starts fréquents).

Avec le modèle Transformers, la fenêtre de conteneur inactif est ajustée dynamiquement en fonction des besoins spécifiques de chaque fonction. Cette approche permet une flexibilité et une adaptation en temps réel aux variations de la charge de travail. Pour le dataset 13, la fenêtre varie entre 14 et 22 minutes, ce qui suggère une charge de travail relativement stable avec des périodes prévisibles d'activité et d'inactivité. Pour le dataset 14, la fenêtre est plus étroite (7 à 15 minutes), indiquant des variations plus fréquentes de la charge de travail. Le dataset 15 présente une fenêtre plus large (3 à 35 minutes), ce qui pourrait indiquer des périodes d'activité très irrégulières, nécessitant des ajustements fréquents pour optimiser la gestion des conteneurs. Enfin, le dataset 16 montre la plus grande variation (5 à 141 minutes), sug-

gérant des schémas d'invocation extrêmement hétérogènes, où une flexibilité maximale est essentielle pour éviter les cold starts tout en minimisant les coûts.

Nombre de Cold Starts

La configuration par défaut avec une fenêtre fixe de 10 minutes entraîne un nombre élevé de cold starts dans toutes les fonctions. On a le dataset 13 qui enregistre 100 cold starts pour 100 invocations faites, ce qui signifie que chaque invocation est susceptible de déclencher un cold start. Le dataset 14 et le dataset 15 montrent des résultats légèrement meilleurs, avec respectivement 66 et 72 cold starts, mais ces chiffres restent élevés. Le dataset 16 présente également un nombre élevé de cold starts (71). Ces résultats indiquent que la fenêtre fixe de 10 minutes est inefficace pour gérer les différentes charges de travail, entraînant des redémarrages fréquents des conteneurs et donc une dégradation des performances.

L'utilisation du modèle Transformers pour ajuster dynamiquement la fenêtre de conteneur inactif réduit considérablement le nombre de cold starts dans tous les datasets. Pour le dataset 13, le nombre de cold starts est réduit de 100 à 45, soit une réduction de 55%. Pour le dataset 14, la réduction est encore plus spectaculaire, passant de 66 à 14, soit une réduction de 79%. Le dataset 15 montre une réduction similaire à celle de du dataset 13, avec 45 cold starts. Et enfin, le dataset 16 présente une réduction de 71 à 25 cold starts, soit une réduction de 65%. Ces résultats montrent que l'ajustement dynamique de la fenêtre de conteneur inactif permet de réduire significativement le nombre de cold starts, améliorant ainsi la performance et la réactivité de la plate-forme.

La comparaison entre la configuration par défaut d'OpenWhisk avec une fenêtre de conteneur inactif fixe et la configuration adaptative utilisant le modèle Transformers démontre clairement l'avantage de l'approche adaptative.

- Pour la gestion de la fenêtre de conteneur inactif : la fenêtre adaptative permet d'ajuster en temps réel la durée pendant laquelle un conteneur reste inactif avant d'être supprimé, en fonction des schémas d'invocation réels. Cela permet une utilisation plus efficace des ressources et réduit les coûts associés à la gestion des conteneurs.
- Pour la réduction des fréquences de cold starts : l'ajustement dynamique de la fenêtre de conteneur inactif réduit significativement le nombre de cold starts, améliorant la performance et la réactivité des fonctions. Une réduction allant jusqu'à 79% des cold starts a été observée pour certaines fonctions, ce qui démontre l'efficacité du modèle Transformers.

Ces résultats mettent en évidence l'importance d'une gestion adaptative et dynamique des ressources dans les environnements FaaS, où les schémas d'invocation peuvent être très variés et imprévisibles. L'utilisation du modèle Transformers pour ajuster la fenêtre de conteneur inactif en fonction des besoins spécifiques de chaque dataset offre une solution robuste et efficace pour améliorer les performances globales de la plate-forme.

IV.4.4. Discussions

Capacité à capter les relations à long terme : l'un des principaux avantages des modèles Transformers est leur capacité à capturer les relations à long terme dans les données séquentielles. Contrairement aux modèles LSTM utilisé dans la revue comme celui de Vahidinia et al. [4], qui utilisent des mécanismes de mémoire pour conserver les informations pertinentes au fil du temps. Dans le contexte des architectures FaaS, cette capacité est cruciale pour prédire les demandes futures et gérer les ressources de manière proactive, afin de mieux réduire les délais et les fréquence des cold starts.

Simplicité et efficacité d'un modèle unique pour les deux problèmes Un autre avantage majeur de notre processus mis en place est sa capacité à résoudre plusieurs problèmes avec un seul modèle. Contrairement aux approches de Kumari et al. [3] et Vahidinia et al. [4], qui nécessitent deux modèles distincts, et à celle de Sharhad et al. [12], qui en nécessite trois, notre approche utilise un modèle unique pour traiter à la fois les problèmes de délai de cold start et de fréquence de cold start. Cela simplifie grandement le processus de déploiement et de maintenance, tout en réduisant les coûts de calcul et de stockage. La flexibilité et la modularité des Transformers permettent d'adapter facilement le modèle à différentes tâches sans avoir à recourir à des architectures complexes et multiples.

Preuve de l'efficacité sur des schémas d'invocations réels Notre modèle Transformers a prouvé son efficacité en production, en utilisant des schémas d'invocations réels issus du dataset Azure Function 2019 [45]. Contrairement aux travaux antérieurs de Vahidinia [4], ou encore de Akash [38] qui se sont basés sur des scénarios générés de manière aléatoire et limités aux hypothèses des auteurs, notre approche s'appuie sur des données réelles et variées. Cette approche démontre non seulement la robustesse de notre modèle, mais aussi sa capacité à gérer efficacement les variations imprévisibles des demandes dans un environnement de production réel. Les résultats obtenus montrent

une amélioration significative par rapport aux méthodes précédentes, tant en termes de réduction des délais de cold start que de fréquence de cold starts.

Pertinence de la méthode L'échantillon utilisé dans notre étude est nettement supérieur et plus varié que ceux utilisés dans les travaux antérieurs. Grâce à la clusterisation du grand dataset Azure Function 2019, nous avons pu sélectionner des sous-datasets représentatifs de différents comportements utilisateurs. Chaque sous-dataset exprime un comportement distinct, mais reste représentatif de l'ensemble du dataset d'où il est tiré, ce qui permet une évaluation plus précise et réaliste des performances de notre modèle. Cette approche garantit que les résultats obtenus sont non seulement fiables mais aussi applicables à une large gamme de scénarios en production.

Pour assurer une reproductibilité exacte de nos évaluations, les fonctions sélectionnées de manière aléatoire dans le vaste ensemble de données Azure peuvent être retrouvées grâce à leur identifiant anonymisé dans le champ "HashFunction" du grand ensemble Azure. Les datasets sélectionnés ainsi que leurs identifiants sont explicitement mentionnés dans le tableau 9.

TABLE IX – Correspondance entre les fonctions et les ensembles de données Azure

Dataset	Azure - HashFunction
dataset 1	d2b1493652b769a990ee452dd64e9408908a1a64262208975ef1d684643598cb
dataset 2 & 7	4868d57abebd2245f60c4b0dff77bb56d7c067beedd6f532b371ba737892563a
dataset 3	a2d02e50b6a378983b9aad011ab44bd75e7b46c41d62d3c0f246cd34fb05ea3
dataset 4	c157ec9e3f10d175f1c03eba15052784c0fd8080a66546a5dfdec93d316288ac
dataset 5 & 11	72d8f02f3f2985d7992c8197bf45984e5cb7610e9fa415e8422c2713ece8507a
dataset 6 & 12	5f9e5a49fe4db63e7f23981dfd48546d745532cfc9a31eb8be0a27328c64195
dataset 8	7c1f08532f5b0377a2b6aee604a6a63eacd325d5f0ae8d2d4d5a8e9cbf4565e2
dataset 9	adf7a08a361395a0628218dfce41e009094997399692f3b2be4eb6e439e93694
dataset 10	8c9ce0de516e04d52c78b54b7d51a5f5be406bdcc2e61d19d18646b064488947d
dataset 13	7b8d0e2f25f1b0847ef396aef3fd18f282d6f7d8182e3fcd87e29e959400b72e
dataset 14	317de9def6a882c72e70d8310d3d49d2f7601f807db59ef66d77b4cd9cb8a374
dataset 15	104a9c6daea2257d92786b81b2dab0ef0ddb24f46350b80dbeb444005812b968
dataset 16	96cba3579b1898979f9d153a18a535d79529b502f3063a6464022cbcf9f57a21
dataset 17	cefe329b7f8b271075030228e0ff8fd1b1944cc223d06e9e507b95ab5b960ab7
dataset 18	3c96e7f2ba94ef190469bdf7ddd041de1affbf45dac7cbac65cf26f2197f0311

IV.5. Conclusion

Notre travail apporte plusieurs contributions significatives dans la gestion des cold starts dans les FaaS. D'abord, nous avons démontré l'efficacité des modèles Transformers, en particulier le modèle de Transformers vanilla adapté aux séries

temporelles, dans la réduction des délais et fréquences de cold starts. En utilisant des données réelles, notre modèle a prouvé sa capacité à capturer les relations à long terme dans les séries temporelles de manière plus efficace que les modèles LSTM des approches précédentes. Ensuite, la clusterisation, utilisant l'algorithme DBSCAN, a permis de segmenter les séries temporelles en sous-datasets représentatifs, facilitant ainsi l'entraînement et l'évaluation des modèles. Cette méthode a amélioré la pertinence et la validité des résultats, en s'assurant que les tests étaient effectués sur des échantillons variés et représentatifs de scénarios réels. Enfin, notre approche a montré qu'un modèle unique peut être utilisé pour résoudre deux problèmes distincts, à savoir le délai de cold start et la fréquence de cold start, simplifiant ainsi le processus de déploiement et de maintenance. Cette approche contraste avec les travaux antérieurs qui nécessitaient plusieurs modèles, rendant notre méthode plus pratique et économique.

CONCLUSION GÉNÉRALE

CONTENTS

Problématique étudiée	88
Analyse critique des résultats obtenus	88
Perspectives	89

Problématique étudiée

Tout au long de notre recherche, nous avons investigué deux problématiques clés dans les architectures Function-as-a-Service (FaaS) : la prédiction des charges de travail et l'atténuation du cold start. Ces deux aspects sont fondamentaux pour optimiser les performances et l'efficacité des systèmes FaaS, particulièrement dans le contexte croissant de l'adoption du cloud computing. Pour la prédiction des charges de travail, notre objectif était de développer un modèle capable de prévoir avec précision les variations des charges afin d'améliorer la gestion des ressources et de réduire les coûts opérationnels. Nous avons introduit un modèle hybride basé sur les réseaux de neurones convolutifs (CNN) et les mémoires à long court terme (LSTM). Concernant l'atténuation du cold start, nous avons visé à minimiser le temps de latence lors de l'activation initiale des fonctions. Pour ce faire, nous avons proposé une méthode utilisant les Transformers, une architecture avancée de deep learning, pour anticiper et réduire ces délais.

Analyse critique des résultats obtenus

Les contributions de notre recherche ont apporté des résultats significatifs. Concernant la prédiction des charges de travail, le modèle CNN-LSTM développé a montré une capacité notable à prédire les charges de travail avec une précision amé-

liorée. En comparaison avec des modèles de la littérature, notre modèle a réduit de manière significative le Root Mean Square Error (RMSE), obtenant une performance de 4,10 contre 6,96. L'intégration des couches convolutionnelles (CNN) avec des couches LSTM a permis une extraction efficace des caractéristiques locales tout en capturant les dépendances temporelles, ce qui a augmenté la précision des prédictions de charge de travail. Cette combinaison a démontré son efficacité en améliorant la capacité du modèle à anticiper les variations de charge, ce qui est crucial pour une gestion optimale des ressources dans les environnements FaaS.

Pour atténuer les cold starts, notre approche basée sur les Transformers a permis de réduire significativement les délais de démarrage des fonctions. Les Transformers, en anticipant les événements de cold start, ont surpassé les modèles LSTM traditionnels en termes de précision de prédiction et de réduction des temps de démarrage. Nos évaluations sur diverses métriques telles que sMAPE, explained variance, RMSE, R2 Score, et spearman correlation ont montré que les Transformers sont particulièrement efficaces pour capturer les relations à long terme dans les séries temporelles. Une réduction de cold start allant jusqu'à 79% a pu être observée. De plus, l'utilisation de la méthode de clusterisation avec l'algorithme DBSCAN a permis de segmenter efficacement les séries temporelles en sous-ensembles représentatifs, facilitant l'entraînement des modèles et garantissant la pertinence des prédictions. Cette segmentation a amélioré la réactivité du système et réduit la fréquence des cold starts, contribuant ainsi à l'efficacité globale des systèmes FaaS.

Perspectives

Les résultats prometteurs de notre étude ouvrent plusieurs perspectives pour des recherches futures, on pourrait citer entre autres :

- Explorer davantage et adapter de nouvelles architectures de Transformers pour améliorer encore la réduction des cold starts, pour offrir de nouvelles perspectives pour la prédiction des charges de travail et l'atténuation du cold start.
- Intégrer des techniques de reinforcement learning pourrait permettre une optimisation dynamique des ressources cloud en temps réel. Ces techniques adaptatives pourraient améliorer la gestion proactive des ressources en répondant aux variations imprévisibles des charges de travail.
- Inclure des mécanismes plus sophistiqués pour la gestion des ressources permettrait de minimiser le gaspillage et d'assurer une allocation optimale.

Par exemple, des algorithmes de gestion de mémoire plus efficaces peuvent être développés pour équilibrer les performances et les coûts opérationnels.

- En incorporant des données provenant de diverses plateformes et des métriques d'utilisation des applications, nous pourrions améliorer la précision des prédictions et la réactivité des systèmes. Cela permettrait une adaptation plus fine des modèles aux différents contextes d'utilisation.

BIBLIOGRAPHIE

- [1] T. N. Nguyen, “Managing Cold-start in The Serverless Cloud with Temporal Convolutional Networks,” Apr. 2023,” doi:10.48550/arXiv.2304.00396, arXiv :2304.00396 [cs, eess]. [Online]. Available : <http://arxiv.org/abs/2304.00396>
- [2] N. T. Noble, Y. P. Dev, and C. T. Joseph, “Machine Learning Based Techniques for Workload Prediction in Serverless Environments,” in *2023 International Conference on Electrical, Electronics, Communication and Computers (ELEXCOM)*. Roorkee, India : IEEE, Aug. 2023. doi:10.1109/ELEXCOM58812.2023.10370421. ISBN 9798350305111 pp. 1–6. [Online]. Available : <https://ieeexplore.ieee.org/document/10370421/>
- [3] A. Kumari, B. Sahoo, and R. K. Behera, “Mitigating Cold-Start Delay using Warm-Start Containers in Serverless Platform,” in *2022 IEEE 19th India Council International Conference (INDICON)*, Nov. 2022, doi:10.1109/INDICON56171.2022.10040220, pp. 1–6, iSSN : 2325-9418.
- [4] P. Vahidinia, B. Farahani, and F. S. Aliee, “Mitigating Cold Start Problem in Serverless Computing : A Reinforcement Learning Approach,” *IEEE Internet of Things Journal*, vol. 10, no. 5, pp. 3917–3927, Mar. 2023, doi:10.1109/JIOT.2022.3165127, conference Name : IEEE Internet of Things Journal.
- [5] Y. Jadeja and K. Modi, “Cloud computing - concepts, architecture and challenges,” Mar. 2012,” doi:10.1109/ICCEET.2012.6203873.
- [6] B. Larras, “CMOS analog implementation of clique-based neural networks,” Ph.D. dissertation, Dec. 2015.
- [7] S. Ali, J. Li, Y. Pei, M. Aslam, Z. Shaukat, and M. Azeem, “An Effective and Improved CNN-ELM Classifier for Handwritten Digits Recognition and Classification,” *Symmetry*, vol. 12, p. 15, Oct. 2020, doi:10.3390/sym12101742.

- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention Is All You Need,” Aug. 2023,” [doi:10.48550/arXiv.1706.03762](https://doi.org/10.48550/arXiv.1706.03762), arXiv :1706.03762 [cs]. [Online]. Available : <http://arxiv.org/abs/1706.03762>
- [9] A. U. Gias and G. Casale, “COCOA : Cold Start Aware Capacity Planning for Function-as-a-Service Platforms,” Jul. 2020, arXiv :2007.01222 [cs]. [Online]. Available : <http://arxiv.org/abs/2007.01222>
- [10] S. Eismann, J. Scheuner, E. v. Eyk, M. Schwinger, J. Grohmann, N. Herbst, C. L. Abad, and A. Iosup, “The State of Serverless Applications : Collection, Characterization, and Community Consensus,” *IEEE Transactions on Software Engineering*, vol. 48, no. 10, pp. 4152–4166, Oct. 2022, [doi:10.1109/TSE.2021.3113940](https://doi.org/10.1109/TSE.2021.3113940), conference Name : IEEE Transactions on Software Engineering. [Online]. Available : <https://ieeexplore.ieee.org/document/9543531>
- [11] I. C. Udousoro, “Machine Learning : A Review,” *Semiconductor Science and Information Devices*, vol. 2, no. 2, pp. 5–14, Nov. 2020, [doi:10.30564/ssid.v2i2.1931](https://doi.org/10.30564/ssid.v2i2.1931), number : 2. [Online]. Available : <https://journals.bilpubgroup.com/index.php/ssid/article/view/1931>
- [12] M. Shahrad, R. Fonseca, Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, “Serverless in the Wild : Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider,” 2020. ISBN 978-1-939133-14-4 pp. 205–218. [Online]. Available : <https://www.usenix.org/conference/atc20/presentation/shahrad>
- [13] M. Golec, G. K. Walia, M. Kumar, F. Cuadrado, S. S. Gill, and S. Uhlig, “Cold Start Latency in Serverless Computing : A Systematic Review, Taxonomy, and Future Directions,” Oct. 2023,” [doi:10.48550/arXiv.2310.08437](https://doi.org/10.48550/arXiv.2310.08437), arXiv :2310.08437 [cs]. [Online]. Available : <http://arxiv.org/abs/2310.08437>
- [14] L. Vaquero, L. Roderio-Merino, J. Caceres, and M. Lindner, “A Break in the Clouds : Towards a Cloud Definition,” *Computer Communication Review*, vol. 39, pp. 50–55, Jan. 2009, [doi:10.1145/1496091.1496100](https://doi.org/10.1145/1496091.1496100).
- [15] P. M. Mell and T. Grance, “The NIST Definition of Cloud Computing,” *NIST*, Sep. 2011, last Modified : 2018-11-10T10 :11-05 :00 Publisher : Peter M. Mell, Timothy Grance. [Online]. Available : <https://www.nist.gov/publications/nist-definition-cloud-computing>

- [16] A. Chinthas, “A Comparative Study of SaaS, PaaS and IaaS in Cloud Computing.” [Online]. Available : https://www.academia.edu/29455291/A_Comparative_Study_of_SaaS_PaaS_and_IaaS_in_Cloud_Computing
- [17] A. A.-D. R. Arpaci-Dusseau, *Operating Systems : Three Easy Pieces*.
- [18] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kubernetes,” *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, Apr. 2016, doi:10.1145/2890784. [Online]. Available : <https://dl.acm.org/doi/10.1145/2890784>
- [19] P. Castro, V. Isahagian, V. Muthusamy, and A. Slominski, “The rise of serverless computing,” *Communications of the ACM*, vol. 62, pp. 44–54, Nov. 2019, doi:10.1145/3368454.
- [20] “The CNCF takes steps toward serverless computing,” Feb. 2018. [Online]. Available : <https://www.cncf.io/blog/2018/02/14/cncf-takes-first-step-towards-serverless-computing/>
- [21] “Serverless Architectures.” [Online]. Available : <https://martinfowler.com/articles/serverless.html>
- [22] G. D. Maayan, “Running Serverless in Production : 7 Best Practices for DevOps,” Feb. 2023. [Online]. Available : <https://devops.com/running-serverless-in-production-7-best-practices-for-devops/>
- [23] “AWS | Lambda - Service PaaS de calculs distribués.” [Online]. Available : <https://aws.amazon.com/fr/lambda/>
- [24] I. H. Sarker, “Deep Learning : A Comprehensive Overview on Techniques, Taxonomy, Applications and Research Directions,” *SN Computer Science*, vol. 2, no. 6, p. 420, Aug. 2021, doi:10.1007/s42979-021-00815-1. [Online]. Available : <https://doi.org/10.1007/s42979-021-00815-1>
- [25] Z. Al Qadi, *Artificial neural networks (ANN)*, Jan. 2021.
- [26] A. Khan, A. Sohail, U. Zahoor, and A. S. Qureshi, “A survey of the recent architectures of deep convolutional neural networks,” *Artificial Intelligence Review*, vol. 53, no. 8, pp. 5455–5516, Dec. 2020, doi:10.1007/s10462-020-09825-6. [Online]. Available : <https://doi.org/10.1007/s10462-020-09825-6>
- [27] K. Jarrett, K. Kavukcuoglu, M. Ranzato, and Y. LeCun, “What is the best multi-stage architecture for object recognition?” in *2009 IEEE 12th International Conference on Computer Vision*, Sep. 2009,

- [doi:10.1109/ICCV.2009.5459469](https://doi.org/10.1109/ICCV.2009.5459469), pp. 2146–2153, iSSN : 2380-7504. [Online]. Available : <https://ieeexplore.ieee.org/document/5459469>
- [28] X. Zhang, R. Wang, T. Zhang, Y. Liu, and Y. Zha, “Short-Term Load Forecasting Using a Novel Deep Learning Framework,” *Energies*, vol. 11, no. 6, p. 1554, Jun. 2018, [doi:10.3390/en11061554](https://doi.org/10.3390/en11061554), number : 6 Publisher : Multidisciplinary Digital Publishing Institute. [Online]. Available : <https://www.mdpi.com/1996-1073/11/6/1554>
- [29] S. Indolia, A. K. Goswami, S. P. Mishra, and P. Asopa, “Conceptual Understanding of Convolutional Neural Network- A Deep Learning Approach,” *Procedia Computer Science*, vol. 132, pp. 679–688, Jan. 2018, [doi:10.1016/j.procs.2018.05.069](https://doi.org/10.1016/j.procs.2018.05.069). [Online]. Available : <https://www.sciencedirect.com/science/article/pii/S1877050918308019>
- [30] K. O’Shea and R. Nash, “An Introduction to Convolutional Neural Networks,” Dec. 2015,” [doi:10.48550/arXiv.1511.08458](https://doi.org/10.48550/arXiv.1511.08458), arXiv :1511.08458 [cs]. [Online]. Available : <http://arxiv.org/abs/1511.08458>
- [31] S. Hochreiter and J. Schmidhuber, “Long Short-term Memory,” *Neural computation*, vol. 9, pp. 1735–80, Dec. 1997, [doi:10.1162/neco.1997.9.8.1735](https://doi.org/10.1162/neco.1997.9.8.1735).
- [32] A. Jindal, M. Gerndt, M. Chadha, V. Podolskiy, and P. Chen, “Function Delivery Network : Extending Serverless Computing for Heterogeneous Platforms,” *Software : Practice and Experience*, vol. 51, no. 9, pp. 1936–1963, Sep. 2021, [doi:10.1002/spe.2966](https://doi.org/10.1002/spe.2966), arXiv :2102.02330 [cs]. [Online]. Available : <http://arxiv.org/abs/2102.02330>
- [33] J. Wen, Y. Liu, Z. Chen, J. Chen, and Y. Ma, “Characterizing Commodity Serverless Computing Platforms,” Feb. 2023,” [doi:10.48550/arXiv.2012.00992](https://doi.org/10.48550/arXiv.2012.00992), arXiv :2012.00992 [cs]. [Online]. Available : <http://arxiv.org/abs/2012.00992>
- [34] D. Barcelona-Pons and P. García-López, “Benchmarking Parallelism in FaaS Platforms,” *Future Generation Computer Systems*, vol. 124, pp. 268–284, Nov. 2021, [doi:10.1016/j.future.2021.06.005](https://doi.org/10.1016/j.future.2021.06.005), arXiv :2010.15032 [cs]. [Online]. Available : <http://arxiv.org/abs/2010.15032>
- [35] I. Matta, “Function-as-a-Service : From an Application Developer’s Perspective,” Jan. 2021. [Online]. Available : https://www.academia.edu/82130749/Function_as_a_Service_From_an_Application_Developer_s_Perspective
- [36] M. Steinbach, A. Jindal, M. Chadha, M. Gerndt, and S. Benedict, “TppFaaS : Modeling Serverless Functions Invocations via Tem-

- poral Point Processes,” *IEEE Access*, vol. PP, pp. 1–1, Jan. 2022, doi:10.1109/ACCESS.2022.3144078.
- [37] K. Djemame, M. Parker, and D. Datsev, “Open-source Serverless Architectures : an Evaluation of Apache OpenWhisk.” IEEE Computer Society, Dec. 2020.” doi:10.1109/UCC48980.2020.00052. ISBN 978-0-7381-2394-3 pp. 329–335. [Online]. Available : <https://www.computer.org/csdl/proceedings-article/ucc/2020/239400a329/1pZ11fOcXHa>
- [38] A. P. Jegannathan, R. Saha, and S. K. Addya, “A Time Series Forecasting Approach to Minimize Cold Start Time in Cloud-Serverless Platform,” in *2022 IEEE International Black Sea Conference on Communications and Networking (BlackSeaCom)*, Jun. 2022, doi:10.1109/BlackSeaCom54372.2022.9858271, pp. 325–330.
- [39] A. Mampage, S. Karunasekera, and R. Buyya, *A Deep Reinforcement Learning based Algorithm for Time and Cost Optimized Scaling of Serverless Applications*, Aug. 2023.
- [40] A. Kumari and B. Sahoo, “ACPM : adaptive container provisioning model to mitigate serverless cold-start,” *Cluster Computing*, vol. 27, pp. 1–28, May 2023, doi:10.1007/s10586-023-04016-8.
- [41] K. Tirkey, A. Kumari, S. Mohanty, and P. B. Sahoo, “A Novel Function Fusion Approach for Serverless Cold Start,” in *2023 International Conference on Communication, Circuits, and Systems (IC3S)*, May 2023, doi:10.1109/IC3S57698.2023.10169477, pp. 1–5.
- [42] P. Vahidinia, B. Farahani, and F. S. Aliee, “Cold Start in Serverless Computing : Current Trends and Mitigation Strategies,” in *2020 International Conference on Omni-layer Intelligent Systems (COINS)*, Aug. 2020, doi:10.1109/COINS49042.2020.9191377, pp. 1–7. [Online]. Available : <https://ieeexplore.ieee.org/document/9191377>
- [43] S. Lee, D. Yoon, S. Yeo, and S. Oh, “Mitigating Cold Start Problem in Serverless Computing with Function Fusion,” *Sensors*, vol. 21, no. 24, p. 8416, Jan. 2021, doi:10.3390/s21248416, number : 24 Publisher : Multidisciplinary Digital Publishing Institute. [Online]. Available : <https://www.mdpi.com/1424-8220/21/24/8416>
- [44] J. Manner, M. Endreß, T. Heckel, and G. Wirtz, “Cold Start Influencing Factors in Function as a Service,” in *2018 IEEE/ACM International Conference*

- on Utility and Cloud Computing Companion (UCC Companion), Dec. 2018, doi:10.1109/UCC-Companion.2018.00054, pp. 181–188.
- [45] “AzurePublicDataset/AzureFunctionsDataset2019.md at master · Azure/AzurePublicDataset.” [Online]. Available : <https://github.com/Azure/AzurePublicDataset/blob/master/AzureFunctionsDataset2019.md>
- [46] “apache/openwhisk,” Jun. 2024, original-date : 2016-02-18T21 :16 :19Z. [Online]. Available : <https://github.com/apache/openwhisk>
- [47] “Monitoring for serverless.” [Online]. Available : <https://dashbird.io/>
- [48] “lambda-warmer,” Nov. 2023. [Online]. Available : <https://www.npmjs.com/package/lambda-warmer>
- [49] X. Liu, J. Wen, Z. Chen, D. Li, J. Chen, Y. Liu, H. Wang, and X. Jin, “FaaSLight : General Application-level Cold-start Latency Optimization for Function-as-a-Service in Serverless Computing,” *Transactions ACM sur le génie logiciel et la méthodologie*, vol. 32, no. 5, pp. 119 :1–119 :29, Jul. 2023, doi:10.1145/3585007. [Online]. Available : <https://doi.org/10.1145/3585007>
- [50] “Cloud Functions.” [Online]. Available : <https://cloud.google.com/functions>
- [51] “Services de cloud computing | Microsoft Azure.” [Online]. Available : <https://azure.microsoft.com/fr-fr>
- [52] “open-lambda/open-lambda,” May 2024, original-date : 2016-06-17T18 :01 :08Z. [Online]. Available : <https://github.com/open-lambda/open-lambda>
- [53] S. Agarwal, M. A. Rodriguez, and R. Buyya, “A Reinforcement Learning Approach to Reduce Serverless Function Cold Start Frequency,” in *2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, May 2021, doi:10.1109/CCGrid51090.2021.00097, pp. 797–803.
- [54] —, “Reinforcement Learning (RL) Augmented Cold Start Frequency Reduction in Serverless Computing,” Aug. 2023. [Online]. Available : [https://www.semanticscholar.org/paper/Reinforcement-Learning-\(RL\)-Augmented-Cold-Start-in-Agarwal-Rodriguez/6576b0ada586b40c9ed1fa032c66370c25b44534](https://www.semanticscholar.org/paper/Reinforcement-Learning-(RL)-Augmented-Cold-Start-in-Agarwal-Rodriguez/6576b0ada586b40c9ed1fa032c66370c25b44534)
- [55] M. I. A. Efati, P. Hajek, M. Z. Abedin, R. U. Azad, M. A. Jaber, S. Aditya, and M. K. Hassan, “Deep-learning model using hybrid adaptive trend

- estimated series for modelling and forecasting sales,” *Annals of Operations Research*, Jul. 2022, doi:10.1007/s10479-022-04838-6. [Online]. Available : <https://doi.org/10.1007/s10479-022-04838-6>
- [56] I. E. Livieris, E. Pintelas, and P. Pintelas, “A CNN–LSTM model for gold price time-series forecasting,” *Neural Computing and Applications*, vol. 32, no. 23, pp. 17 351–17 360, Dec. 2020, doi:10.1007/s00521-020-04867-x. [Online]. Available : <https://doi.org/10.1007/s00521-020-04867-x>
- [57] K. Team, “Keras documentation : Conv1D layer.” [Online]. Available : https://keras.io/api/layers/convolution_layers/convolution1d/
- [58] L. Zhao, B. Cheng, and J. Chen, “A Hybrid Time Series Model based on Dilated Conv1D and LSTM with Applications to PM2.5 Forecasting,” *Aust. J. Intell. Inf. Process. Syst.*, 2019. [Online]. Available : <https://www.semanticscholar.org/paper/A-Hybrid-Time-Series-Model-based-on-Dilated-Conv1D-Zhao-Cheng/5ab4208e8eb941bf442e24cc71d604a9b2c65c1a>
- [59] W. Lu, J. Li, Y. Li, A. Sun, and J. Wang, “A CNN-LSTM-Based Model to Forecast Stock Prices,” *Complexity*, vol. 2020, no. 1, p. 6622927, 2020, doi:10.1155/2020/6622927, _eprint : <https://onlinelibrary.wiley.com/doi/pdf/10.1155/2020/6622927>. [Online]. Available : <https://onlinelibrary.wiley.com/doi/abs/10.1155/2020/6622927>
- [60] H. Xie, L. Zhang, and C. P. Lim, “Evolving CNN-LSTM Models for Time Series Prediction Using Enhanced Grey Wolf Optimizer,” *IEEE Access*, vol. 8, pp. 161 519–161 541, 2020, doi:10.1109/ACCESS.2020.3021527, publisher : IEEE. [Online]. Available : https://journals.scholarsportal.info/details/21693536/v8inone/161519_ecmftspuegwo.xml
- [61] K. Rasul, “kashif/pytorch-transformer-ts,” Jun. 2024, original-date : 2022-03-21T15 :40 :42Z. [Online]. Available : <https://github.com/kashif/pytorch-transformer-ts>
- [62] “blog/time-series-transformers.md at main · huggingface/blog.” [Online]. Available : <https://github.com/huggingface/blog/blob/main/time-series-transformers.md>
- [63] D. Chicco, M. J. Warrens, and G. Jurman, “The coefficient of determination R-squared is more informative than SMAPE, MAE, MAPE, MSE and RMSE in regression analysis evaluation,” *PeerJ Computer Science*, vol. 7,

- p. e623, Jul. 2021, doi:10.7717/peerj-cs.623, publisher : PeerJ Inc. [Online]. Available : <https://peerj.com/articles/cs-623>
- [64] V. Jayaraman, S. Parthasarathy, A. R. Lakshminarayanan, and H. K. Singh, “Predicting the Quantity of Municipal Solid Waste using XGBoost Model,” in *2021 Third International Conference on Inventive Research in Computing Applications (ICIRCA)*, Sep. 2021, doi:10.1109/ICIRCA51532.2021.9544094, pp. 148–152. [Online]. Available : <https://ieeexplore.ieee.org/document/9544094>
- [65] “NRMSE (Normalized Root Mean Squared Error),” Mar. 2023, section : 'IT'. [Online]. Available : <https://computer-nerd-zettelkasten.tistory.com/entry/NRMSE-Normalized-Root-Mean-Squared-Error>
- [66] R. Kassel, “Corrélations de Pearson et de Spearman : Tout comprendre,” Jun. 2023. [Online]. Available : <https://datascientest.com/correlations-de-pearson-et-de-spearman>